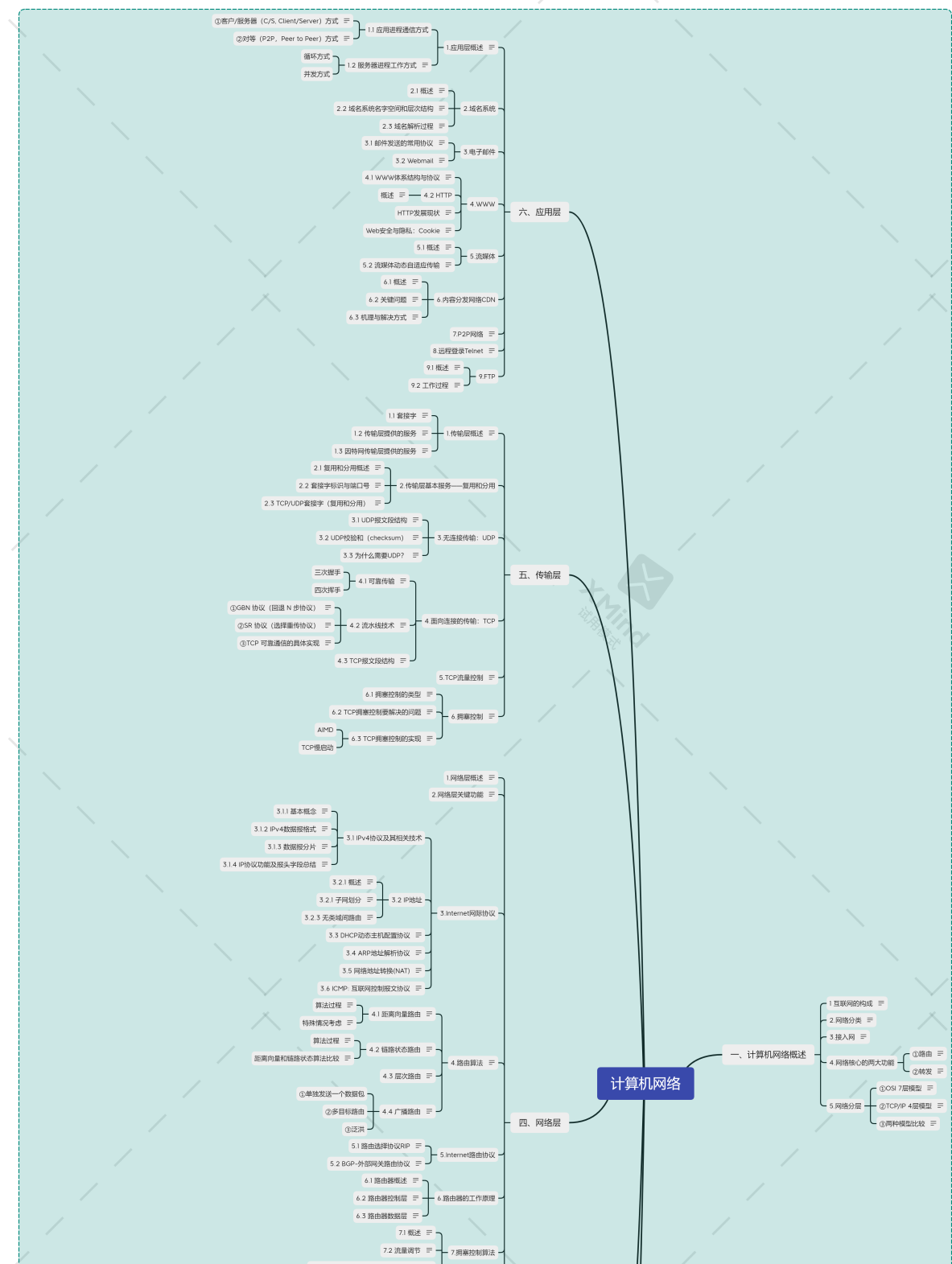


@[TOC]

前言

为了准备期末考试，同时也是为了之后复习方便，特对计算机网络的知识进行了整理。本篇内容大部分是来源于我们老师上课的ppt。而我根据自己的理解，将老师的PPT整理成博文的形式以便大家复习查阅，同时对于一些不是很清楚的地方，我去查阅了相关资料进行补充，当然也会有少部分个人看法来带其中来帮助大家理解。

应评论要求这里放上本文的思维导图以供大家更好的查看



3.接入网

接入网的用途

- 接入网的用途是将主机连接到边缘路由器上
- 边缘路由器是端系统Host去往任何其他远程端系统的路径上的第一台路由器

各种异构网络通过边缘路由器接入

接入网分类：

- 光纤到户FTTH
- 数字用户线DSL
- 同轴电缆
- 无线接入
- 企业和家庭网络

4.网络核心的两大功能

①路由

确定数据分组从源到目标所使用的路径（全局操作）

②转发

路由器或交换机将接收到的数据分组转发出去（即移动到该设备的某个输出接口）（本地操作）

5.网络分层

①OSI 7层模型



数据链路层 (Data Link Layer)

- 实现相邻 (Neighboring) 网络实体间的数据传输
- 成帧 (Framing) : 从物理层的比特流中提取出完整的帧
- 错误检测与纠正: 为提供可靠数据通信提供可能
- 物理地址 (MAC address) : 48位, 理论上唯一网络标识, 烧录在网卡, 不便更改
- 流量控制, 避免“淹没” (overwhelming) : 当快速的发送端遇上慢速的接收端, 接收端缓存溢出
- 共享信道上的访问控制 (MAC) : 同一个信道, 同时传输信号。如同: 同一个Wifi热点 (AP) 连接着多个无线用户 (手机), 则多个用户同时需要发送数据, 如何控制发送顺序?

网络层 (Network Layer)

- 将数据包跨越网络从源设备发送到目的设备 (host to host)
- 路由 (Routing) : 在网络中选取从源端到目的端转发路径, 常常会根据网络可达性动态选取最佳路径, 也可以使用静态路由
- 路由协议: 路由器之间交互路由信息所遵循的协议规范, 使得单个路由器能够获取网络的可达性等信息
- 服务质量 (QoS) 控制: 处理网络拥塞、负载均衡、准入控制、保障延迟
- 异构网络互联: 在异构编址和异构网络中路由寻址和转发

传输层 (Transport Layer)

- 将数据从源端口发送到目的端口（进程到进程）
- 网络层定位到一台主机（host），传输层的作用域具体到主机上的某一个进程
- 网络层的控制主要面向运营商，传输层为终端用户提供端到端的数据传输控制
- 两类模式：可靠的传输模式，或不可靠传输模式
- 可靠传输：可靠的端到端数据传输，适合于对通信质量有要求的应用场景，如文件传输等
- 不可靠传输：更快捷、更轻量的端到端数据传输，适合于对通信质量要求不高，对通信响应速度要求高的应用场景，如语音对话、视频会议等

会话层 (Session Layer)

- 利用传输层提供的服务，在应用程序之间建立和维持会话，并能使会话获得同步

表示层 (Presentation Layer)

- 关注所传递信息的语法和语义，管理数据的表示方法，传输的数据结构

应用层 (Application Layer)

- 通过应用层协议，提供应用程序便捷的网络服务调用

②TCP/IP 4层模型



③两种模型比较

OSI的失败：糟糕的时机、技术、实现、政策

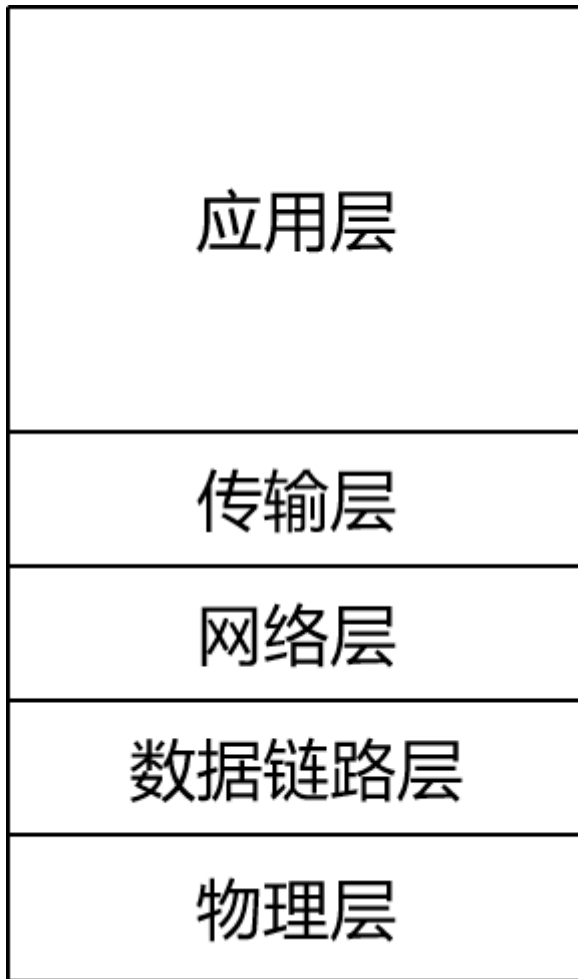
OSI模型的不足

- **从未真正被实现**
 - TCP/IP已成为事实标准，OSI缺少厂家支持
- **技术实现糟糕**
 - OSI分层欠缺技术考虑：会话层、表示层很少内容；数据链路层、网络层内容繁杂。模型和协议过于复杂
 - 分层间功能重复：差错控制、流量控制等在不同层反复出现
- **非技术因素**
 - TCP/IP实现为UNIX一部分，免费
 - OSI被认为是政府和机构的强加标准

TCP/IP模型的不足

- **核心概念未能体现**
 - 未明确区分服务、接口和协议等核心概念
- **不具备通用性**
 - 不适于描述TCP/IP之外的其它协议栈
- **混用接口与分层的设计**
 - 链路层和物理层一起被定义为网络接口层，而非真正意义上的分层
- **模型欠缺完整性**
 - 未包含物理层与数据链路层
 - 物理层与数据链路层是至关重要的部分

注：我们教材的是以下分层来讲述的



二、物理层

- 物理层考虑的是怎样才能**在连接各种计算机的传输媒体上传输数据比特流**，而不是指具体的传输媒体。
- 物理层的作用是要尽可能地**屏蔽掉不同传输媒体和通信手段的差异**。
- 用于物理层的协议也常称为**物理层规程** (procedure)。

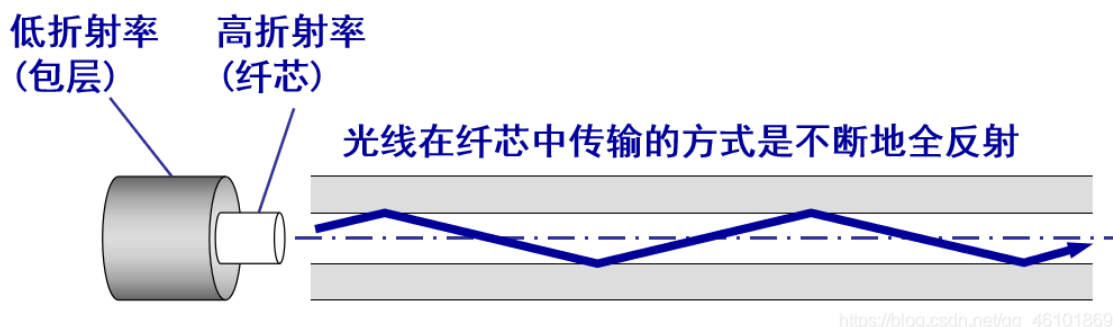
1.物理介质

①引导型介质

信号在固体介质中传播，例如铜、光纤、同轴电缆

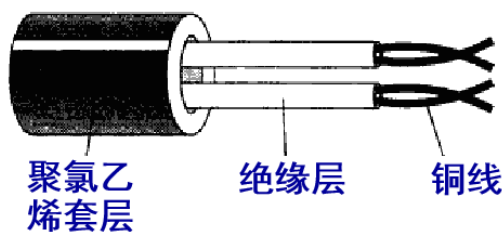
光纤

- 高速运行
高速点对点传输 (10-100 Gbps)
- 低错误率
中继器相距很远，对电磁噪声免疫

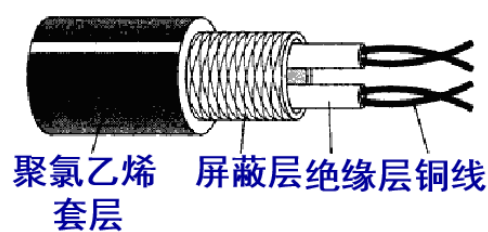


双绞线

- 两根绝缘铜线互相缠绕为一对
- 电话线为1对双绞线，网线为4对双绞线，广泛用于计算机网络(以太网) 双向传输
- 第5类: 100 Mbps~1 Gbps; 第6类: 10Gbps
- 传输距离一般为100米



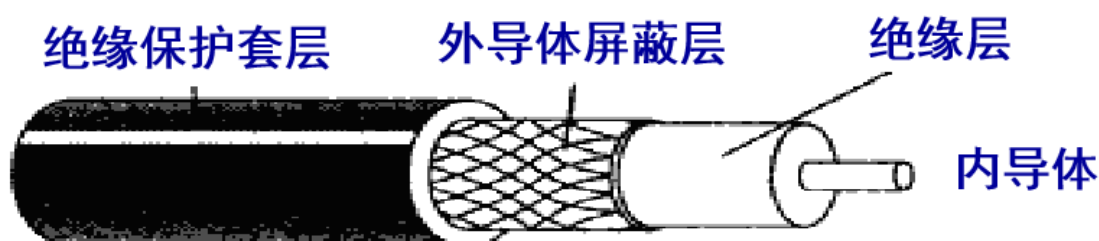
(a) 无屏蔽双绞线



(b) 屏蔽双绞线

同轴电缆

- 两根同心铜导线，双向传输
- 电缆上的多个频率通道
- 带宽可达100Mbps
- 传输距离一般为200米



同轴电缆的结构

https://blog.csdn.net/qq_46101869

②非引导型介质

信号自由传播，例如无线电（陆地无线电、卫星无线电信道）

无线电

- 电磁频谱中各种“波段”携带的信号
- 没有物理“电线”
- 不依赖介质的广播
- 半双工（发送方到接收方）

无线链路类型

- 无线局域网（WiFi）
10-100 Mbps；10米
- 广域（如3/4/5G蜂窝）
在~10公里范围内
- 蓝牙：短距离，有限速率
- 地面微波：点对点；45 Mbps
- 同步卫星：36000km高空，280毫秒的往返时延
- 低轨卫星：近地，但围绕地球高速运动，需要大量卫星才能覆盖地球

##

2.数据交换方式

①分组交换

分组交换采用把一个个小的数据包存储转发传输来实现数据交换。

主要的一些缺点：

- 1、不具有实时性。
- 2、存在延时。
- 3、会造成通信阻塞。
- 4、存在无用的重复数据。
- 5、会出现丢包的情况。

优点：

- 1、设计简单。
- 2、资源利用率很高。

②电路交换

电路连接的三个阶段：

- 1、建立连接。
- 2、数据传输。
- 3、释放连接。

优点：

- 1、传输速度快、高效。
- 2、实时。

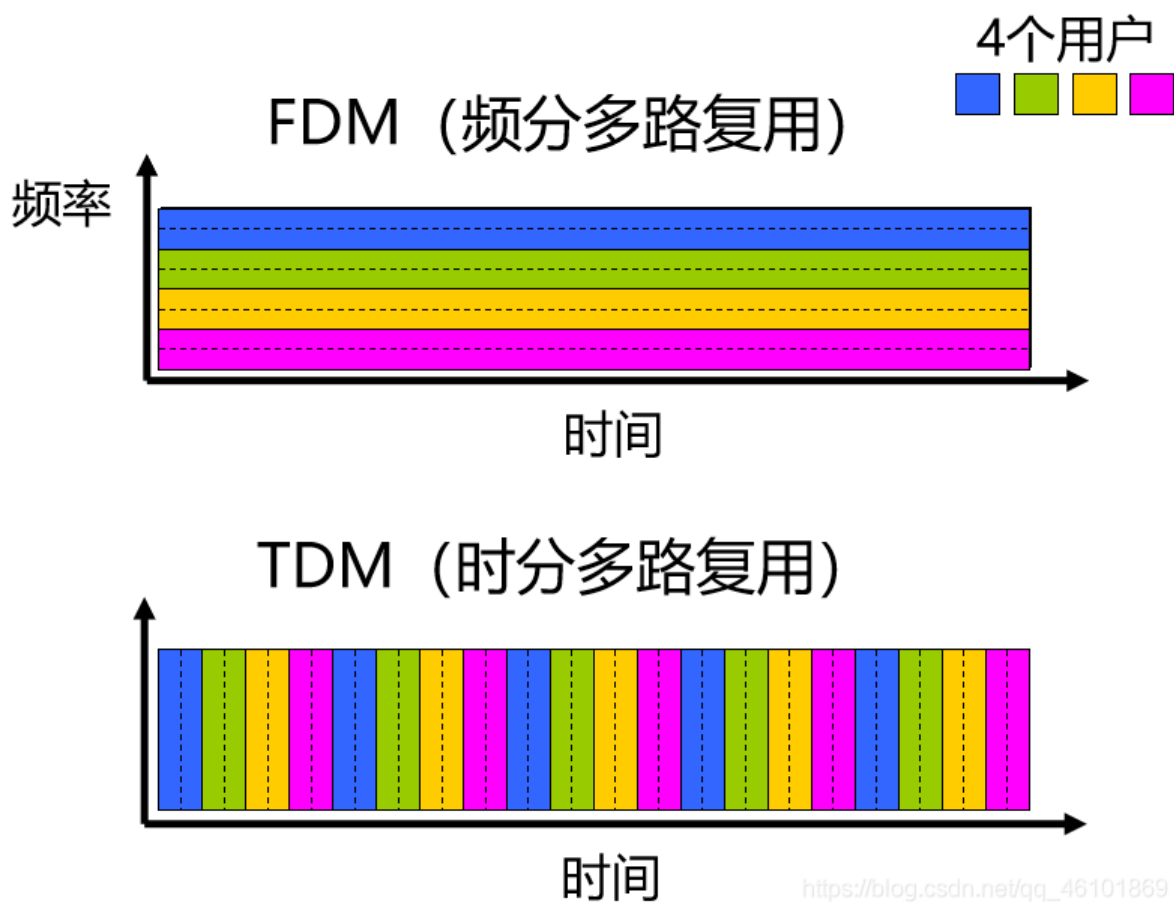
缺点：

- 1、资源利用率低。
- 2、新建连接需要占据一定的时间，甚至比通话的时间还长。

电路交换的多路复用

频分多路复用FDM

时分多路复用TDM



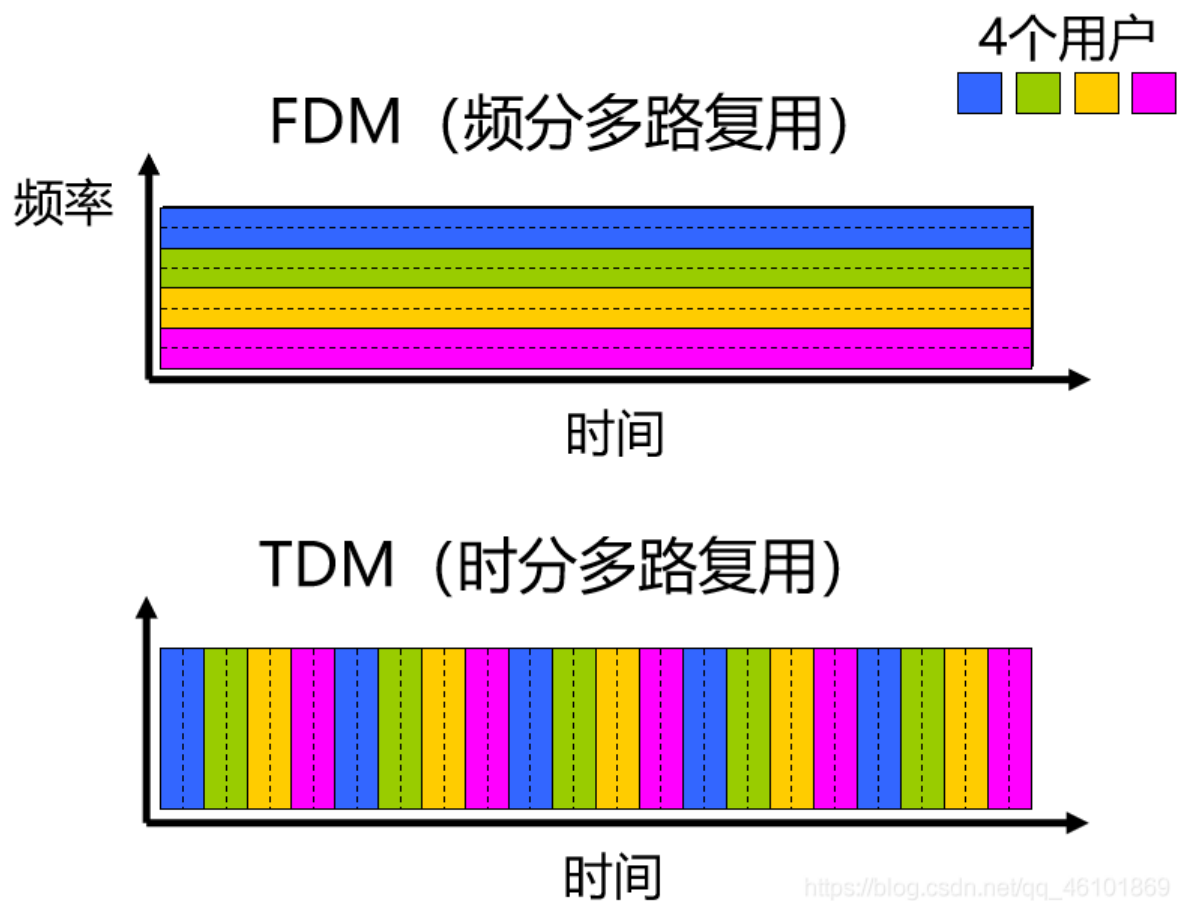
3.信道复用

复用 (multiplexing) 是通信技术中的基本概念。

它允许用户使用一个共享信道进行通信，降低成本，提高利用率。

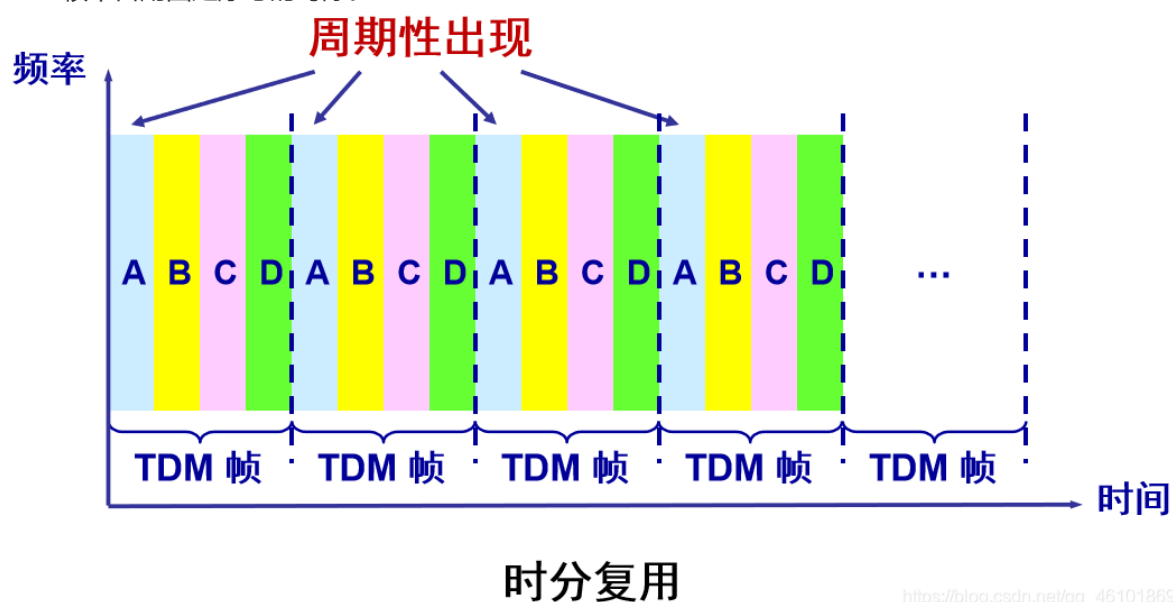
①频分复用

- 将整个带宽分为多份，用户在分配到一定的频带后，在通信过程中自始至终都占用这个频带。
- 频分复用的所有用户在同样的时间占用不同的带宽资源（请注意，这里的“带宽”是频率带宽而不是数据的发送速率）。



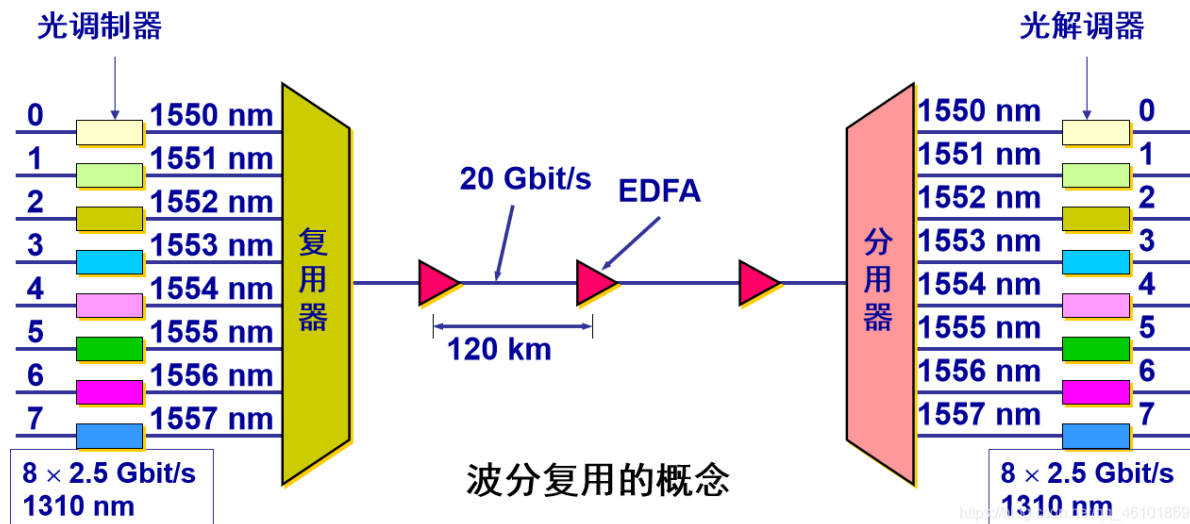
②时分复用

时分复用则是将时间划分为一段段等长的时分复用帧 (TDM 帧)。每一个时分复用的用户在每一个 TDM 帧中占用固定序号的时隙。



③波分复用

波分复用就是光的频分复用。使用一根光纤来同时传输多个光载波信号。



④码分复用

各用户使用经过特殊挑选的不同码型，因此彼此不会造成干扰。

三、数据链路层

1.功能（要解决的问题）

成帧（Framing）

将比特流划分成“帧”的主要目的是为了检测和纠正物理层在比特传输中可能出现的错误，数据链路层功能需借助“帧”的各个域来实现

差错控制（Error Control）

处理传输中出现的差错，如位错误、丢失等

流量控制（Flow Control）

确保发送方的发送速率，不大于接收方的处理速率，避免接收缓冲区溢出

2.数据链路层提供的服务

1.无确认 无连接 服务（Unacknowledged connectionless）

- 接收方不对收到的帧进行确认
- 适用场景：误码率低的可靠信道；实时通信；
- 网络实例：以太网

2.有确认 无连接 服务（Acknowledged connectionless）

- 每一帧都得到单独的确认
- 适用场景：不可靠的信道（无线信道）
- 网络实例：802.11

3.有确认 有连接 服务（Acknowledged connection-oriented）

- 适用场景：长延迟的不可靠信道

3.成帧（Framing）

3.1 要解决的关键问题：如何标识一个帧的开始？

- 接收方必须能从物理层接收的比特流中明确区分出一帧的开始和结束，这个问题被称为帧同步或帧定界
- 关键：选择何种定界符？定界符出现在数据部分如何处理？

3.2 成帧 (framing) 的方式

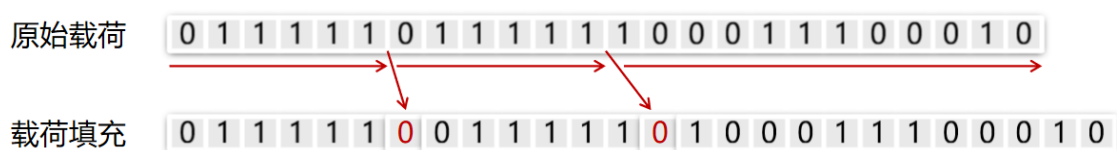
①带比特填充的定界符法

定界符：两个0比特之间，连续6个1比特，即01111110，0x7E



https://blog.csdn.net/qq_46101869

发送方检查有效载荷：若在有效载荷中出现连续5个1比特，则直接插入1个0比特



https://blog.csdn.net/qq_46101869

接收方的处理：

若出现连续5个1比特，

若下一比特为0，则为有效载荷，直接丢弃0比特；

若下一比特为1，则连同后一比特的0，构成定界符，一帧结束

②物理层编码违例

- 核心思想：**选择的定界符不会在数据部分出现**
- 4B/5B编码方案
4比特数据映射成5比特编码，剩余的一半码字（16个码字）未使用，可以用做帧定界符
例如：00110组合不包含在4B/5B编码中，可做帧定界符
- 前导码
存在很长的 前导码 (preamble)，可以用作定界符
例如：传统以太网、802.11
- 曼切斯特编码 / 差分曼切斯特编码
正常的信号在周期中间有跳变，持续的高电平（或低电平）为违例码，可以用作定界符
例如：802.5令牌环网

4.差错控制

4.1 背景

链路层存在的一个问题：**信道的噪声导致数据传输问题**

- 差错** (incorrect)：数据发生错误
- 丢失** (lost)：接收方未收到
- 乱序** (out of order)：先发后到，后发先到
- 重复** (repeatedly delivery)：一次发送，多次接收

解决方案：**差错检测与纠正、确认重传**

- 确认：接收方校验数据（差错校验），并给发送方应答，防止差错
- 定时器：发送方启动定时器，防止丢失

- 顺序号：接收方检查序号，防止乱序递交、重复递交

4.2 差错检验与纠正

目标

保证一定差错检测和纠错能力的前提下，如何减少冗余信息量？

考虑的问题

- 信道的特征和传输需求
- 冗余信息的计算方法、携带的冗余信息量
- 计算的复杂度等

两种主要策略

- 检错码 (error-detecting code)

在被发送的数据块中，包含一些冗余信息，但这些信息只能使接收方推断是否发生错误但不能推断哪位发生错误，接收方可以请求发送方重传数据主要用在高可靠、误码率较低的信道上，例如光纤链路偶尔发生的差错，可以通过重传解决差错问题

- 纠错码 (error-correcting code)

发送方在每个数据块中加入足够的冗余信息，使得接收方能够判断接收到的数据是否有错，并能纠正错误（定位出错的位置）主要用于错误发生比较频繁的信道上，如无线链路也经常用于物理层，以及更高层（例如，实时流媒体应用和内容分发）使用纠错码的技术通常称为前向纠错（FEC，Forward Error Correction）

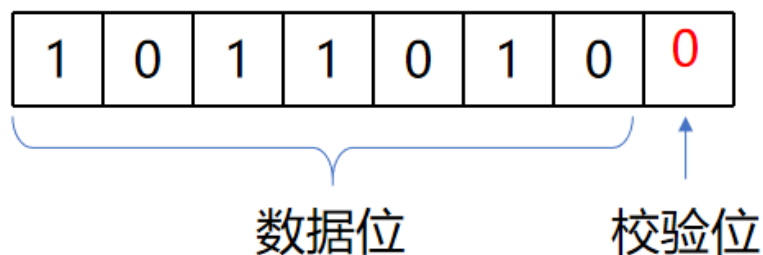
常用的检错码包括：

①奇偶检验 (Parity Check)

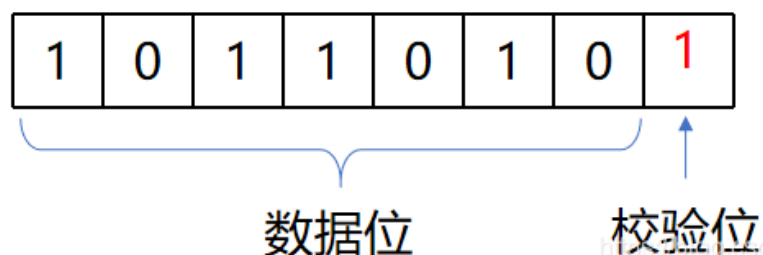
1位奇偶校验是最简单、最基础的检错码。

1位奇偶校验：增加1位校验位，可以检查奇数位错误。

- 偶校验：保证1的个数为偶数个，例如：



- 奇校验：保证1的个数为奇数个，例如：

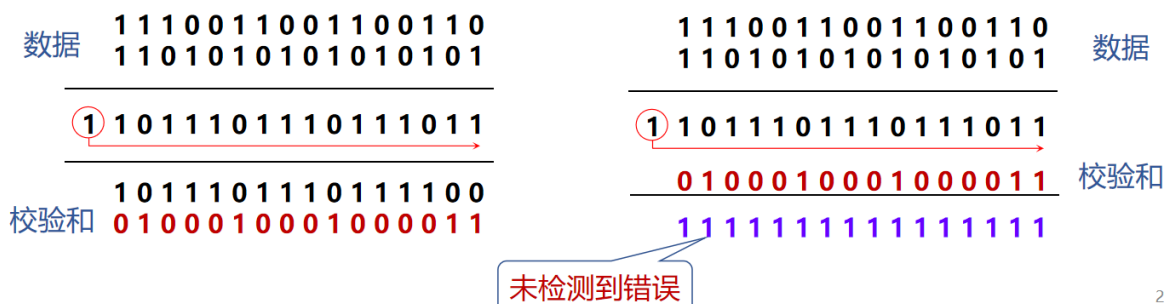


②校验和 (Checksum)

主要用于TCP/IP体系中的网络层和传输层

发送方：进行 16 位二进制补码求和运算，计算结果取反，随数据一同发送

接收方：进行 16 位二进制补码求和运算（包含校验和），结果非全1，则检测到错误



21

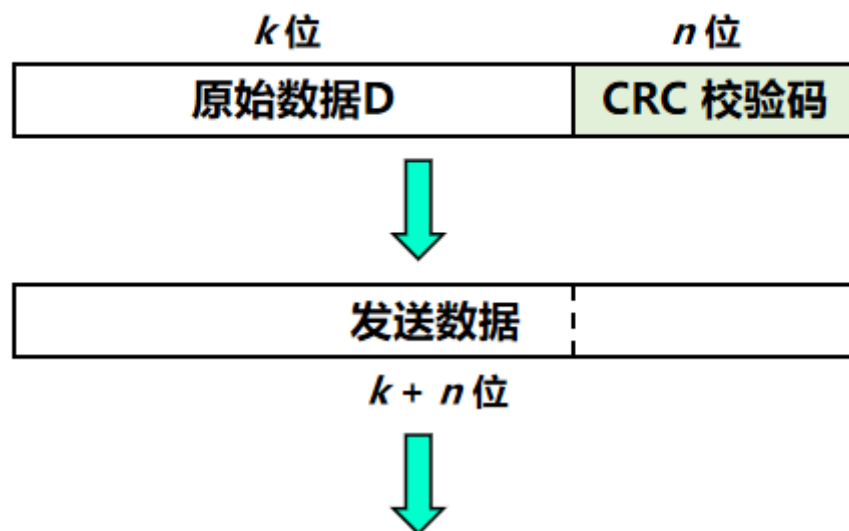
https://blog.csdn.net/qn_46101869

③循环冗余校验 (Cyclic Redundancy Check, CRC)

数据链路层广泛使用的校验方法

CRC校验码计算方法

- 设原始数据D为k位二进制位模式
- 如果要产生n位CRC校验码，事先选定一个n+1位二进制位模式G (称为生成多项式，收发双方提前商定)，G的最高位为1
- 将原始数据D乘以 2^n (相当于在D后面添加 n 个 0)，产生k+n位二进制位模式，用G对该位模式做模2除，得到余数R (n位，不足n位前面用0补齐) 即为CRC校验码

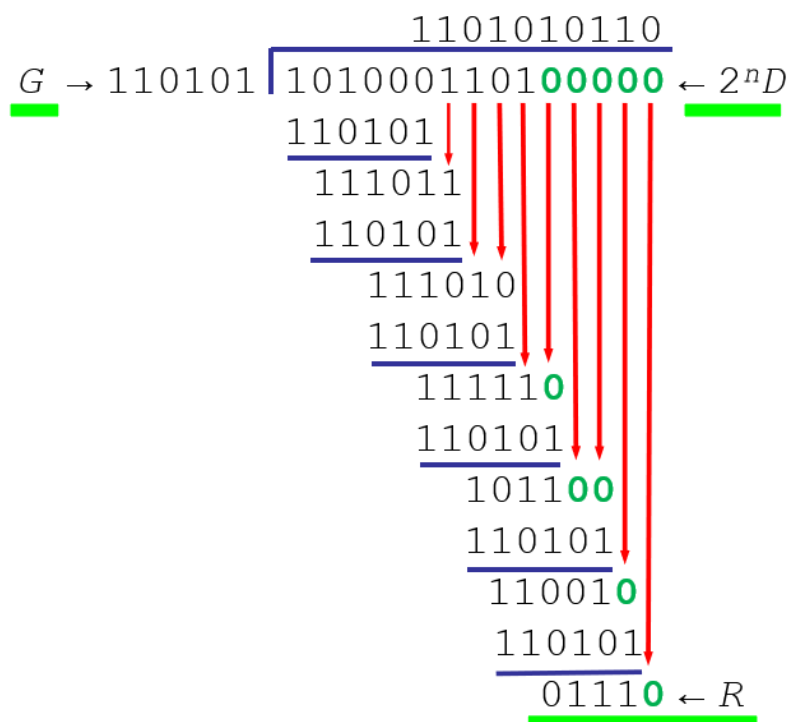


CRC校验能力：能检测出
少于n+1位的突发错误

https://blog.csdn.net/qn_46101869

CRC校验码计算示例

- $D = 1010001101$
- $n = 5$
- $G = 110101$ 或 $G = x^5 + x^4 + x^2 + 1$
- $R = 01110$
- 实际传输数据: 101000110101110



https://blog.csdn.net/qq_46101869

④汉明码

目标：以奇偶校验为基础，找到出错位置，提供**1位**纠错能力

- 给定 n 位待发送的数据，首先确定校验位的个数 k （根据 $2^k \geq k+n+1$ ）
- 确定校验位在码流中的位置， $2^i, i=0,1,2,\dots$ ，得到校验位 $P_1, P_2, P_4, P_8, \dots$
- 确定分组，即每个校验位分别负责哪些数据位
 - P_1 负责位置号的二进制符合 $XXXX1$ 形式的数据位，即 $1,3,5,7,9,\dots$
 - P_2 负责位置号的二进制符合 $XXX1X$ 形式的数据位，即 $2,3,6,7,10,\dots$
 - P_3 负责位置号的二进制符合 $XX1XX$ 形式的数据位，即 $4,5,6,7,12,13,14,15,\dots$
- 基于偶校验确定每组的校验位（0或1）

注：汉明码是采用奇偶校验的码。它采用了一种非常巧妙的方式，把这串数字分了组，通过分组校验来确定哪一位出现了错误。

实际的海明码编码的过程也并不复杂，我们通过用不同过的校验位，去匹配多个不同的数据组，确保任何一个数据位出错，都会产生一个多个校验码位出错的唯一组合。这样，在出错的时候，我们就可以反过来找到出错的数据位，并纠正过来。当只有一个校验码位出错的时候，我们就知道实际出错的是校验码位了。

5.流量控制

链路层存在的另一个问题：接收方的处理速率

- 接收方的接收缓冲区溢出

解决方案

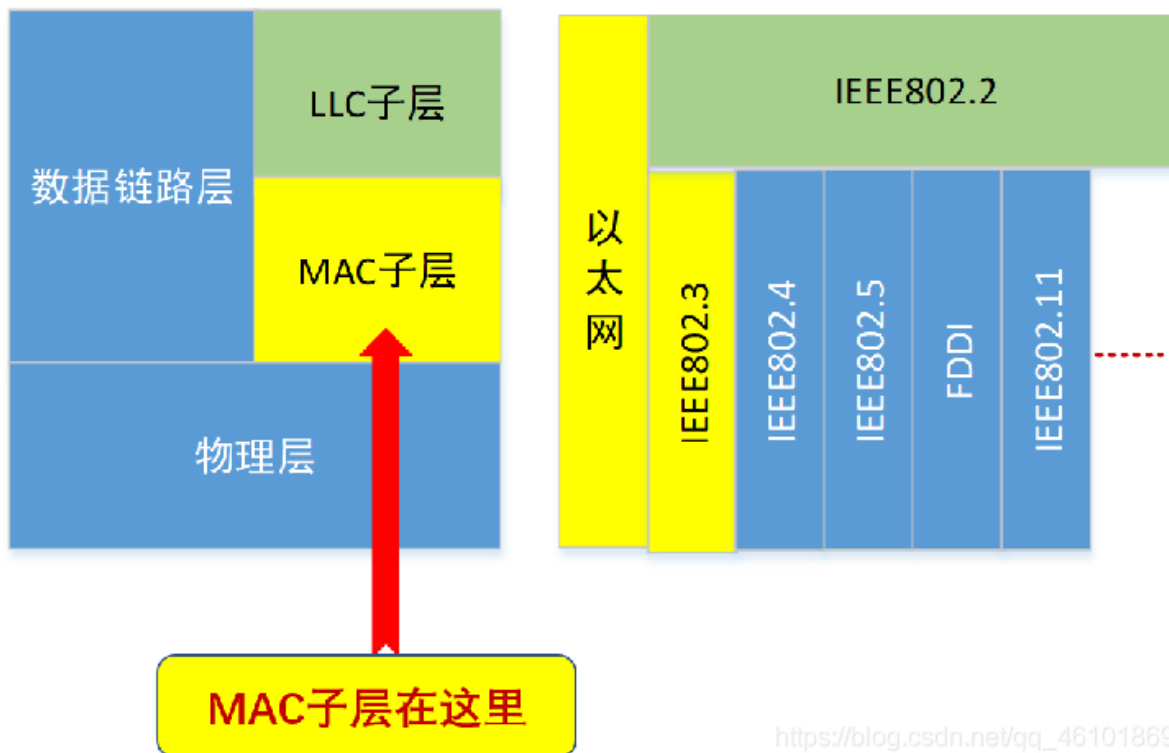
- 基于**反馈** (feedback-based) 的流量控制
接收方反馈，发送方调整发送速率
- 基于**速率** (rate-based) 的流量控制
发送方根据内建机制，自行限速

6.媒体接入控制 MAC (Medium Access Control)子层

数据链路层分为两个子层：

MAC子层：介质访问

LLC子层：承上启下（弱层）



https://blog.csdn.net/qq_46101869

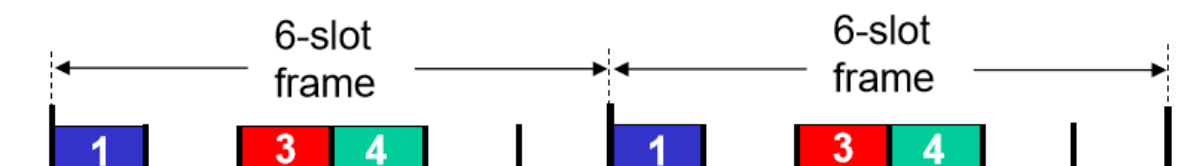
6.1 信道分配问题

① 时分多址接入-TDMA

TDMA: time division multiple access

- 按顺序依次接入并使用信道
- 每个用户使用固定且相同长度的时隙
- 某时隙轮到某用户使用，该用户没有数据要发送，则该时隙被闲置

例子: 6-user LAN, 1,3,4时隙有数据发送, 2,5,6时隙被闲置

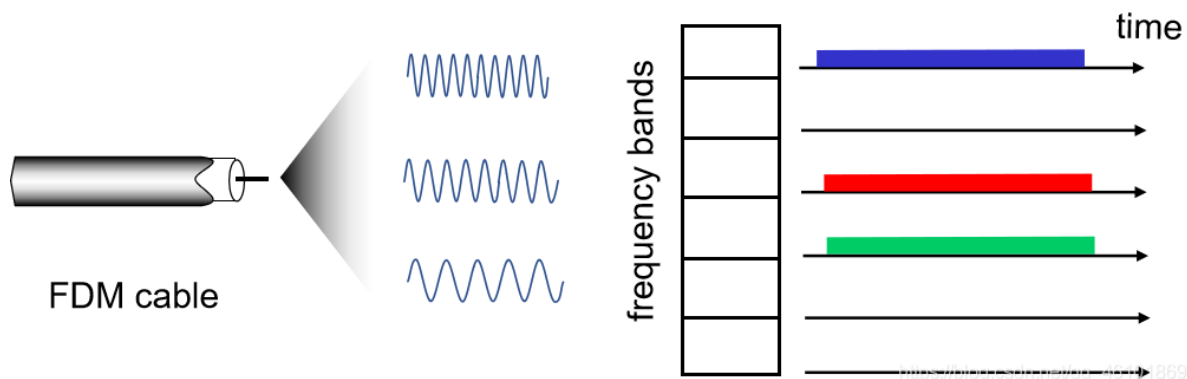


② 频分多址接入-FDMA

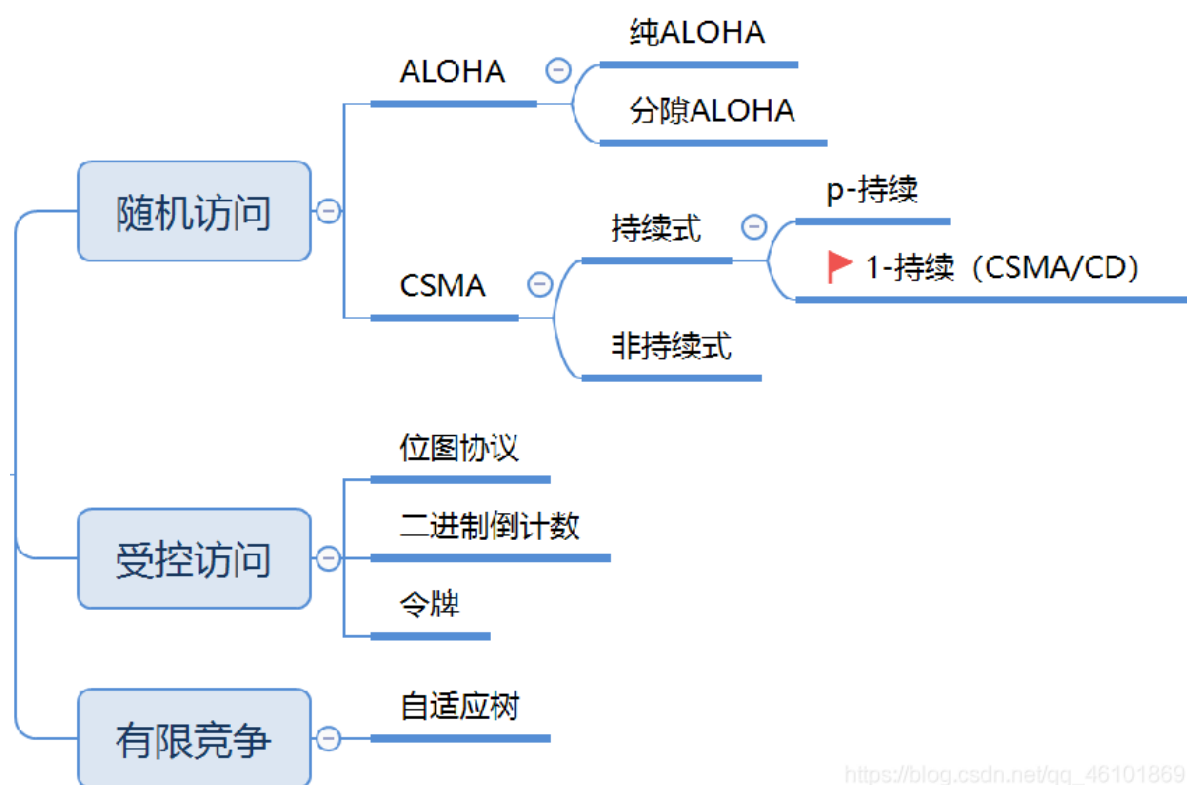
FDMA: frequency division multiple access

- 信道总频带被划分为多个相同宽度的子频带
- 每个用户占用一个子频带，不管用户是否有数据发送

例子: 6-user LAN, 1,3,4频带有数据发送, 2,5,6频带被闲置



6.2 多路访问协议



6.2.1 随机访问协议

特点: 冲突不可避免

① ALOHA

纯ALOHA协议

原理: 想发就发!

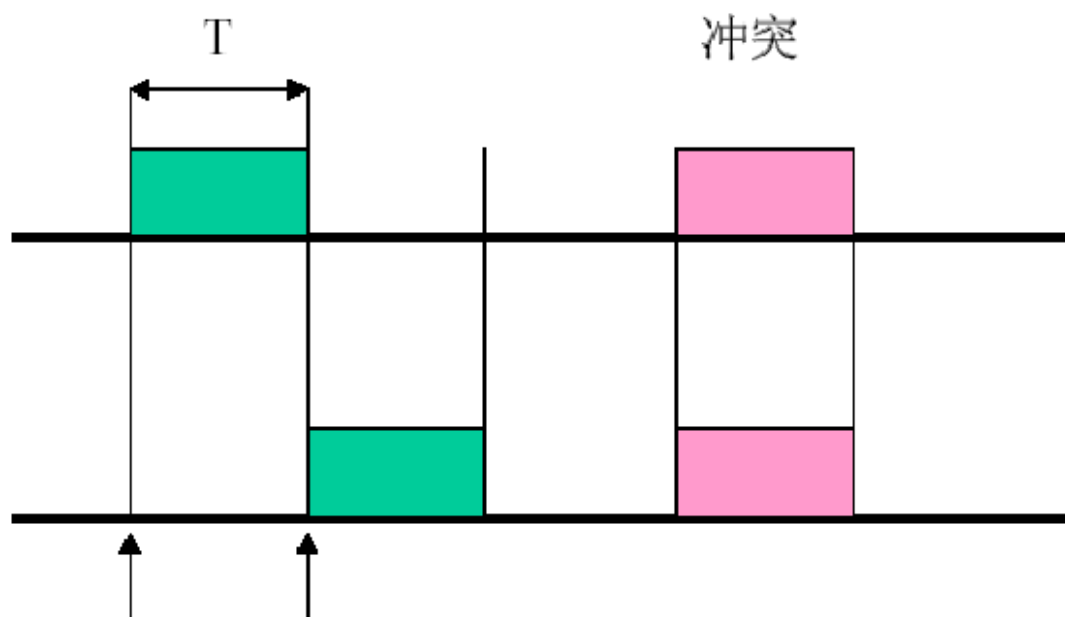
特点:

- 冲突: 两个或以上的帧
- 随时可能冲突
- 冲突的帧完全破坏
- 破坏了的帧要重传

分隙ALOHA

- 分隙ALOHA是把时间分成时隙 (时槽)
- 时隙的长度对应一帧的传输时间。

- 帧的发送必须在时隙的起点。
- 冲突只发生在时隙的起点



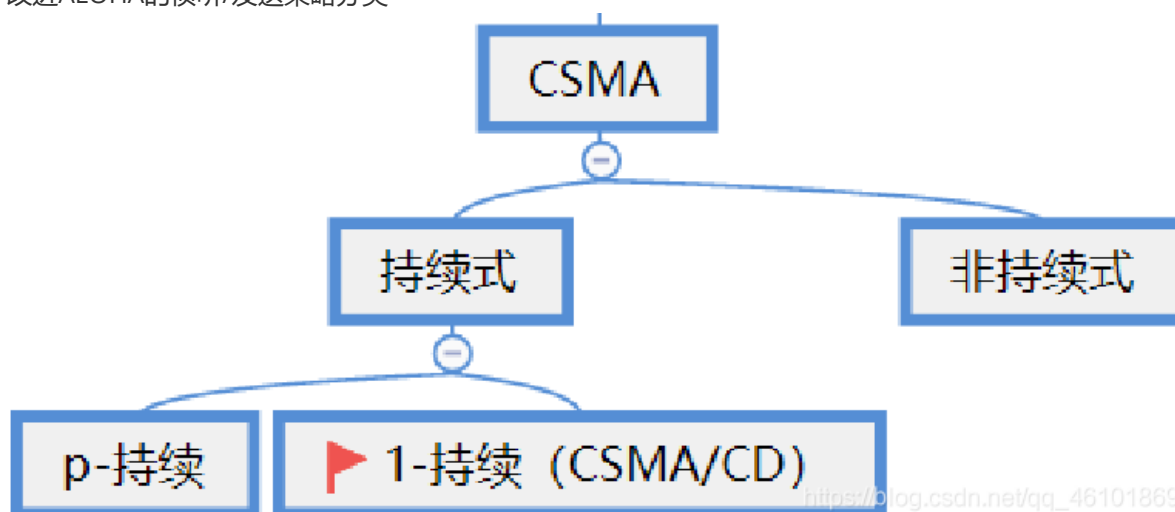
只能在时隙起点发送

https://blog.csdn.net/qq_46101869

②载波侦听多路访问协议CSMA

特点：“先听后发”

改进ALOHA的侦听/发送策略分类



https://blog.csdn.net/qq_46101869

非持续式CSMA

1.特点

- ①经侦听，如果介质空闲，开始发送
- ②如果介质忙，则等待一个随机分布的时间，然后重复步骤①

2.好处

等待一个随机时间可以减少再次碰撞冲突的可能性

3.缺点

等待时间内介质上如果没有数据传送，这段时间是浪费的

持续式CSMA

p-持续式CSMA

1.特点

- ①经侦听，如介质空闲，那么以 p 的概率发送，以 $(1-p)$ 的概率延迟一个时间单元发送
- ②如介质忙，持续侦听，一旦空闲重复①
- ③如果发送已推迟一个时间单元，再重复步骤①

1-持续式CSMA

1.特点

- ①经侦听，如介质空闲，则发送
- ②如介质忙，持续侦听，一旦空闲立即发送
- ③如果发生冲突，等待一个随机分布的时间再重复步骤①

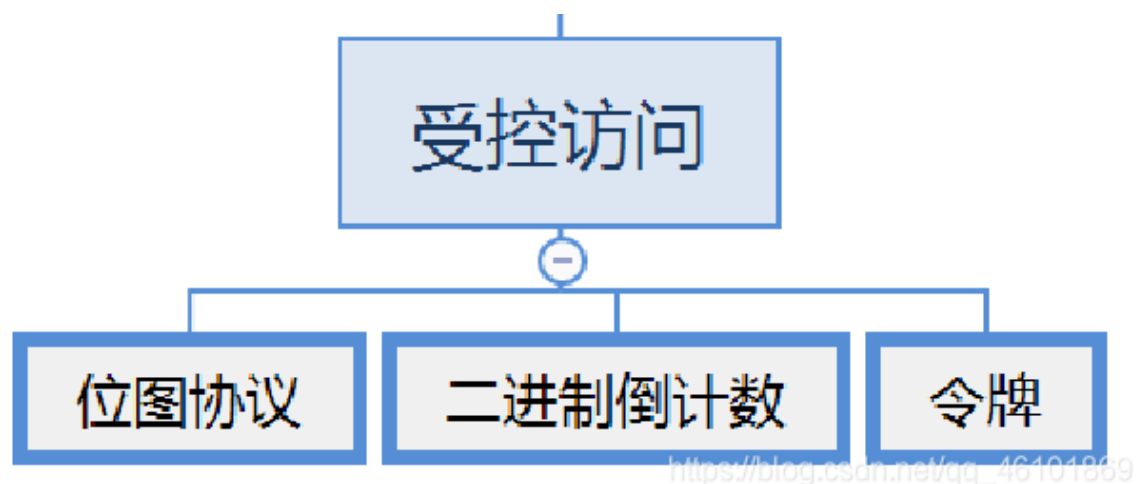
2.好处：持续式的延迟时间要少于非持续式

3.主要问题：如果两个以上的站等待发送，一旦介质空闲就一定会发生冲突

4.注意

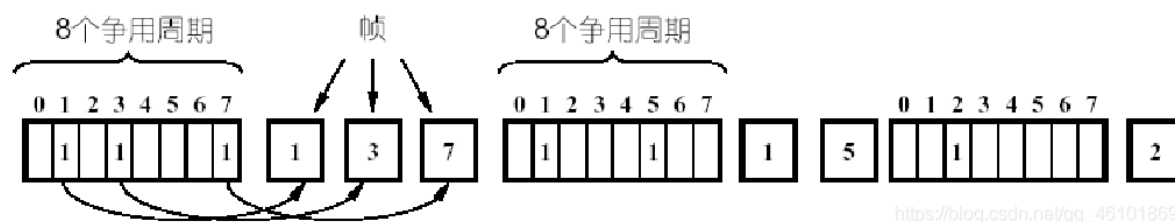
1-持续式是p-持续式的特例

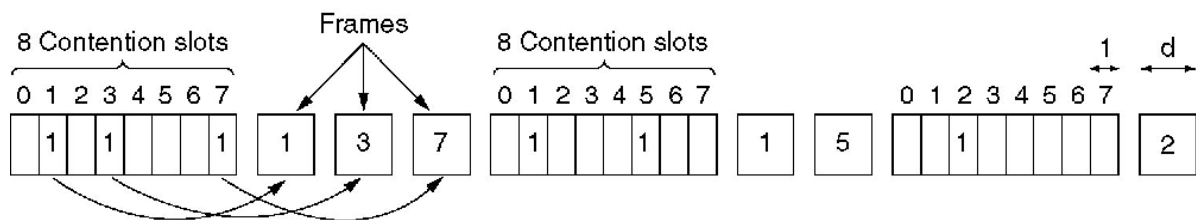
6.2.2 受控访问协议



特点：克服了冲突

①位图协议（预留协议）





竞争期：在自己的时槽内发送竞争比特

- 举手示意
- 资源预留

传输期：按序发送

- 明确的使用权，避免了冲突

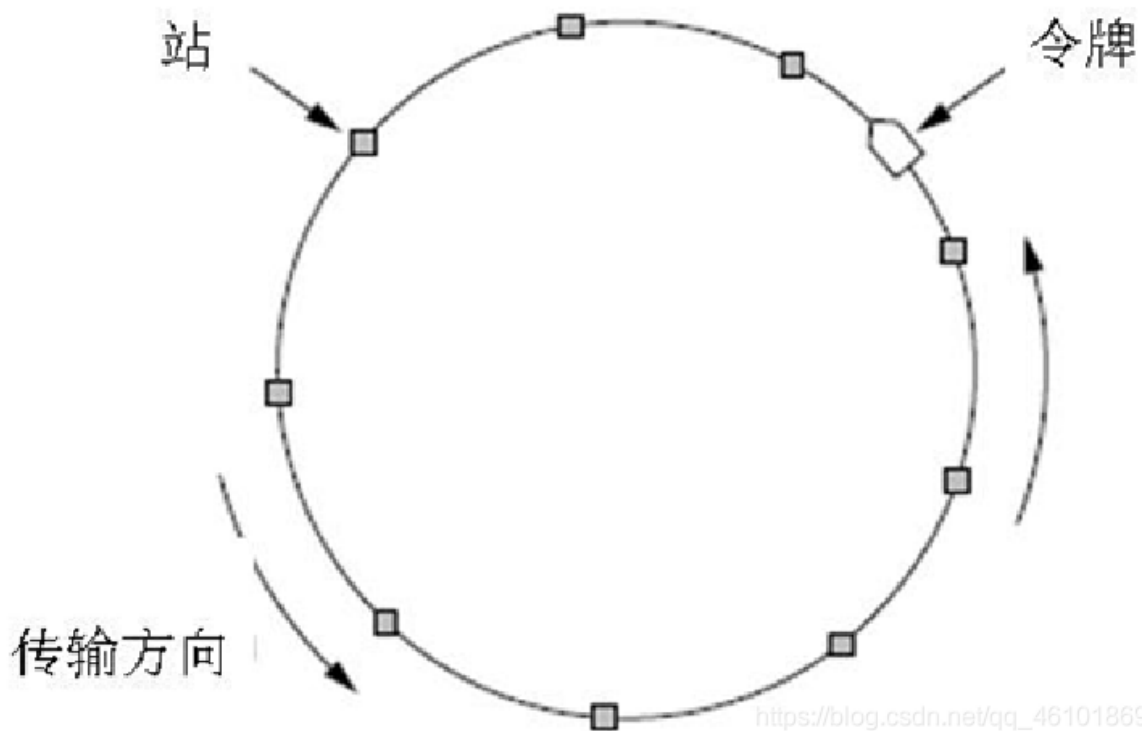
②令牌传递

令牌：发送权限

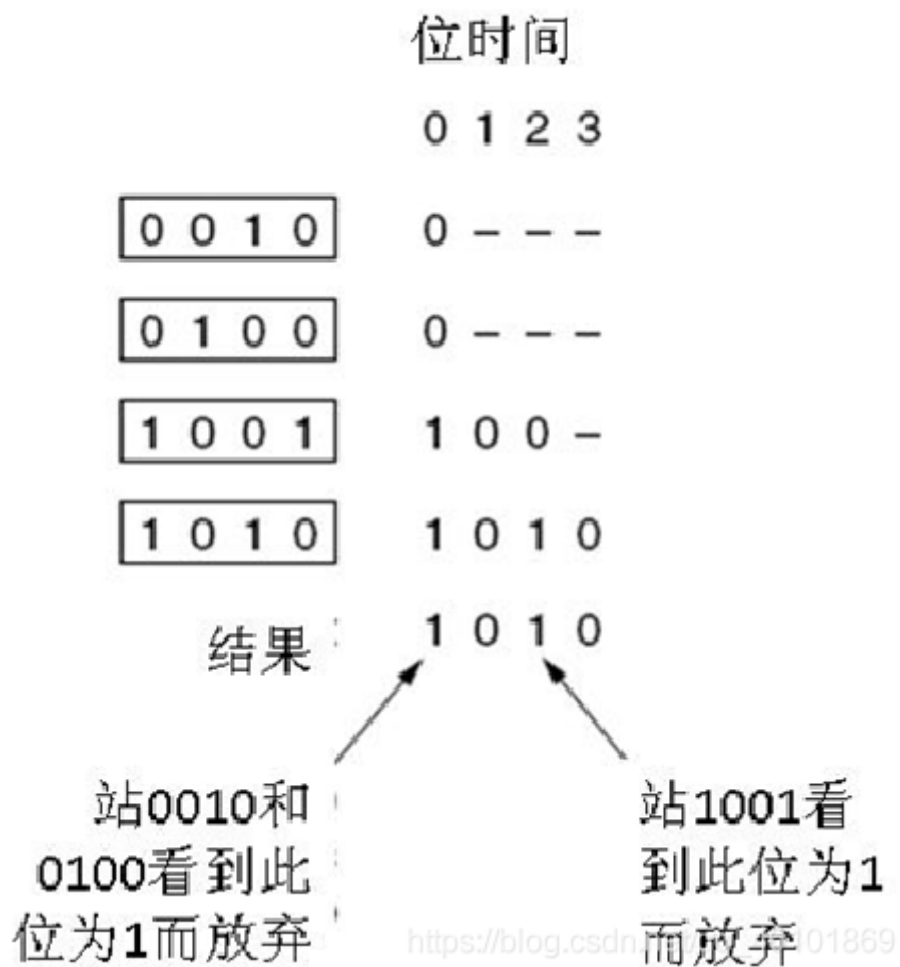
令牌的运行：发送工作站去抓取，获得发送权

- 除了环，令牌也可以运行在其它拓扑上，如令牌总线
- 发送的帧需要目的站或发送站将其从共享信道上去除；防止无限循环

缺点：令牌的维护代价



③二进制倒数计数协议



站点：编序号，序号长度相同

竞争期：有数据发送的站点从高序号到低序号排队，高者得到发送权

特点：高序号站点优先

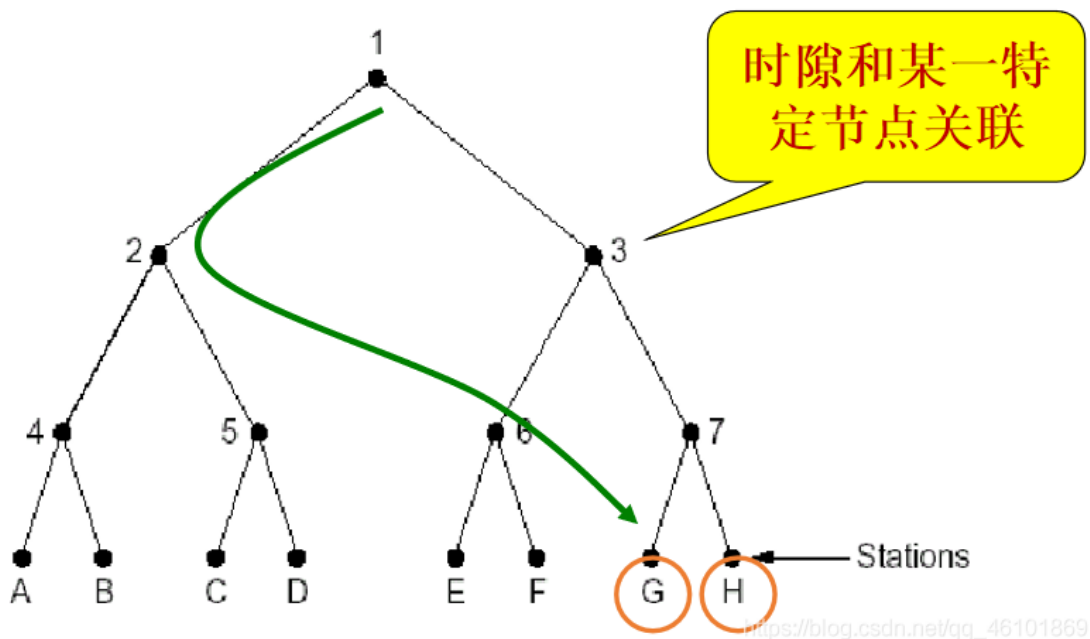
6.2.3 有限竞争协议

利用上述二者的优势

①自适应树搜索协议 (Adaptive Tree Walk Protocol)

- 在一次成功传输后的第一个竞争时隙，所有站点同时竞争。
- 如果只有一个站点申请，则获得信道。
- 否则在下一竞争时隙，有一半站点参与竞争（递归），下一时隙由另一半站点参与竞争

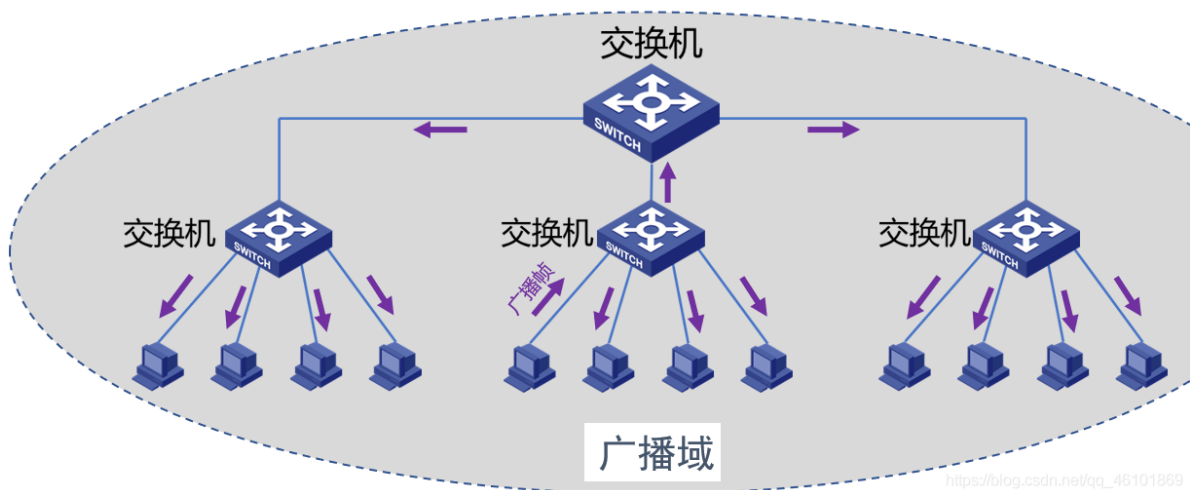
- 即所有站点构成一棵完全二叉树。



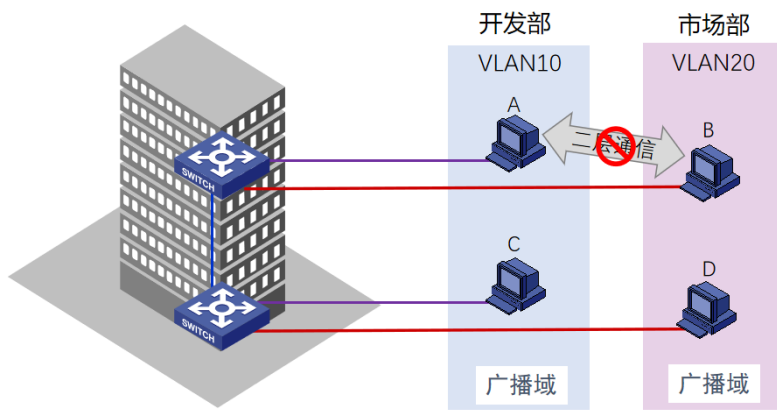
6.3虚拟局域网VLAN

广播域 (Broadcasting Domain)

- 广播域是广播帧能够到达的范围；
- 缺省情况下，交换机所有端口同属于一个广播域，无法隔离广播域；
- 广播帧在广播域中传播，占用资源，降低性能，且具有安全隐患。



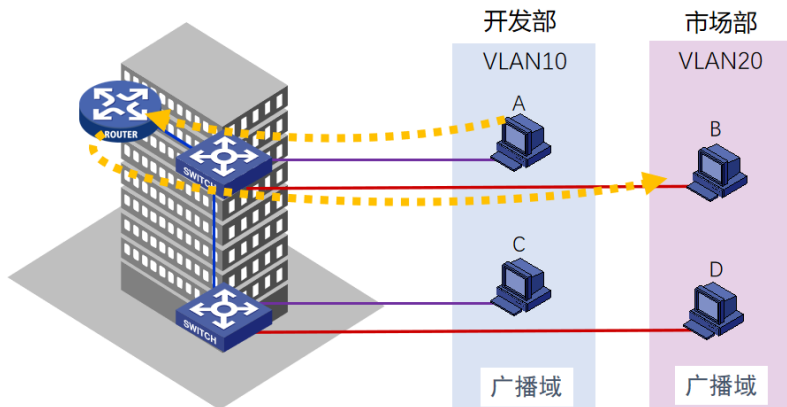
VLAN是一个在物理网络上根据用途，工作组、应用等来逻辑划分的局域网络，与用户的物理位置没有关系。



不同VLAN的成员
不能直接进行二层通信

https://blog.csdn.net/qn_46101869

通过路由器或三层交换机进行VLAN间路由，实现VLAN间通信。

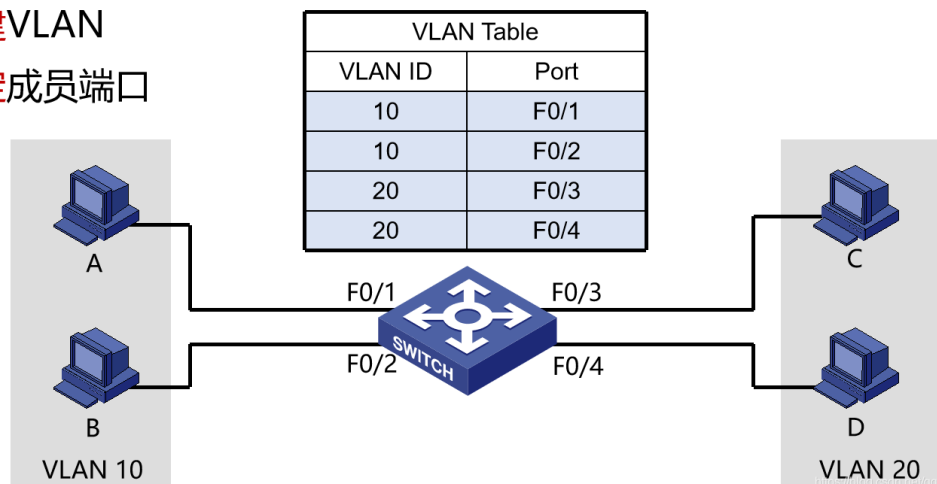


不同VLAN的成员通信
需要通过三层设备

https://blog.csdn.net/qn_46101869

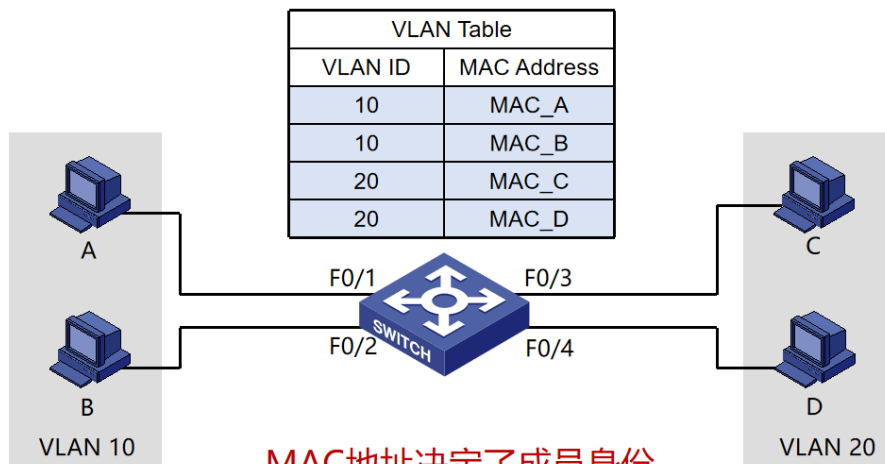
VLAN类型

- 基于端口的VLAN
 - 创建VLAN
 - 指定成员端口



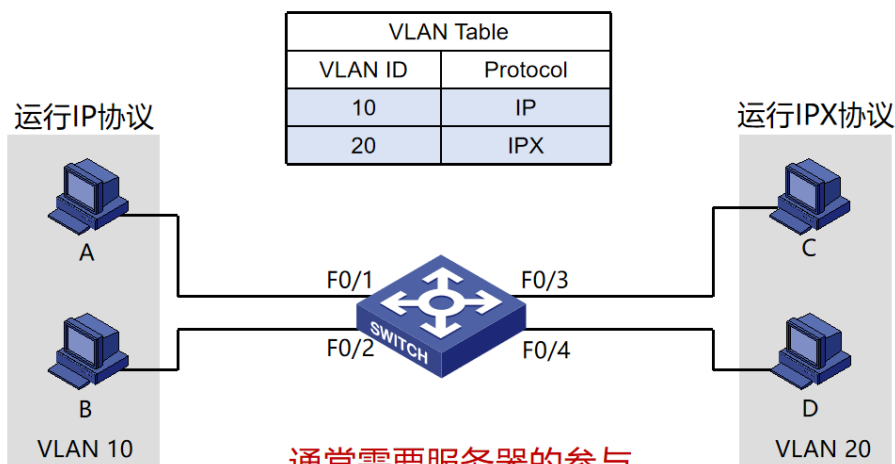
https://blog.csdn.net/qn_46101869

- 基于MAC地址的VLAN



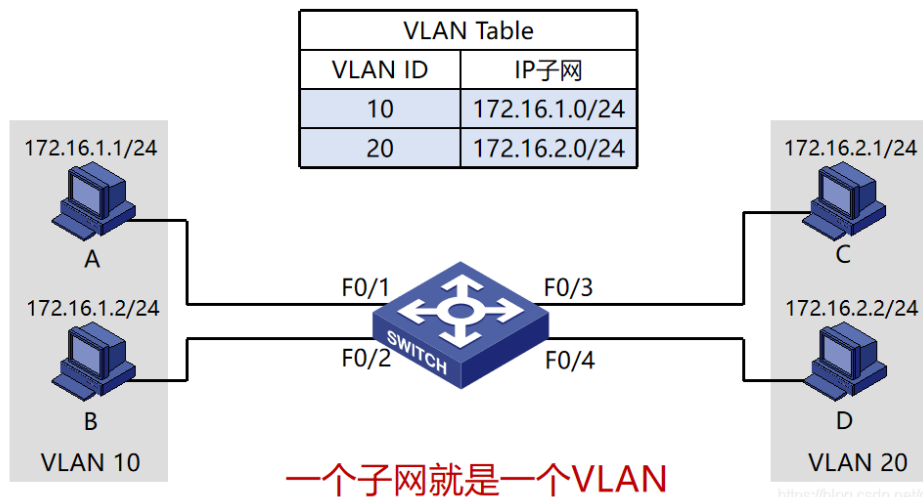
https://blog.csdn.net/qq_46101869

- 基于协议的VLAN



https://blog.csdn.net/qq_46101869

- 基于子网的VLAN

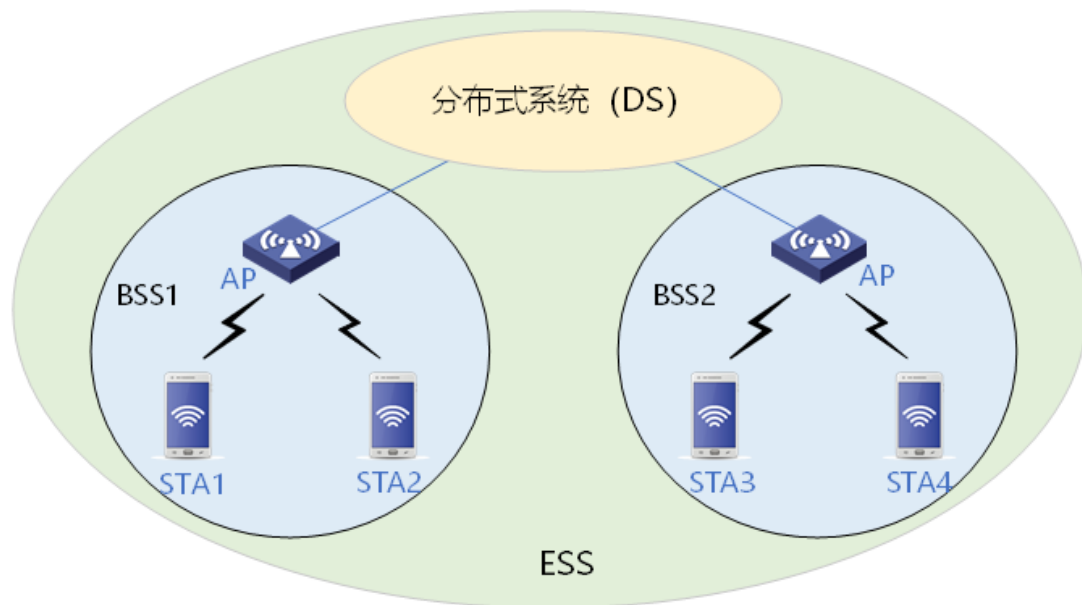


https://blog.csdn.net/qq_46101869

6.4无线局域网WLAN

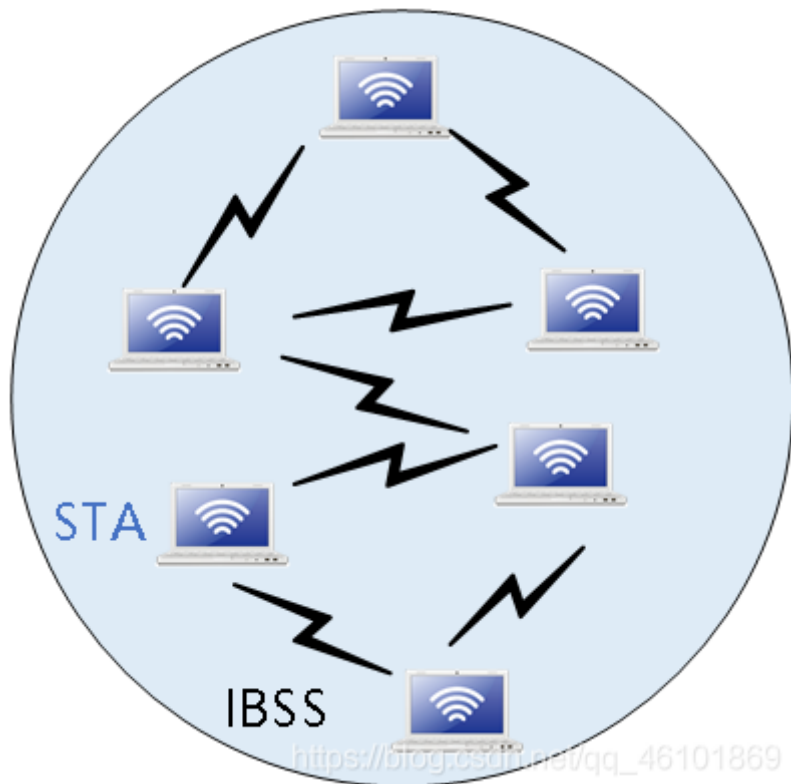
无线局域网 (Wireless Local Area Network, WLAN): 指以无线信道作为传输介质的计算机局域网

基础架构模式 (Infrastructure)



- 分布式系统 (DS)
- 访问点 (AP)
- 站点 (STA)
- 基本服务集 (BSS)
- 扩展服务集 (ESS)
- 站点之间通信通过AP转发

自组织模式 (Ad hoc)



- 站点 (STA)
- 独立基本服务集 (IBSS)
- 站点之间直接通信
- 共享同一无线信道

无线局域网需要解决的问题

1. 有限的无线频谱带宽资源

- 通道划分、空间重用
- 提高传输速率，解决传输问题
- 提高抗干扰能力和保密性

2. 共享的无线信道

- 介质访问控制方法 (CSMA/CA)
- 可靠性传输、安全性

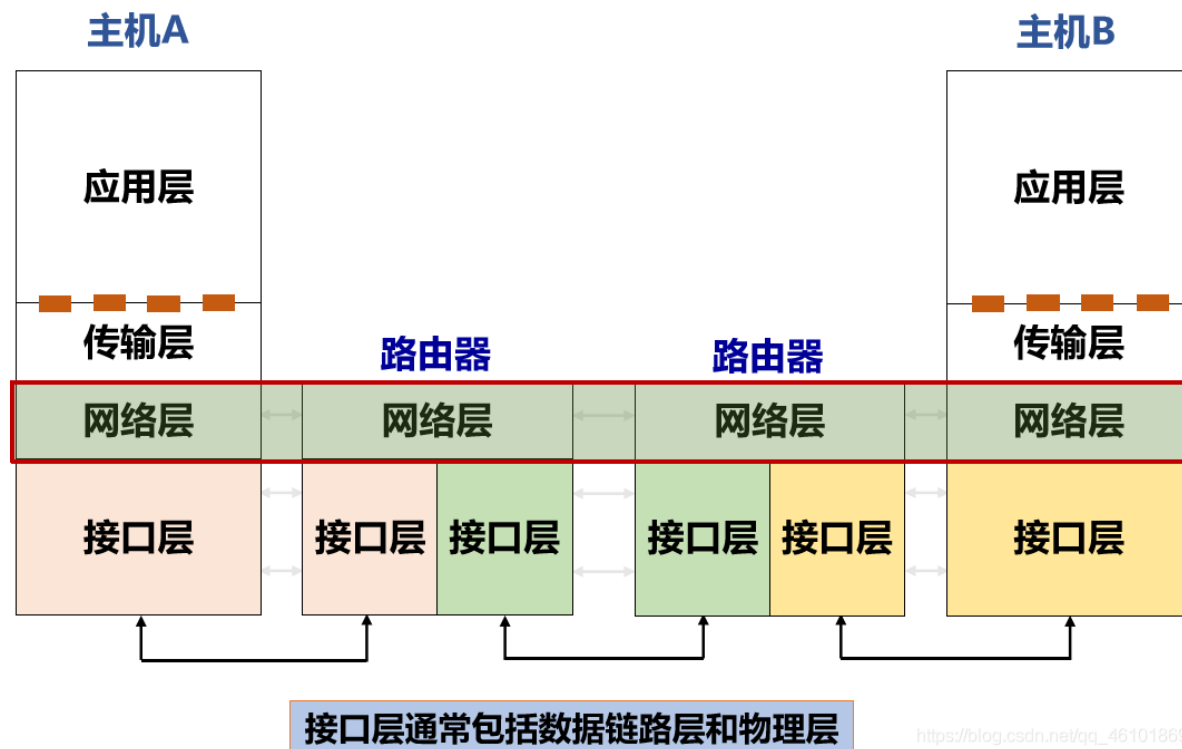
3. 组网模式管理

- BSS构建、认证、关联
- 移动性支持 (漫游)
- 睡眠管理 (节能模式)

四、网络层

1. 网络层概述

网络层在数据链路层提供的两个相邻端点之间的数据帧的传送功能上，进一步管理网络中的数据通信，将数据设法从源端经过若干个中间节点传送到目的端，从而向运输层提供最基本的端到端的数据传送服务。



2.网络层关键功能

- 路由（控制面）
 - 选择数据报从源端到目的端的路径
 - 核心：路由算法与协议
- 转发（数据面）
 - 将数据报从路由器的输入接口传送到正确的输出接口

3.Internet网际协议

3.1 IPv4协议及其相关技术

3.1.1 基本概念

IPv4协议，网际协议版本4，一种无连接的协议，是互联网的核心，也是使用最广泛的网际协议版本，其后继版本为IPv6

internet协议执行**两个基本功能**

- 寻址(addressing)
- 分片(fragmentation)

3.1.2 IPv4数据报格式

IP 数据报由首部和数据两部分组成



- **版本**：4bit，表示采用的IP协议版本
- **首部长度**：4bit，表示整个IP数据报首部的长度
- **区分服务**：8bit，该字段一般情况下不使用
- **总长度**：16bit，表示整个IP报文的长度,能表示的最大字节为 $2^{16}-1=65535$ 字节
- **标识**：16bit，IP软件通过计数器自动产生，每产生1个数据报计数器加1；在ip分片以后，用来标识同一片分片
- **标志**：3bit，目前只有两位有意义。
MF，置1表示后面还有分片，置0表示这是数据报片的最后1个；
DF，不能分片标志，置0时表示允许分片
- **片偏移**：13bit，表示IP分片后，相应的IP片在总的IP片的相对位置
- **生存时间TTL(Time To Live)**：8bit,表示数据报在网络中的生命周期，用通过路由器的数量来计量，即跳数（每经过一个路由器会减1）
- **协议**：8bit，标识上层协议（TCP/UDP/ICMP...）
- **首部校验和**：16bit，对数据报首部进行校验，不包括数据部分
- **源地址**：32bit，标识IP片的发送源IP地址
- **目的地址**：32bit，标识IP片的目的地IP地址
- **选项**：可扩充部分，具有可变长度，定义了安全性、严格源路由、松散源路由、记录路由、时间戳等选项
- **填充**：用全0的填充字段补齐为4字节的整数倍

3.1.3 数据报分片

分片原因

数据报长度大于传输链路的MTU

MTU (Maximum Transmission Unit)，最大传输单元

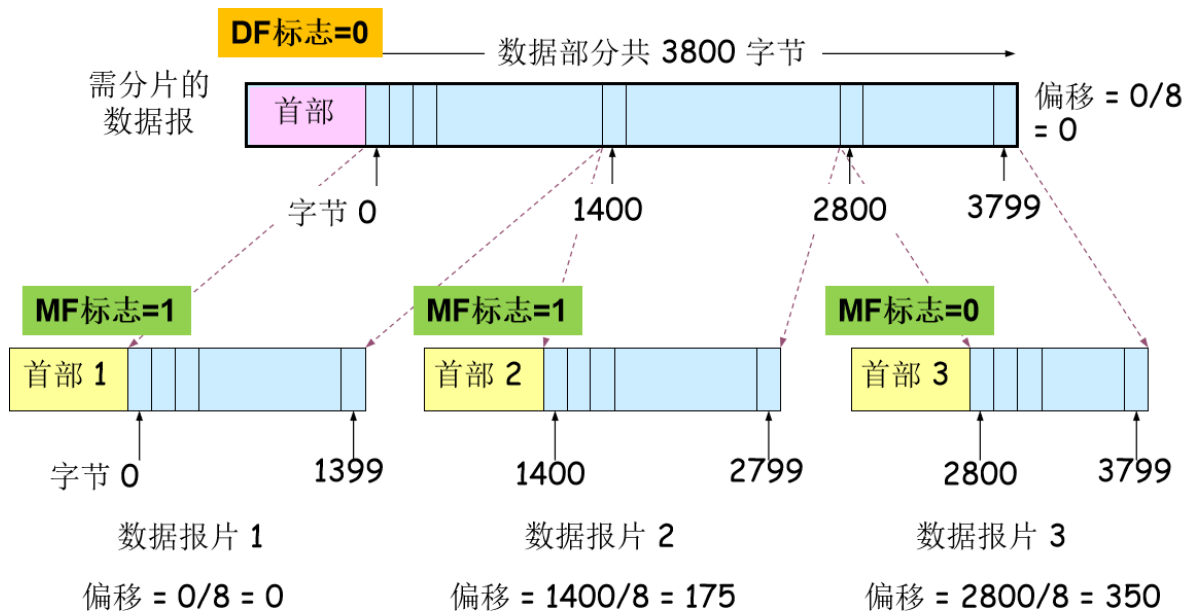
- 链路MTU
- 路径MTU (Path MTU)

分片策略

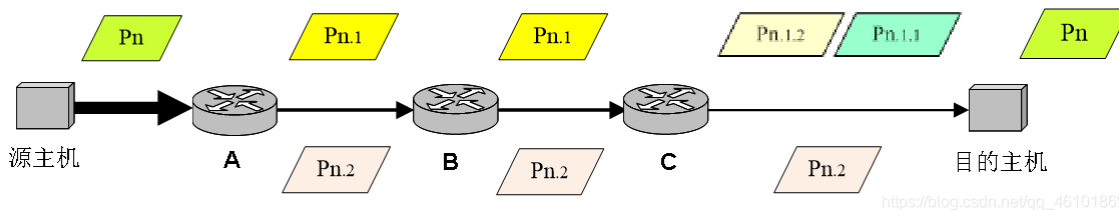
- 允许途中分片：根据下一跳链路的MTU实施分片
- 不允许途中分片：发出的数据报长度小于路径MTU（路径MTU发现机制）

重组策略

- 途中重组，实施难度太大
- 目的端重组（互联网采用的策略）
- 重组所需信息：原始数据报编号、分片偏移量、是否收集所有分片



原始报文和分片报文具有相同的IP标识（IP头部字段）



IPv4分片策略

- IPv4分组在传输途中可以多次分片
 - 源端系统，中间路由器（可通过标志位设定是否允许路由器分片，DF标志位）
- IPv4分片只在目的IP对应的目的端系统进行重组
- IPv4分片、重组字段在基本IP头部
 - 标识、标志、片偏移
- IPv6分片机制有较大变化（见IPv6部分的介绍）

注：分组 (packet) 与 帧(frame)的关系



3.1.4 IP协议功能及报头字段总结

网络层基本功能

- 支持多跳寻路将IP数据报送达目的端：**目的IP地址**
- 表明发送端身份：**源IP地址**
- 根据IP头部协议类型，提交给不同上层协议处理：**协议**

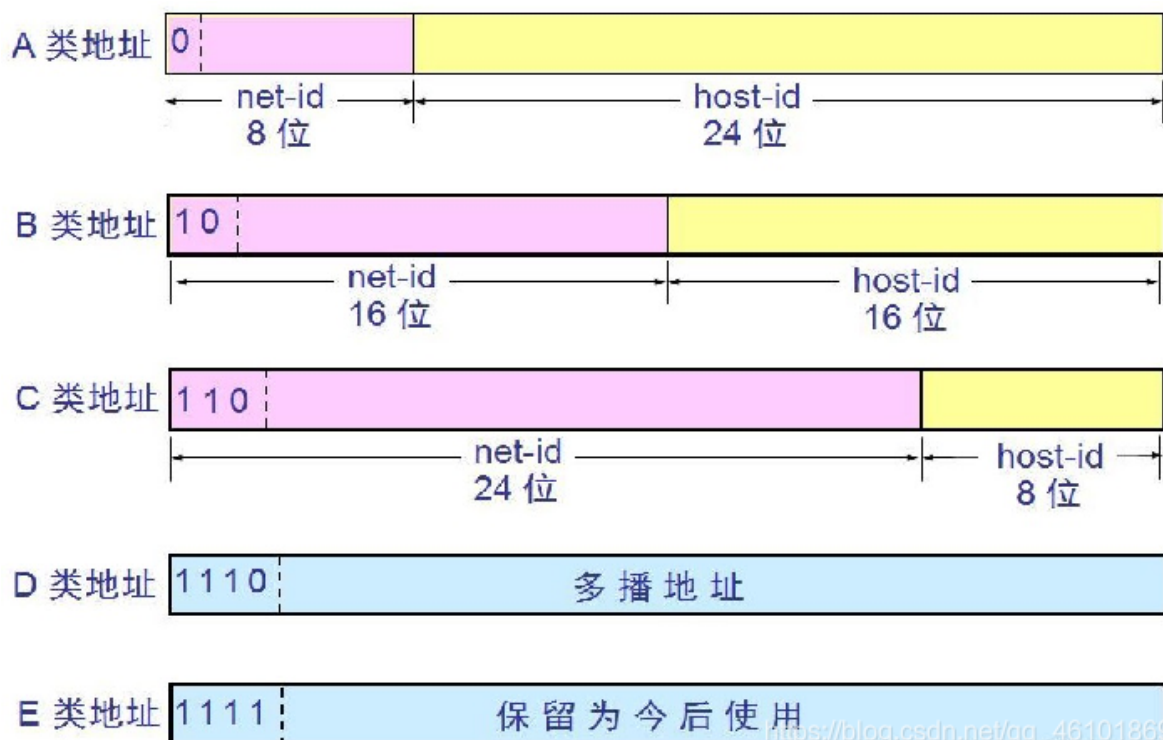
其它相关问题

- 数据报长度大于传输链路的MTU的问题，通过**分片机制**解决：**标识、标志、片偏移**
- 防止循环转发浪费网络资源（路由错误、设备故障...），通过**跳数限制**解决：**生存时间TTL**
- IP报头错误导致无效传输，通过**头部校验和**解决：**首部校验和**

3.2 IP地址

3.2.1 概述

- **IP地址**，网络上的每一台主机（或路由器）的每一个接口都会分配一个全球唯一的32位的标识符
- 将IP地址划分为固定的类，每一类都由两个字段组成
- 网络号相同的这块连续IP地址空间称为地址的**前缀**，或**网络前缀**
- IP地址共分为A、B、C、D、E五类，A类、B类、C类为单播地址
- IP地址的书写采用点分十进制记法，其中每一段取值范围为0到255

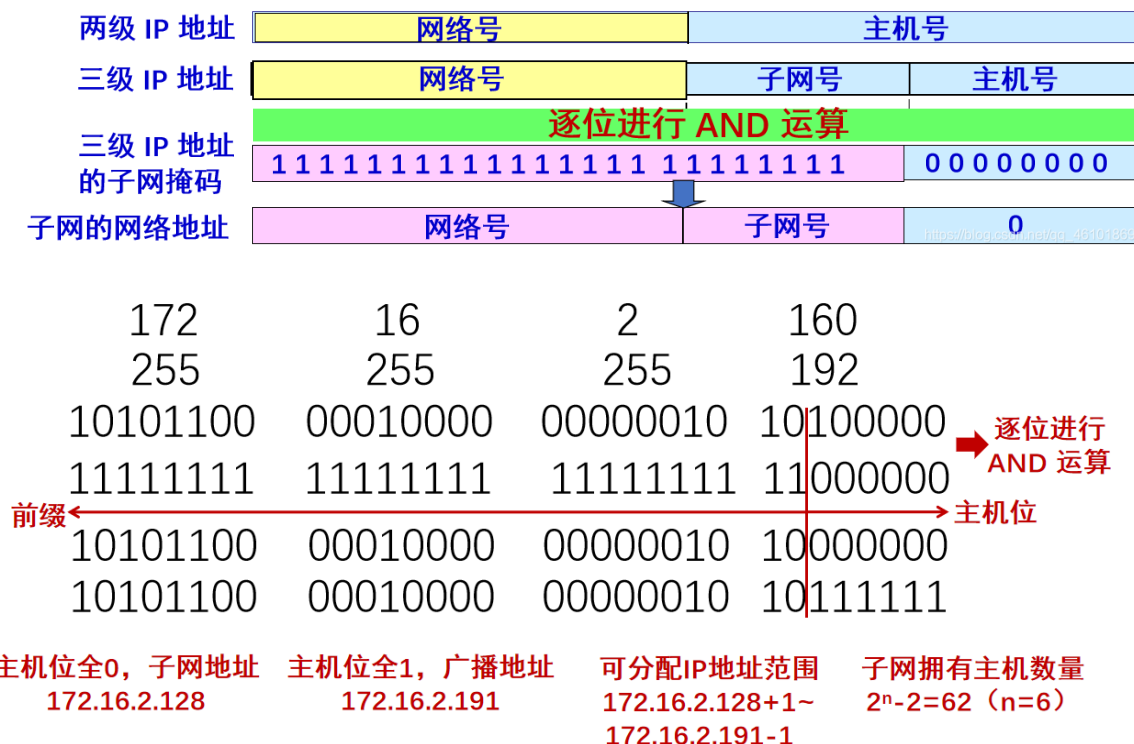


IP特殊地址

地址	用途
全0网络地址	只在系统启动时有效, 用于启动时临时通信, 又叫主机地址
网络127.0.0.0	指本地节点(一般为127.0.0.1), 用于测试网卡及TCP/IP软件, 这样浪费了1700万个地址
全0主机地址	用于指定网络本身, 称之为网络地址或者网络号
全1主机地址	用于广播, 也称定向广播, 需要指定目标网络
0.0.0.0	指任意地址
255.255.255.255	用于本地广播, 也称有限/受限广播, 无须知道本地网络地址

3.2.1 子网划分

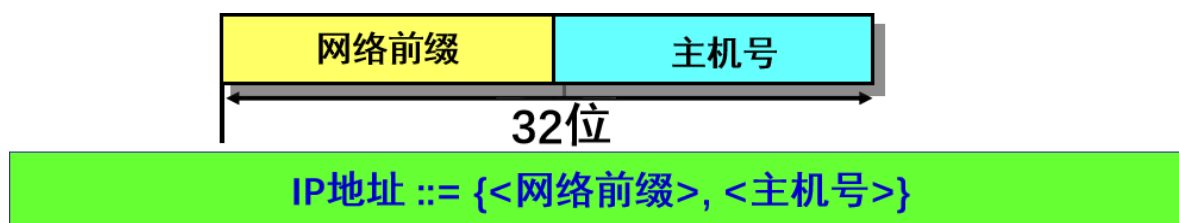
- 子网划分(subnetting), 在网络内部将一个网络块进行划分以供多个内部网络使用, 对外仍是一个网络
- 子网(subnet), 一个网络进行子网划分后得到的一系列结果网络称为子网
- 子网掩码(subnet mask), 与IP地址一一对应, 是32 bit的二进制数, 置1表示网络位, 置0表示主机位
- 子网划分减少了IP地址的浪费、网络的组织更加灵活、便于维护和管理



https://blog.csdn.net/tq_45101869

3.2.3 无类域间路由

- 将32位的IP地址划分为前后两个部分, 并采用斜线记法, 即在IP地址后加上"/", 然后再写上网络前缀所占位数



- 一个 CIDR 地址块可以表示很多地址，这种地址的聚合常称为**路由聚合**（route aggregation），也称为**构成超网（supernet）**
- 聚合技术在Internet中大量使用，它允许前缀重叠，数据包按具体路由的方向发送，即具有最少IP地址的**最长匹配前缀**

注：当路由器收到一个IP数据包时，它会将数据包的目的IP地址与自己本地路由表中的所有路由表进行**逐位（Bit-By-Bit）对比**，直到找到**匹配度最长**的条目，这就是**最长前缀匹配机制**。（路由比对之前会用相应路由的目的网络掩码进行逻辑与运算，再拿结果与路由路径比对）

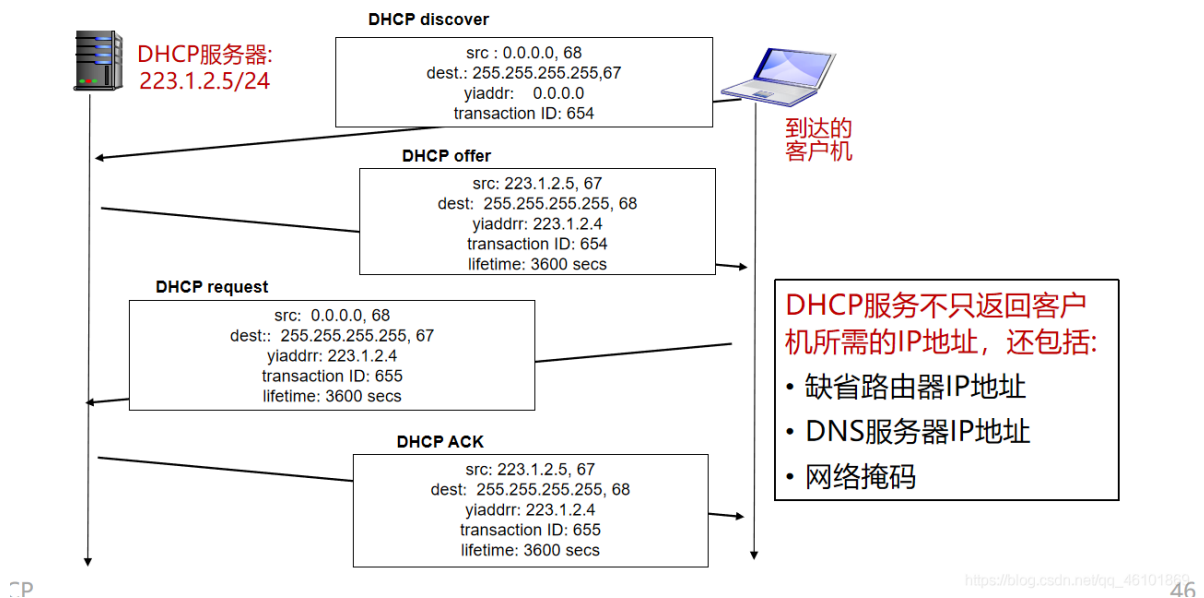
3.3 DHCP动态主机配置协议

DHCP：**动态主机配置协议**

- 当主机加入IP网络，允许主机从DHCP服务器动态获取IP地址
- 可以有效利用IP地址，方便移动主机的地址获取

工作模式：**客户/服务器模式（C/S）**

基于 UDP 工作，服务器运行在 67 号端口，客户端运行在 68 号端口



- DHCP 客户从UDP端口68以广播形式向服务器发送发现报文（DHCPDISCOVER）
- DHCP 服务器广播发出提供报文（DHCPOFFER）
- DHCP 客户从多个DHCP服务器中选择一个，并向其以广播形式发送DHCP请求报文（DHCPREQUEST）
- 被选择的DHCP服务器广播发送确认报文（DHCPACK）

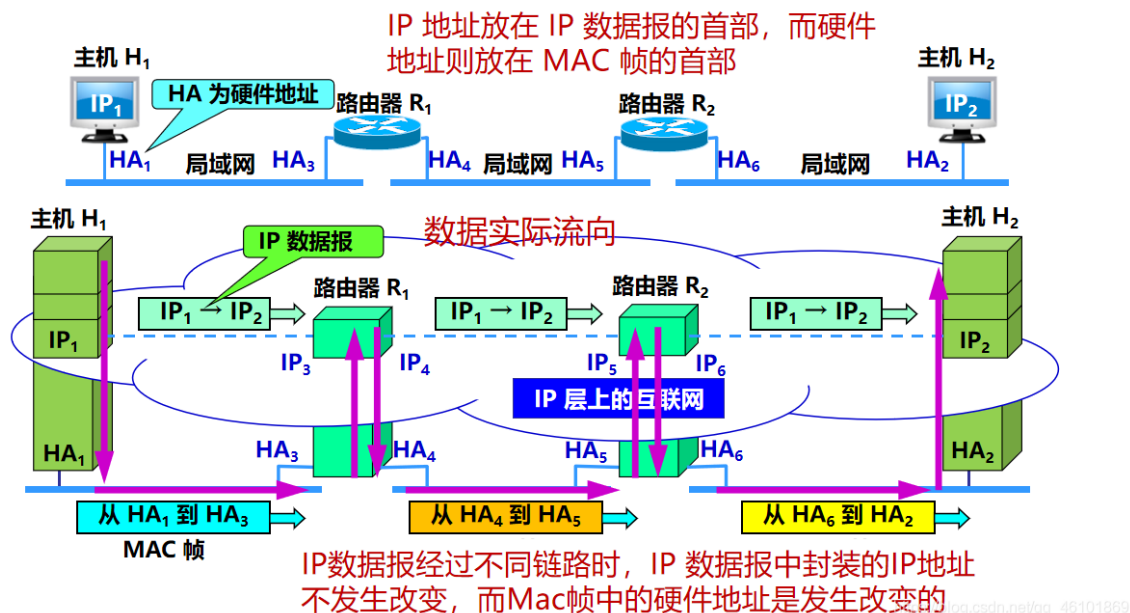
3.4 ARP地址解析协议

背景

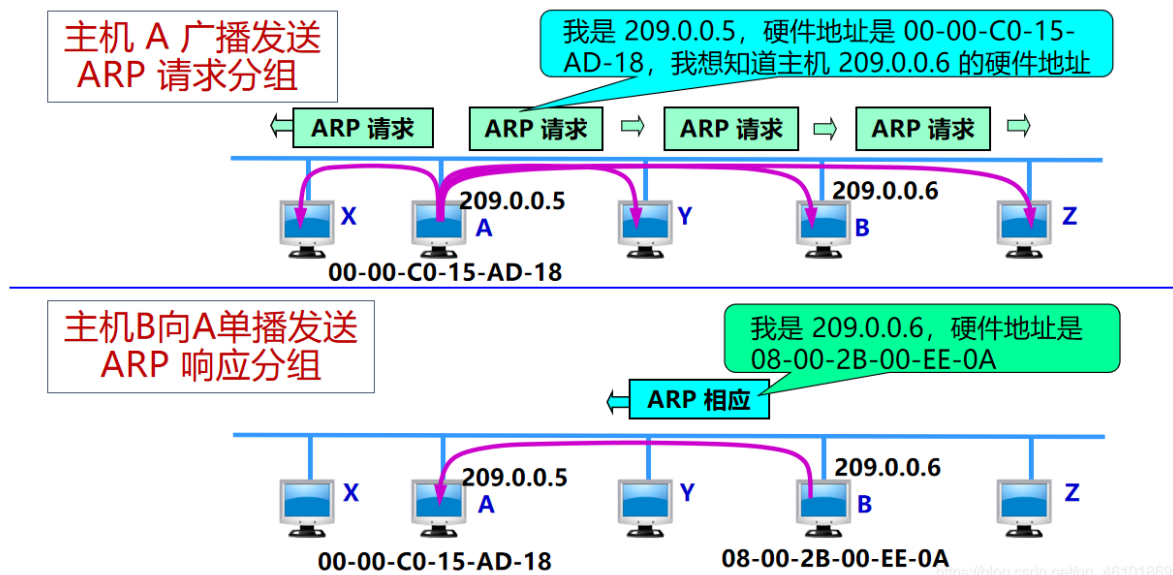
网络设备有数据要发送给另一台网络设备时，必须要知道对方的网络层地址（即IP地址）。IP地址由网络层来提供，但是仅有IP地址是不够的，IP数据报文必须封装成帧才能通过数据链路进行发送。数据帧必须要包含目的MAC地址，因此发送端还必须获取到目的MAC地址。通过目的IP地址二获取的MAC地址的过程是由ARP（Address Resolution Protocol）协议来实现的。

注：因为下层协议是通过MAC地址来确定各自身份的，所以下层发送必须要MAC地址

IP 与 MAC地址



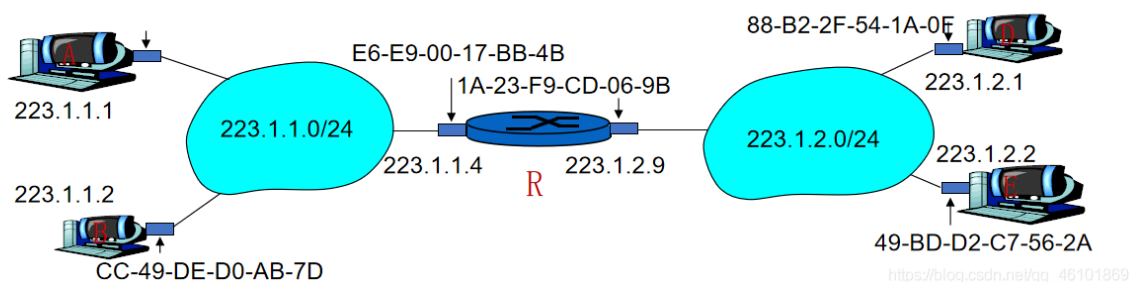
ARP 协议工作过程



- A 已知 B 的 IP 地址，需要获得 B 的 MAC 地址（物理地址）
- 如果 A 的 ARP 表中缓存有 B 的 IP 地址与 MAC 地址的映射关系，则直接从 ARP 表获取
- 如果 A 的 ARP 表中未缓存有 B 的 IP 地址与 MAC 地址的映射关系，则 A 广播包含 B 的 IP 地址的 ARP query 分组
 - 在局域网上的所有节点都可以接收到 ARP query
- B 接收到 ARP query 分组后，将自己的 MAC 地址发送给 A
- A 在 ARP 表中缓存 B 的 IP 地址和 MAC 地址的映射关系
 - 超时时删除

路由到另一个局域网

例：从A经过R到E



- A创建IP数据包（源为A、目的为E）
- 在源主机A的路由表中找到路由器R的IP地址223.1.1.4
- A根据R的IP地址223.1.1.4，使用ARP协议获得R的MAC地址
- A创建数据帧（目的地址为R的MAC地址）
- 数据帧中封装A到E的IP数据包
- A发送数据帧，R接收数据帧

3.5 网络地址转换(NAT)

定义

网络地址转换(NAT)用于解决IPv4地址不足的问题，是一种将私有（保留）地址转化为公有IP地址的转换技术

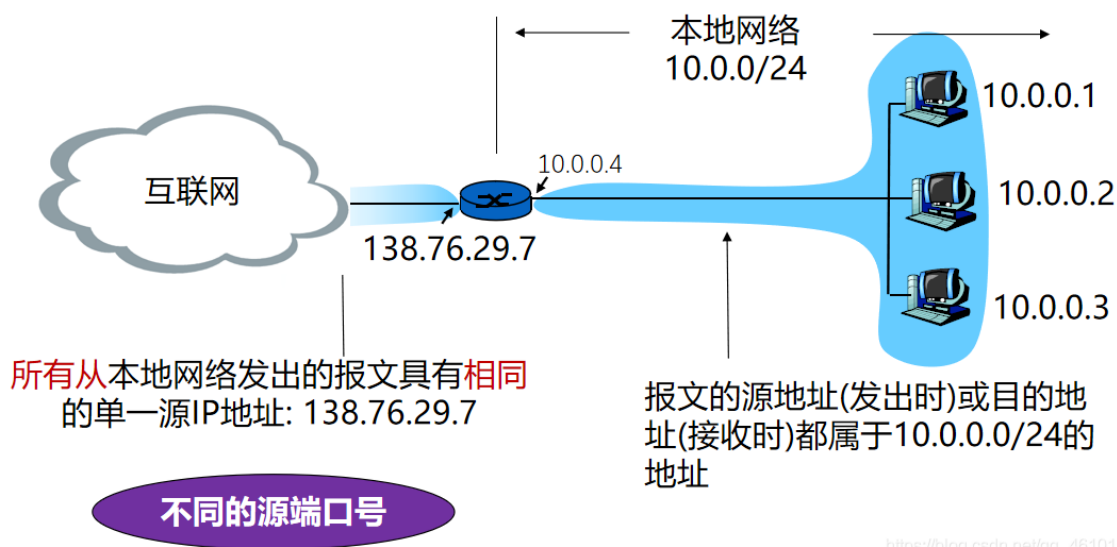
私有IP地址：

A类地址：10.0.0.0--10.255.255.255

B类地址：172.16.0.0--172.31.255.255

C类地址：192.168.0.0--192.168.255.255

NAT工作机制



- **出数据报**：外出数据报用 NAT IP地址(全局), 新port # 替代 源IP地址(私有), port #
- **NAT转换表**：每个 (源IP地址, port #)到(NAT IP地址, 新port #) 映射项
- **入数据报**：对每个入数据报的地址字段用存储在NAT表中的(源IP地址, port #)替代对应的 (NAT IP地址, 新port #)
- NAT根据不同的IP上层协议进行NAT表项管理
TCP, UDP, ICMP
- 传输层TCP/UDP拥有16-bit 端口号字段
所以一个WAN侧地址可支持60,000个并行连接

NAT的优势

- 节省合法地址，减少地址冲突
- 灵活连接Internet
- 保护局域网的私密性

3.6 ICMP: 互联网控制报文协议

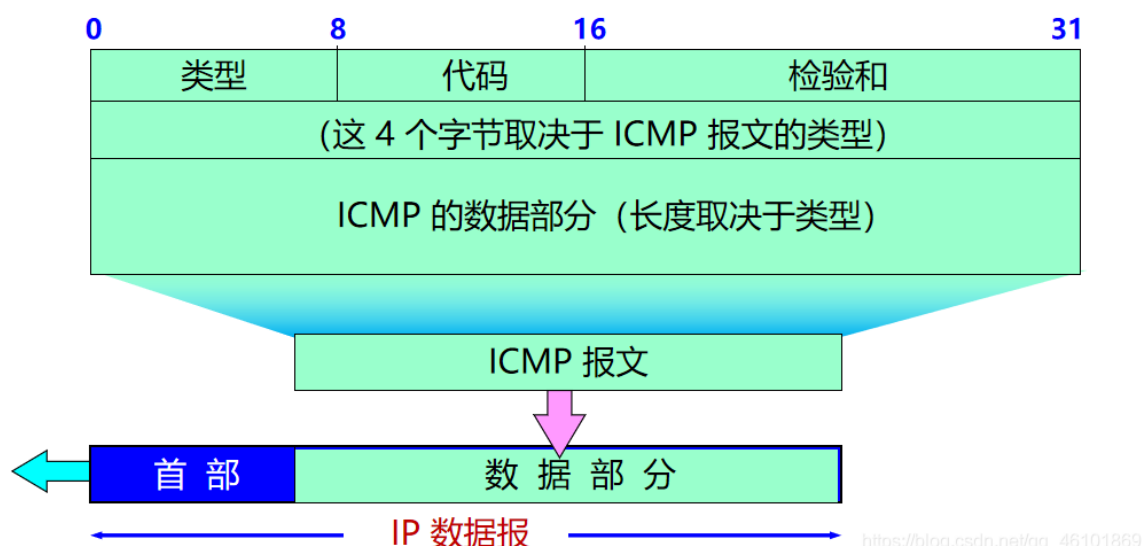
ICMP: 互联网控制报文协议

- ICMP 允许主机或路由器报告差错情况和提供有关异常情况的报告
- 由主机和路由器用于网络层信息的通信
- ICMP 报文携带在IP 数据报中：IP上层协议号为1

ICMP报文类型

- **ICMP 差错报告报文**
 - 终点不可达：不可达主机、不可达网络，无效端口、协议
- **ICMP 询问报文**
 - 回送请求/回答 (ping使用)

ICMP 报文格式



- ICMP报文的前 4 个字节包含格式统一的三个字段：类型、代码、校验和
- 相邻的后四个字节内容与ICMP的报文类型有关

ICMP报文类型及功能

ICMP报文类型	类型值	功能描述
差错报告报文	3	终点不可达
	5	改变路由(Redirect)
	11	时间超时
	12	参数问题
询问报文	8或0	回送(Echo)请求或应答
	13或14	时间戳(Timestamp)请求或回答

注：根据RFC 6633规定，已不再使用类型值为4、10或9、15或16、17或18的报文。

类型	代码	功能描述
0	0	回送应答 (ping)
3	0	目的网络不可达
3	1	目的主机不可达
3	2	目的协议不可达
3	3	目的端口不可达
3	6	目的网络未知
3	7	目的主机未知
8	0	回送请求 (ping)
9	0	路由通告
10	0	路由发现
11	0	TTL过期
12	0	坏的IP首部

4.路由算法

路由算法须满足的特性：

- 正确性
- 简单性
- 鲁棒性
- 稳定性
- 公平性
- 有效性

根据路由算法是否随网络的通信量或拓扑自适应划分

- 静态路由选择策略（非自适应路由选择）
- 动态路由选择策略（自适应路由选择）

4.1 距离向量路由

算法基本思想

- 每个节点周期性地向邻居发送它自己到某些节点的距离向量；
- 当节点x接收到来自邻居的新DV估计，它使用B-F方程更新其自己的DV：
$$D_x(y) \leftarrow \min_v \{c(x,v) + D_v(y)\} \quad \text{for each node } y \in N$$
- 上述过程迭代执行， $D_x(y)$ 收敛为实际最小费用 $dx(y)$

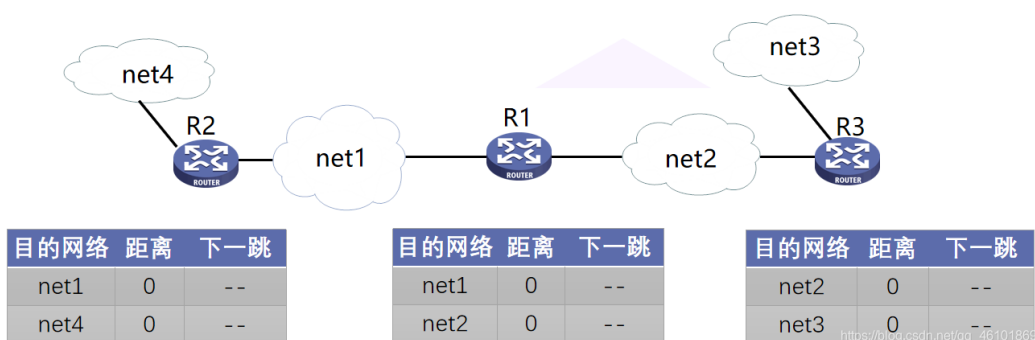
距离向量算法特点：**迭代的、分布式的**

- 每次本地迭代由下列引起：本地链路费用改变、邻居更新报文
- 分布式：各节点依次计算，相互依赖

注：路由器只掌握物理相连的邻居以及链路费用

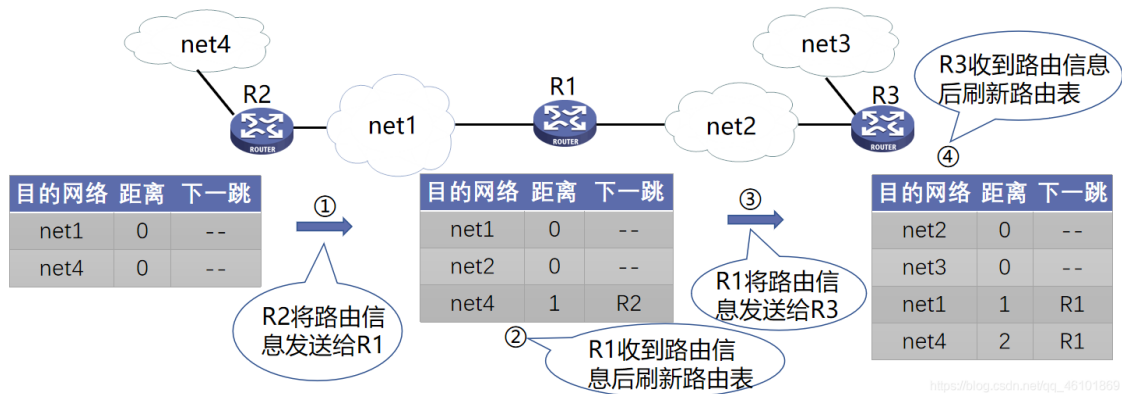
算法过程

- 路由器启动时初始化自己的路由表
 - 初始路由表包含所有直接相连的网络路径，距离均为0

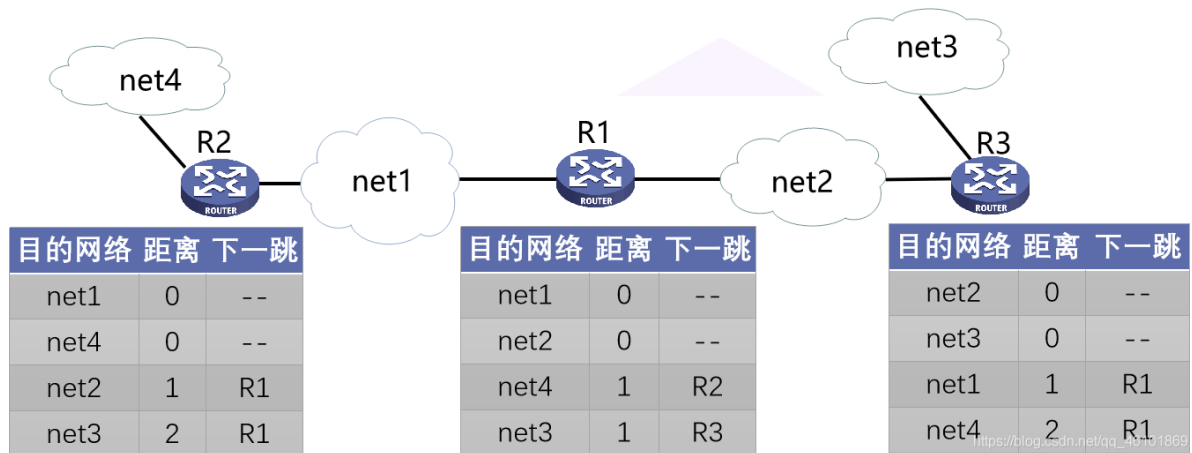


- 路由器周期性地向其相邻路由器广播自己知道的路由信息

- 相邻路由器可以根据收到的路由信息修改和刷新自己的路由表

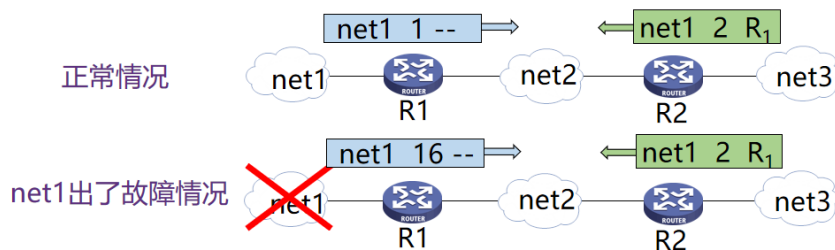


- 路由器经过若干次更新后，最终都会知道到达所有网络的最短距离
- 所有的路由器都得到正确的路由选择信息时网络进入“收敛”（convergence）状态



特殊情况考虑

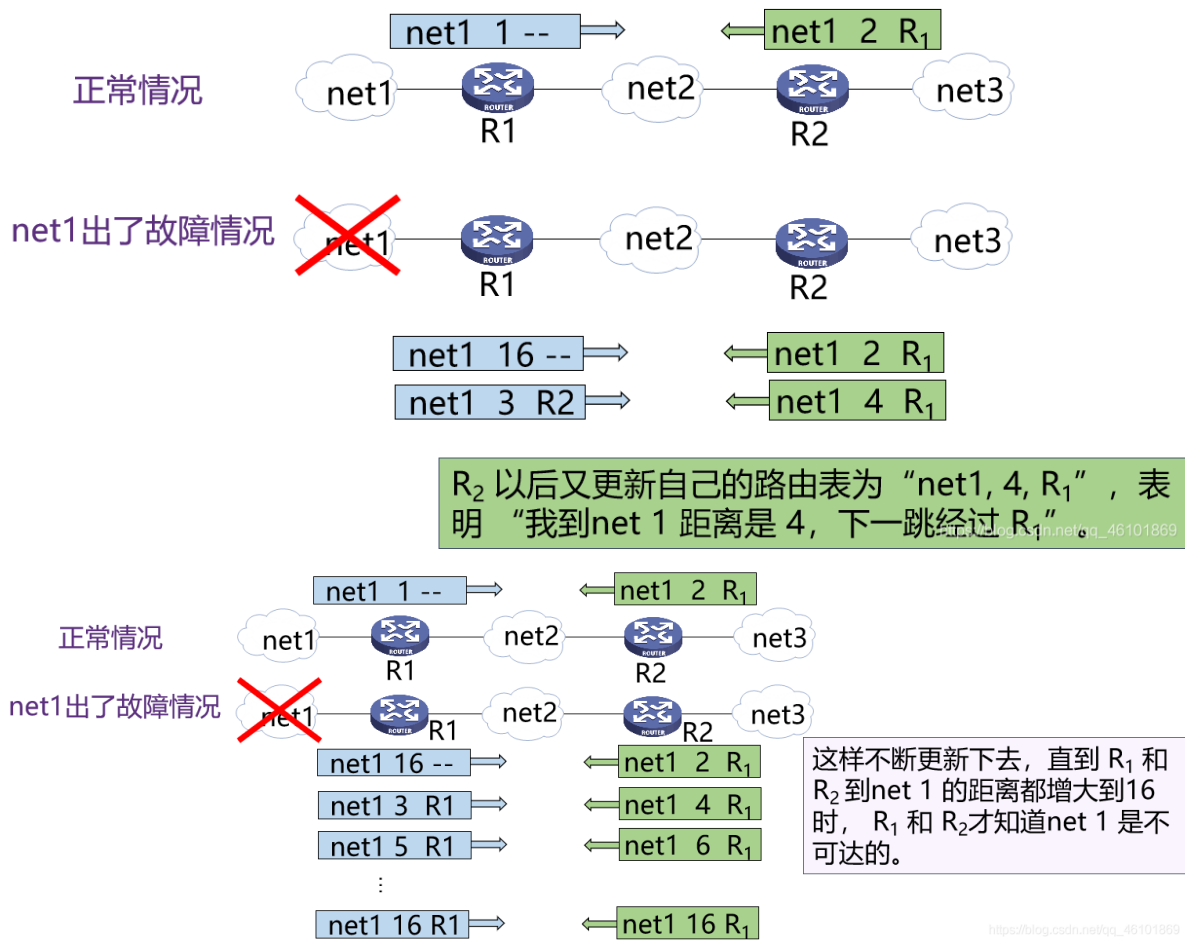
计数到无穷问题（The Count-to-Infinity Problem）



R₁ 说：“我到net1 的距离是 16（表示无法到达），是直接交付。”

但 R₂ 在收到 R₁ 的更新报文之前，还发送原来的报文，因为这时 R₂ 并不知道 R₁ 出了故障。

https://blog.csdn.net/hq_46101869



好消息传播快，坏消息传播慢，是距离向量路由的一个主要缺点

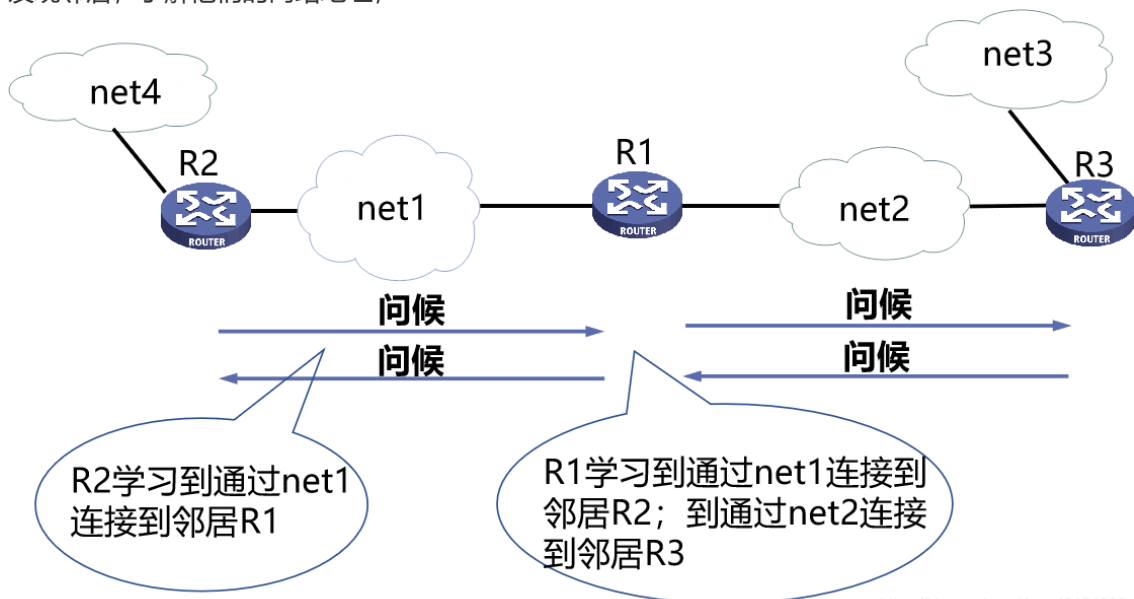
4.2 链路状态路由

注：所有路由器掌握完整的网络拓扑和链路费用信息

算法过程

链路状态（Link State）路由可分为五个部分：

1. 发现邻居，了解他们的网络地址；



2. 设置到每个邻居的成本度量；

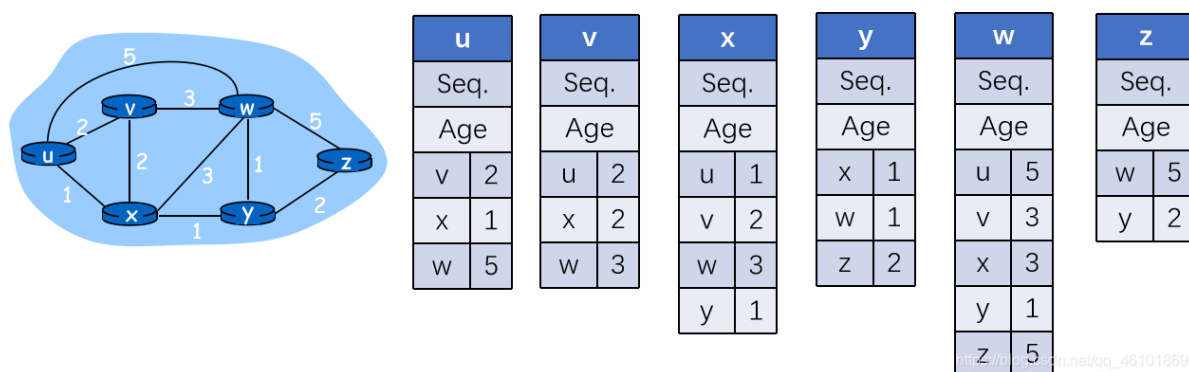
- 开销/度量/代价：

- 自动发现设置或人工配置
- 度量：带宽、跳数、延迟、负载、可靠性等
- 常用度量：链路带宽（反比）
 - 例如：1-Gbps以太网的代价为1，100-Mbps以太网的代价为10
- 可选度量：延迟
 - 发送一个echo包，另一端立即回送一个应答
 - 通过测量往返时间RTT，可以获得一个合理的延迟估计值

3. 构造一个分组，分组中包含刚收到的所有信息；

构造链路状态分组（link state packet, LSP）

- 发送方标识
- 序列号
- 年龄
- 邻居列表



4. 将此分组发送给其他的路由器；

- 每个LSP分组包含一个序列号，且递增
- 路由器记录所收到的所有（源路由器、序列号）对
- 当一个新分组到达时，路由器根据记录判断：
 - 如果是新分组，洪泛广播
 - 如果是重复分组，丢弃
 - 如果是过时分组，拒绝

5. 计算到其他路由器的最短路径。

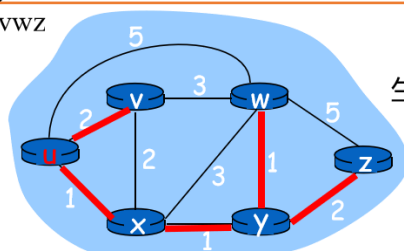
Dijkstra算法示例

$D(k)$ ：从计算节点到目的节点k当前路径代价

$p(k)$ ：从计算节点到目的节点k的路径中k节点的前继节点

步骤	集合N	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2,u	5,u	1,u	∞	∞
→ 1	ux	2,u	4,x		2,x	∞
→ 2	uxy	2,u	3,y			4,y
→ 3	uxyv		3,y			4,y
→ 4	uxyvw					4,y
→ 5	uxyvwz					

生成最短路径树



生成路由表

目的	下一跳	代价
v	v	2
w	x	3
x	x	1
y	x	2
z	x	4

距离向量和链路状态算法比较

网络状态信息交换的范围

- DV:邻居间交换
- LS:全网扩散

网络状态信息的可靠性

- DV:部分道听途说
- LS:自己测量

4.3 层次路由

产生原因

- 过于庞大的路由表存储、查找困难，路由信息交互开销高

现实情况：

- 地址分配往往是随机的，难以进行高效的地址聚合
- 每个网络的网络管理员有自己的管理方法和思路，并不希望每个路由器都干涉本网络内部的地址分配等问题

层次路由可以解决：

- 网络扩展性问题：当网络扩大时，控制路由表条目和路由表存储空间的增长
- 管理的自治问题：网络管理员可以控制和管理自己网络的路由

基本思路

- 互联网由大量不同的网络互连，每个管理机构控制的网络是自治的
- 自治系统（AS，Autonomous System）
 - 一个管理机构控制之下的网络
 - 一个AS内部通常使用相同的路由算法/路由协议，使用统一的路由度量（跳数、带宽、时延...）
 - 不同的AS可以使用不同的路由算法/路由协议
 - 每个AS有一个全球唯一的ID号：AS ID
- 自治系统内的还可以进一步划分层次：私有自治系统或区域

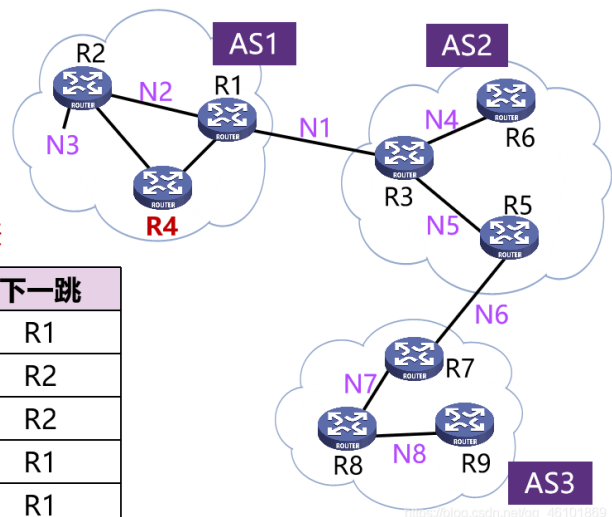
效果

路由不分层的情况下
R4的路由表

目的地	下一跳
N1	R1
N2	R2
N3	R2
N4	R1
N5	R1
N6	R1
N7	R1
N8	R1

R4的层次表

目的地	下一跳
N1	R1
N2	R2
N3	R2
AS2内的所有网络	R1
AS3内的所有网络	R1



https://blog.csdn.net/qz_46101889

4.4 广播路由

广播 (Broadcasting)： 源主机同时给全部目标地址发送同一个数据包

实现方法

①给每个主机单独发送一个数据包

- 效率低、浪费带宽
- Server需要知道每个目的地址

②多目标路由 (multi-destination routing)

- 在需要转发的路由器线路复制一次该数据报
- 网络利用率高
- Server依然需要知道所有的目的地址

注：以上方法难以实现

③泛洪 (flooding)

- 一种将数据包发送到所有网络节点的简单方法
- 将每个进入数据包发送到除了进入线路外的每条出去线路

用途

- 保证性：一种有效广播手段，可确保数据包被传送到网络中每个节点
- 鲁棒性：即使大量路由器被损坏，也能找到一条路径（如果存在）
- 简单性：仅需知道自己的邻居

无控制的泛洪

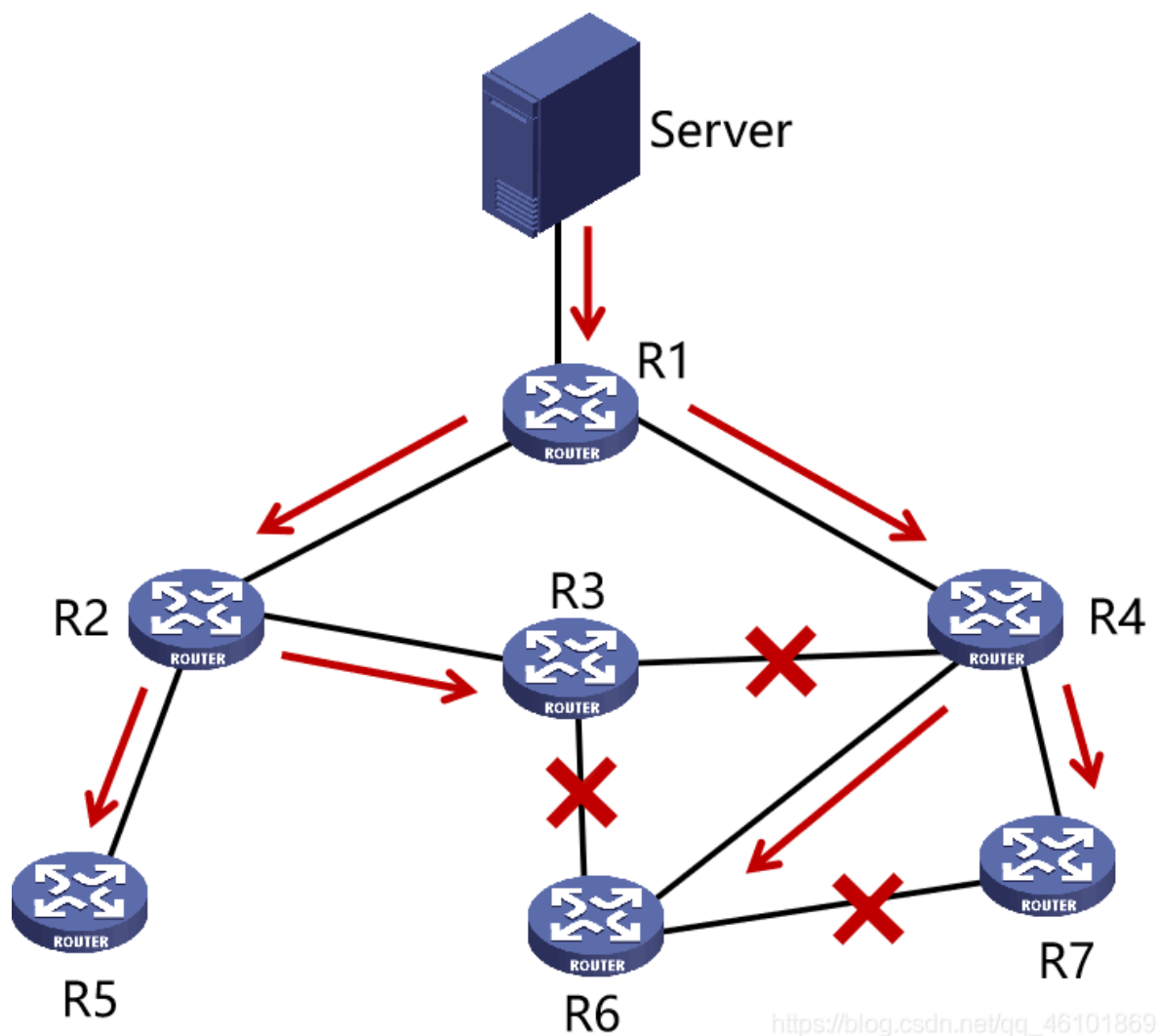
- 实现广播最显而易见的技术
- 环路可能导致广播风暴
- 路由器可能收到多个副本
- 节点需要跟踪已泛洪的数据包以阻止洪泛
- 即使利用跳数来限制，也会出现成倍爆炸

解决方法：受控制的泛洪（每个路由器进行有选择的泛洪）

- 序号控制泛洪
- 逆向路径转发

④生成树 (spanning tree)

- 源节点向所有属于该生成树的特定链路发送分组
- 改进了逆向路径转发
- 没有环路
- 最佳使用带宽
- 最少副本，消除了冗余分组
- 一个路由器可以不必知道整颗树，只需要知道在一颗树中的邻居即可



5. Internet路由协议

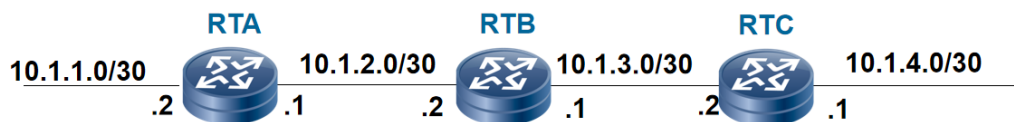
5.1 路由选择协议RIP

概述

- 路由选择协议RIP (Routing Information Protocol) 是基于距离矢量算法的协议
- 使用跳数衡量到达目的网络的距离
 - RIP 认为一个**好的路由**就是它通过的路由器的数目少，即“**距离短**”
 - RIP 允许一条路径最多只能包含 15 个路由器
- RIP协议的基本思想
 - 仅和**相邻路由器**交换信息
 - 路由器交换的内容是自己的**路由表**
 - 周期性更新： **30s**

工作过程

- 初始化

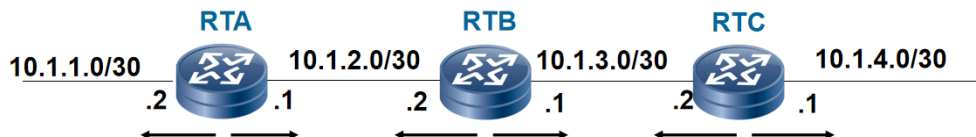


目标网络	下一跳	距离
10.1.1.0	--	0
10.1.2.0	--	0

目标网络	下一跳	距离
10.1.2.0	--	0
10.1.3.0	--	0

目标网络	下一跳	距离
10.1.3.0	--	0
10.1.4.0	--	0

- 周期性更新



目标网络	下一跳	距离
10.1.1.0	--	0
10.1.2.0	--	0
10.1.3.0	10.1.2.2	1
10.1.4.0	10.1.2.2	2

目标网络	下一跳	距离
10.1.2.0	--	0
10.1.3.0	--	0
10.1.1.0	10.1.2.1	1
10.1.4.0	10.1.3.2	1

目标网络	下一跳	距离
10.1.3.0	--	0
10.1.4.0	--	0
10.1.2.0	10.1.3.1	1
10.1.1.0	10.1.2.1	2

小结

RIP协议的特点

- 算法简单，易于实现
- 收敛慢
- 需要交换的信息量较大

RIP协议的适用场合

- 中小型网络

RIP协议的防环路机制

- 触发更新
- 毒性反转
- 水平分割
- 其他

5.2 BGP-外部网关路由协议

路由协议

- 内部网关协议 IGP：有 RIP 和、OSPF、ISIS 等多种具体的协议
- 外部网关协议 EGP：目前使用的协议就是 BGP

边界网关协议BGP (Border Gateway Protocol)

- 目前互联网中唯一实际运行的自治域间的路由协议

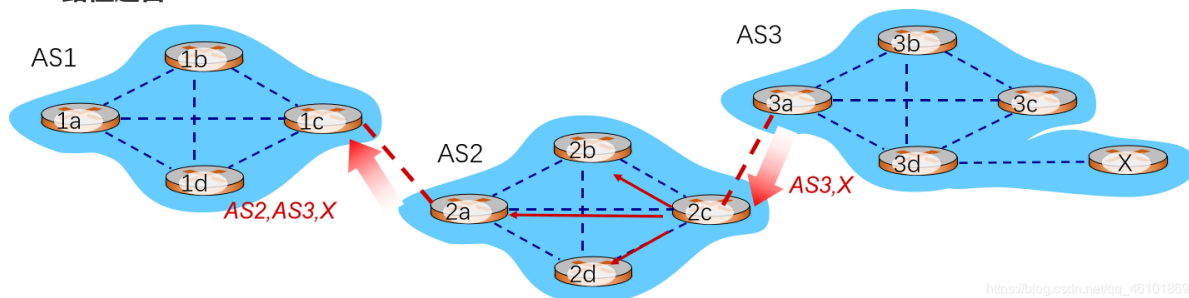
BGP功能

- eBGP：从相邻的AS获得网络可达信息
- iBGP：将网络可达信息传播给AS内的路由器
- 基于网络可达信息和策略决定到其他网络的“最优”路由

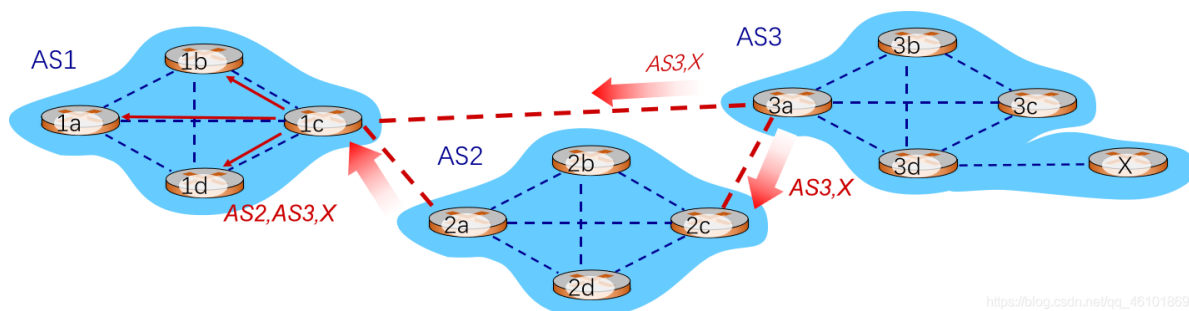
BGP会话: 两个BGP路由器通过TCP连接交换BGP报文

- 通告到不同网络前缀的路径，即路径向量协议

BGP路径通告



- AS2的路由器2c从AS3的路由器3a接收到路径AS3, X
- 根据AS2的策略，AS2的路由器2c接受路径AS3, X，通过iBGP传播给AS2的所有路由器
- 根据AS2策略，AS2的路由器2a通过eBGP向AS1的路由器1c通告从AS3的路由器3a接收到路径AS2, AS3, X



路由器可能会学到多条到达目的网络的路径:

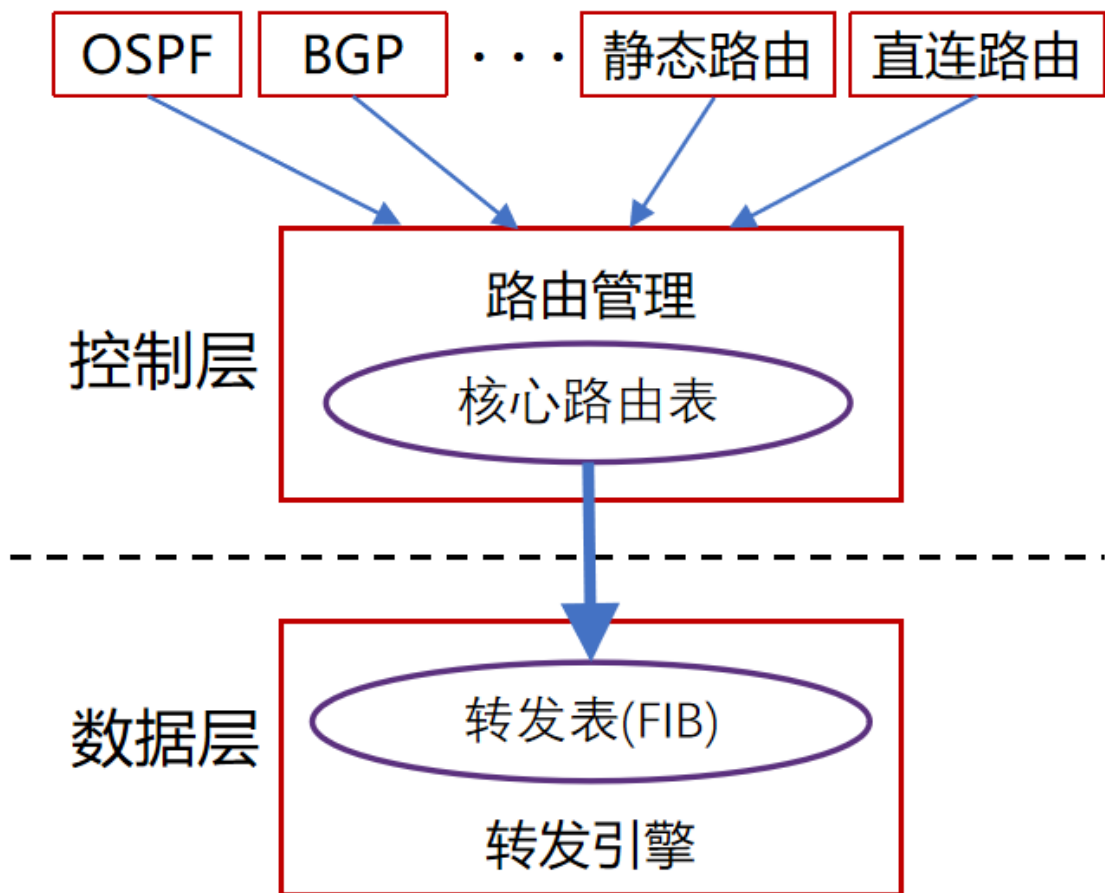
- AS1的路由器1c从2a学到路径AS2, AS3, X
- AS1的路由器1c从3a学到路径AS3, X
- 由策略，AS1路由器1c可能选择路径AS3, X，并在AS1中通过iBGP通告路径

6. 路由器的工作原理

6.1 路由器概述

路由器是互联网最主要的网络设备，包含2个核心功能

- 控制层：运行各种路由协议：BGP、OSPF、RIP，学习去往不同目的的转发路径：路由表
- 数据层：根据上述路由表，将收到的IP分组转发到正确的下一跳链路



https://blog.csdn.net/qz_46101869

6.2 路由器控制层

- 路由器可同时运行多个路由协议
- 路由器也可不运行任何路由协议，只使用静态路由和直连路由
- 路由管理根据路由优先级，选择最佳路由，形成核心路由表
- 控制层将核心路由表下发到数据层，形成转发表（FIB）
- 若存在多个“去往同一目的IP前缀”的不同类型路由，路由器根据优先级选择最佳路由
- 优先级数值越小，优先级越高

路由种类	路由优先级
直连路由	0
静态路由	1
eBGP路由	20
OSPF路由	110
RIP路由	120
iBGP路由	200

https://blog.csdn.net/qz_46101869

6.3 路由器数据层

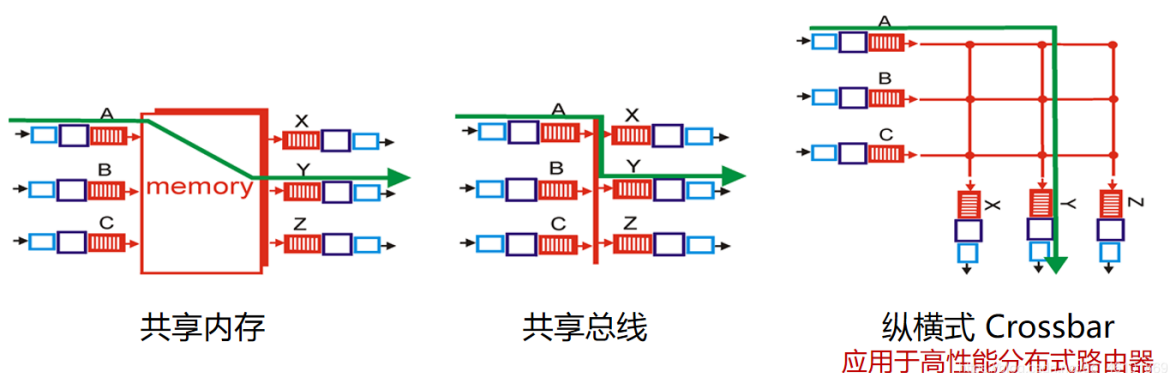
路由器中IP报文转发核心功能

- 链路层解封装，IP头部校验
- 获取报文目的IP地址
- 用目的IP地址，基于最长前缀匹配规则查询转发表
- 查询失败，丢弃报文
- 查询成功
 - 获取转发出口和下一跳IP地址
 - IP头部“TTL”字段值减1，重新计算IP头部“校验和”
 - 重新进行链路层封装，发送报文

注：普通IP报文转发过程中，路由器不查看传输层及以上层协议的内容

- IP报文在路由器转发前后的变化
 - 链路层封装更新，IP头部“TTL”减1，IP头部“校验和”更新
- 数据报在不同硬件单元的处理
 - 报文输入的接口卡
 - 链路层解封装
 - 转发表查询（该工作在输入接口卡处理）
 - 通过交换结构将报文排队发往目的接口卡（发送过快将产生拥塞）
 - 交换结构
 - 从输入接口卡发往输出接口卡
 - 报文输出的接口卡
 - 从交换结构接收报文（排队进行后续处理，到达太快将产生拥塞）
 - 链路层封装
 - 从输出接口发送报文

3种典型的交换结构



7. 拥塞控制算法

7.1 概述

拥塞

网络中存在太多的数据包导致数据包传输延迟或丢失，从而导致网络吞吐量下降

拥塞控制 (congestion control)

需要确保通信子网能够承载用户提交的通信量，是一个全局性问题，涉及主机、路由器等多种因素

产生拥塞的原因

- 主机发送到网络的数据包数量过多，超过了网络的承载能力
- 突发的流量填满了路由器的缓冲区，造成某些数据包会被丢弃

拥塞控制的基本策略

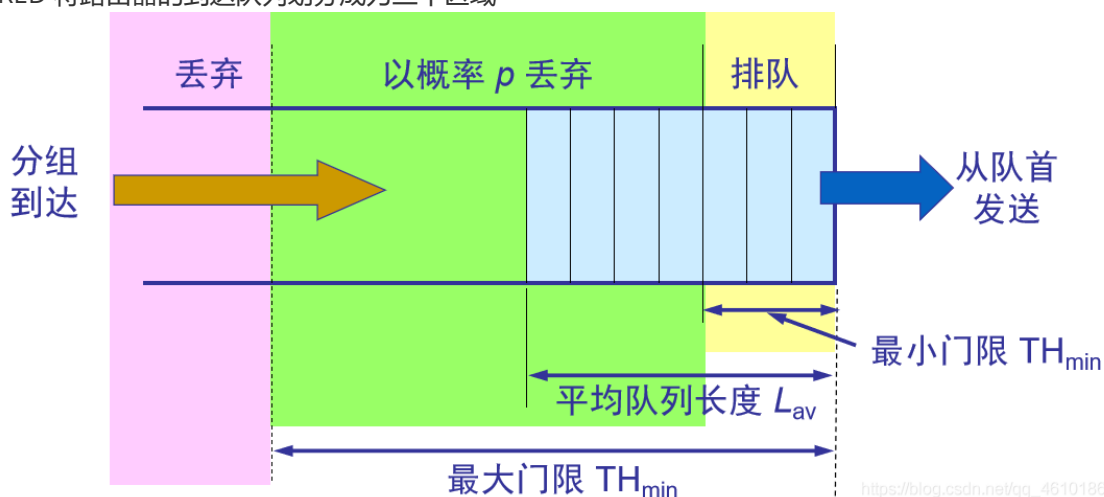
- **开环控制**---事先对通信流参数进行协商，协商后，不管网络是拥塞还是带宽充足，参数不能动态改变
 - 开环控制属于预防性拥塞控制，它竭力使网络总是处于无拥塞状态运行。开环控制的方法包括决定什么时候接受新流量，什么时候丢弃数据包和丢弃哪些数据包。其缺点是没有考虑网络的当前状态
- **闭环控制**---根据网络状态进行**动态控制**，包括两部分：**反馈机制**和**控制机制**。闭环控制方法分为两个子类：**显式反馈**与**隐式反馈**

7.2 流量调节

- **抑制包(Choke Packets)**: 用于通知发送方减小发送量，路由器选择一个被拥塞的数据包，给该数据包的源主机返回一个抑制包，抑制包中的目的地址取自该拥塞数据包。源主机收到抑制包后，减少发向特定目的地址的流量
- **逐跳的抑制包(Hop-by-Hop Choke Packets)**: 在高速或长距离网络中，由于源主机响应太慢，抑制包算法对拥塞控制的效果并不好，可采用逐跳抑制方法；其核心思想是抑制包对它经过的每个路由器都起作用，能够迅速缓解发生拥塞处的拥塞，但要求上游路由器有更大的缓冲区
- **显式拥塞通告 (ECN, Explicit Congestion Notification)**，在IP包头中记录数据包是否经历了拥塞。在数据包转发过程中，路由器可以在包头中标记为经历拥塞，然后接收方在它的下一个应答数据包里显示该标记作为显式拥塞信号
- **RFC2474中重新定义TOS域为包含一个6位的区分服务码点(DSCP) 和2位未使用位；RFC3168重新定义RFC2474中TOS域未使用的两位为ECN域**，包含如下值：
 - 00: 发送主机不支持ECN
 - 01或者10: 发送主机支持ECN
 - 11: 路由器正在经历拥塞

7.3 随机早期检测RED (Random Early Detection)

- 使路由器的队列维持两个参数，即队列长度最小门限 TH_{min} 和最大门限 TH_{max}
- RED 对每一个到达的数据报都先计算平均队列长度 L_{av}
- 若平均队列长度小于最小门限 TH_{min} ，则将新到达的数据报放入队列进行排队
- 若平均队列长度超过最大门限 TH_{max} ，则将新到达的数据报丢弃
- 若平均队列长度在最小门限 TH_{min} 和最大门限 TH_{max} 之间，则按照某一概率 p 将新到达的数据报丢弃
- RED 将路由器的到达队列划分成为三个区域



丢弃概率 p 与 Th_{min} 和 Th_{max} 的关系

- 当 $LAV < Th_{min}$ 时, 丢弃概率 $p = 0$
- 当 $LAV > Th_{max}$ 时, 丢弃概率 $p = 1$
- 当 $Th_{min} < LAV < Th_{max}$ 时, $0 \leq p \leq 1$
例如, 按线性规律变化, 从 0 变到 p_{max}

8.服务质量

8.1概述

问题的提出

- 互联网本身只能提供“尽力而为的服务”或称“尽最大努力交付的服务”
- 当互联网越来越多的用于传输多媒体信息时, 由于这些实时业务对网络的传输延时、延时抖动等特性较为敏感, 这样网络的传输质量就难以保障了
- IP网络不能保证特定业务的QoS要求, 已经成为IP网络发展的巨大障碍
- 网络的服务质量越来越多的引起人们的关注, 甚至成为网络技术研究的热点问题

什么是网络服务质量? (QoS, Quality of Service)

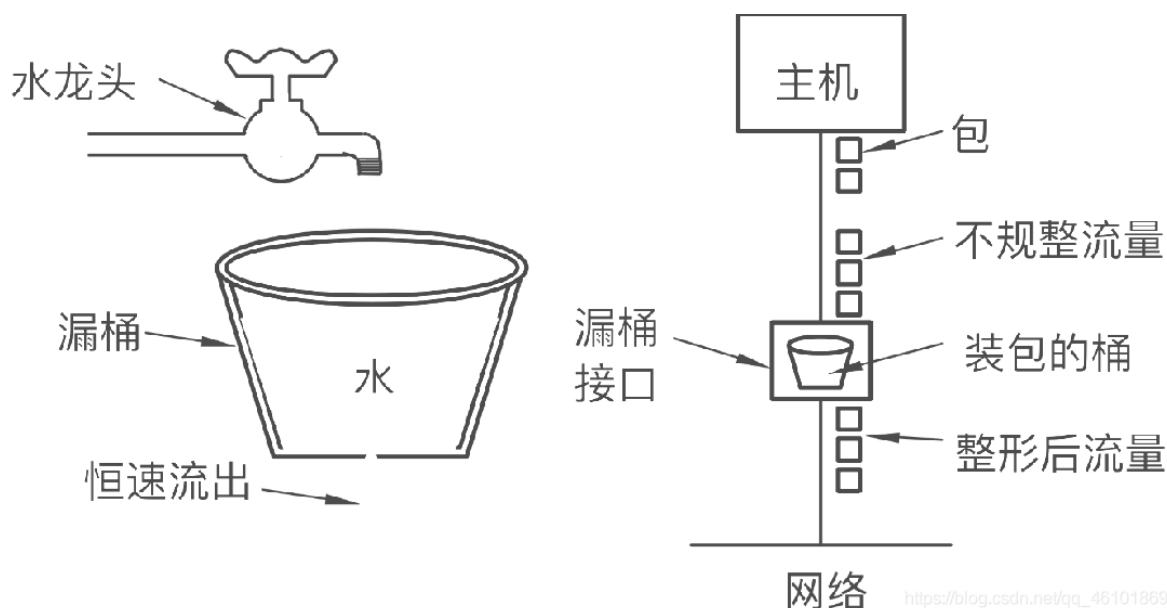
QoS是网络在传输数据流时要满足一系列服务请求, 具体可以量化为**带宽、时延、抖动、丢包率**等性能指标

8.2 流量整形

流量整形(traffic shaping): **其作用是限制流出某一网络的某一连接的流量与突发, 使这类报文以比较均匀的速度向外发送**

①漏桶算法

- 到达的数据包(网络层的PDU)被放置在底部具有漏孔的桶中(数据包缓存)
- 漏桶最多可以排队 b 个字节, 漏桶的这个尺寸受限于内存。如果数据包到达的时候漏桶已经满了, 那么数据包应被丢弃
- 数据包从漏桶中漏出, 以常量速率(r 字节/秒)注入网络, 因此平滑了突发流量



②令牌桶算法

- 产生令牌：周期性的以速率 r 向令牌桶中增加令牌，桶中的令牌不断增多。如果桶中令牌数已到达上限，则丢弃多余令牌
- 消耗令牌：输入数据包会消耗桶中的令牌。在网络传输中，数据包的大小通常不一致。大的数据包相较于小的数据包消耗的令牌要多
- 判断是否通过：输入数据包经过令牌桶时存在两种可能：输出该数据包或者被丢弃。当桶中的令牌数量可以满足数据包对令牌的需求，则将数据包输出，否则将其丢弃

注：漏桶算法和令牌桶算法的区别在于漏桶算法输出的流量永远不可能超过一定限制，而令牌桶算法可以容忍短时间内的高流量输出，对于一些突发流量的需求比较友好

8.3 数据包调度

在同一个流的数据包之间以及在竞争流之间**分配路由器资源的算法称为包调度算法**，它负责分配带宽和其他路由器资源，负责确定把缓冲区中的哪些数据包发送到输出链路上

- 先来先服务FCFS (First-Come First-Serve)
- 公平队列算法 (Fair Queueing/Round Robin)
- 加权公平队列算法 (Weighted Fair Queueing)
- 优先级调度 (Priority Scheduling)

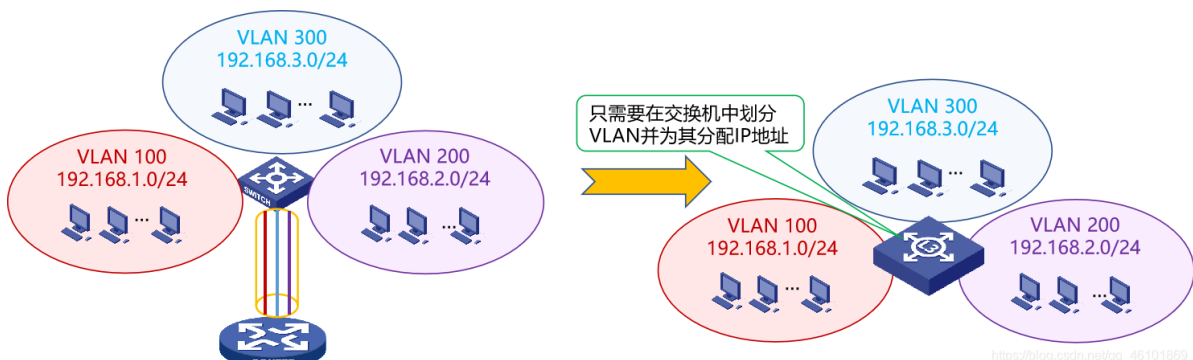
9.三层交换

9.1 三层交换的技术背景

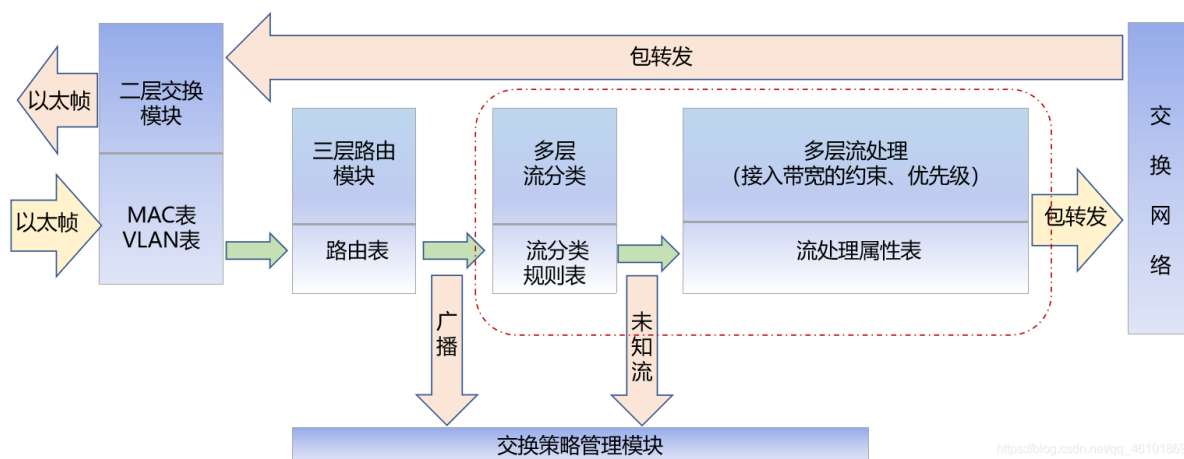
- 二层交换网络中的广播，限制了网络规模的扩展
 - 交换机对目标地址无法匹配的数据帧进行广播转发
 - 交换机对目标地址为广播地址的数据帧进行广播转发
 - 交换机为维护生成树状态产生大量的桥协议数据单元(Bridge Protocol Data Unit, BPDU)
 - 这些广播帧会大量消耗网络资源，并频繁影响用户的数据通信
- VLAN虽然可以将广播的影响限定在一定范围内，但同时也隔离了正常的用户间数据通信
- 传统路由器致力于解决VLAN间互联互通，但是其转发效率和拓扑复杂性带来的网络通信瓶颈无法有效应对规模扩展

9.2 三层交换的动机

- 利用第三层协议中的信息来加强第二层交换功能，形成带有路由功能的交换
- 融合VLAN间的二层隔离和三层互通，消除大规模网络中广播对性能的影响
- 简化网络配置，简化网络拓扑，优化网络管理，降低网络部署成本



9.3 三层交换机的工作原理



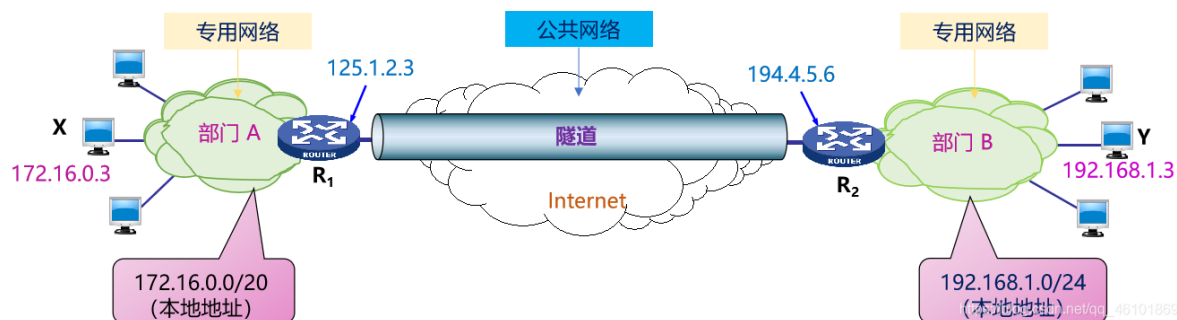
10.虚拟专用网VPN (Virtual Private Network)

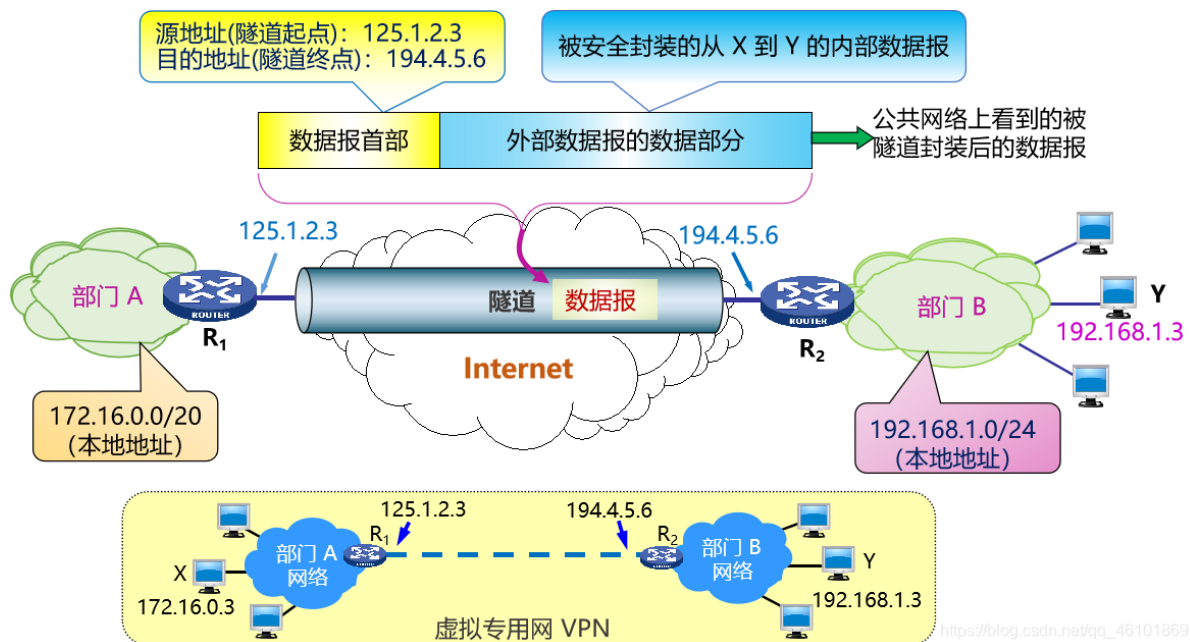
10.1 VPN的技术背景

- 计算机网络按用途分类
 - 专用、公用
- 专用网络的实现方式
 - 使用专用链路：铺设、租用（费用高，不灵活）
 - 使用公共链路：不安全
- 虚拟专用网VPN (Virtual Private Network)
 - 专用网络的经济、可靠、灵活的解决方案
 - 利用安全隧道技术将专用网络在公共网络上扩展
- VPN的设计原则
 - 安全性、隧道与加密、数据验证、用户验证、防火墙与攻击检测

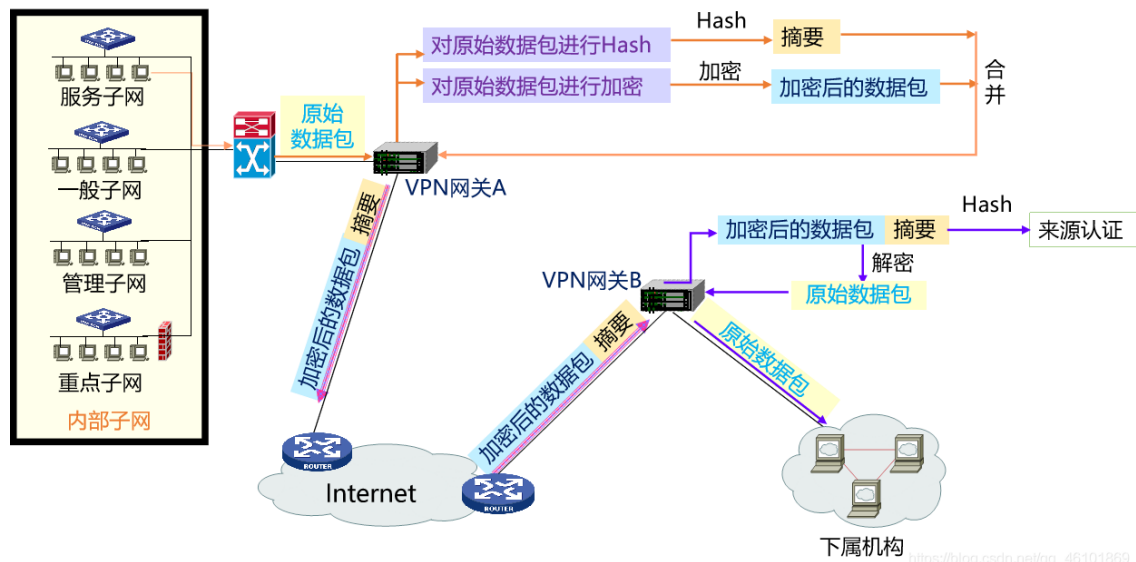
10.2 VPN的原理

- VPN指利用公用网络架设专用网络的远程访问技术
- VPN通过隧道技术在公共网络上模拟出一条点到点的逻辑专线，从而达到安全数据传输的目的



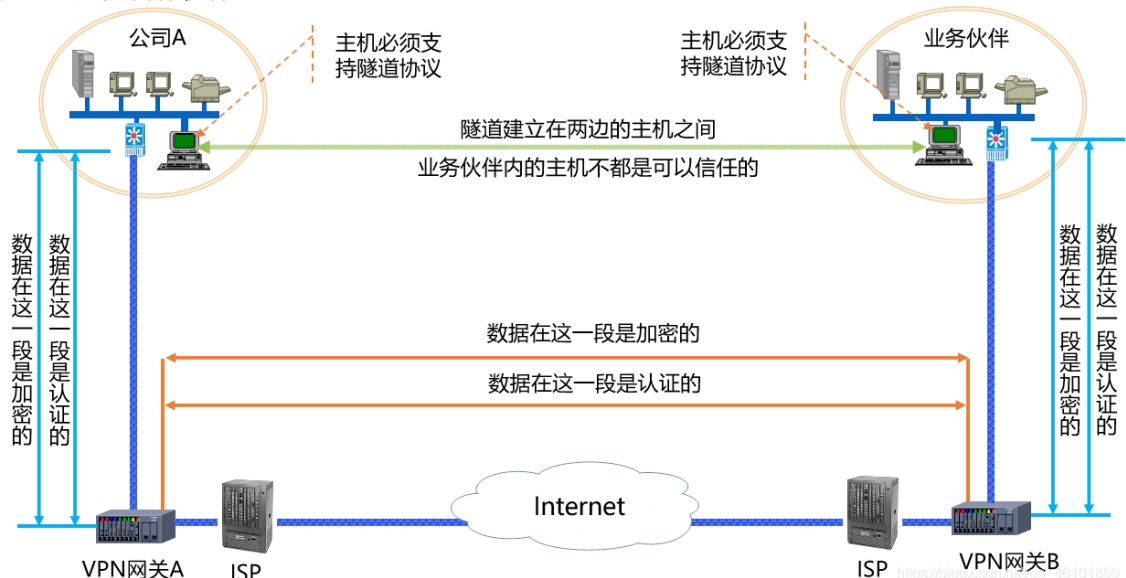


- VPN对数据机密性和完整性的保护

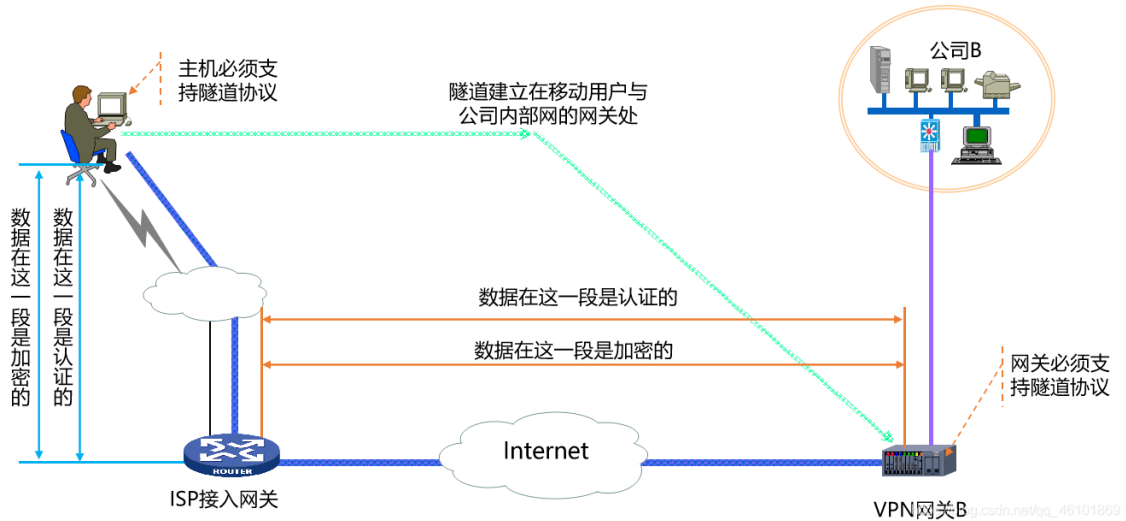


10.3 VPN的实现方式

- 用VPN连接合作伙伴



- 用VPN实现专用网络的远程访问



11. IPv6协议

初始动机：应付“32-bit地址空间耗尽”问题（CIDR和NAT都无法从根本上解决地址短缺问题），增加地址空间

IPv6 地址

- 地址长度为128bit，是IPv4地址长度的4倍
- IPv6地址空间数量约为 3×10^{38}
- IPv6地址表示法，冒分十六进制， $x:x:x:x:x:x:x:x$
 - 简化方法：每个x前面的0可省略，可把连续的值为0的x表示为“::”，且“::”只能出现1次
 - 简化前地址，2001:0DA8:0000:0000:200C:0000:0000:00A5
 - 简化后地址，2001:DA8:0000:0000:200C::A5

五、传输层

1.传输层概述

- 传输层位于应用层和网络层之间：
 - 基于网络层提供的服务，向分布式应用程序提供通信服务
- 按照因特网的“端到端”设计原则：
 - **应用程序只运行在终端上**，即不需要为网络设备编写程序
- 站在应用程序的角度：
 - 传输层应提供**进程之间本地通信的抽象**：即运行在不同终端上的应用进程仿佛是直接连在一起的

1.1 套接字

- 设想在应用程序和网络之间存在一扇“门”：
 - 需要发送报文时：发送进程将报文推到门外
 - 门外的运输设施（因特网）将报文送到接收进程的门口
 - 需要接收报文时：接收进程打开门，即可收到报文
- 在TCP/IP网络中，这扇“门”称为**套接字 (socket)**，是**应用层和传输层的接口**，也是应用程序和网络之间的API

1.2 传输层提供的服务

- 因特网的网络层提供“尽力而为”的服务：
 - 网络层尽最大努力在终端间交付分组，但不提供任何承诺
 - 具体来说，不保证交付，不保证按序交付，不保证数据完整，不保证延迟，不保证带宽等
- 传输层的**有所为、有所不为**：
 - 传输层可以通过差错恢复、重排序等手段提供可靠、按序的交付服务
 - 但传输层无法提供延迟保证、带宽保证等服务

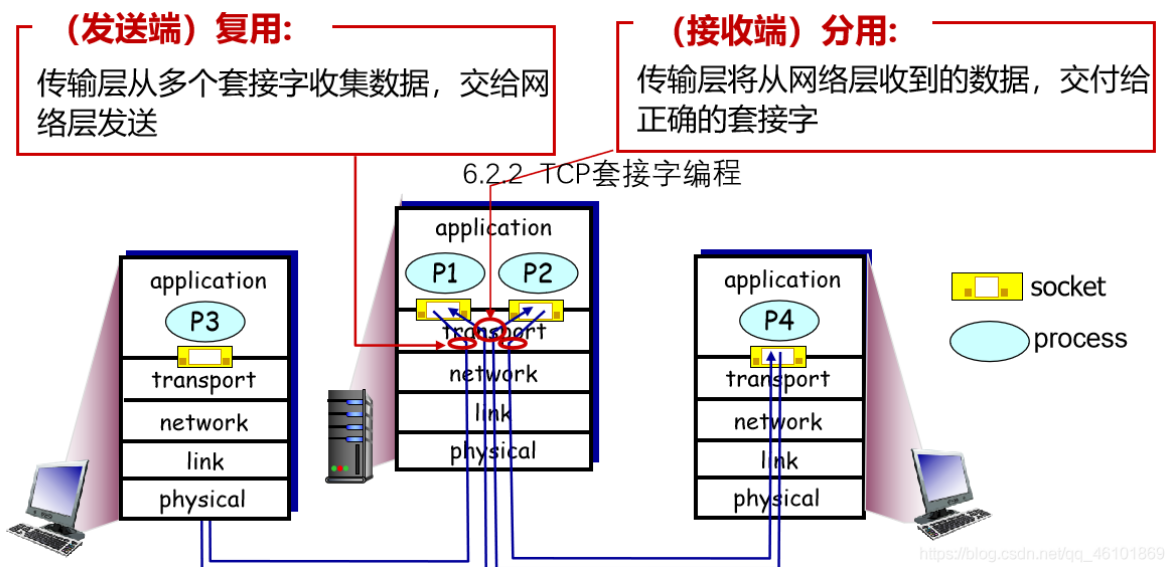
1.3 因特网传输层提供的服务

- 最低限度的传输服务：
 - 将终端-终端的数据交付扩展到进程-进程的数据交付
 - 报文检错
- 增强服务：
 - 可靠数据传输
 - 流量控制
 - 拥塞控制
- 因特网传输层通过UDP协议和TCP协议，向应用层提供两种不同的传输服务：
 - UDP协议：仅提供最低限度的传输服务
 - TCP协议：提供基础服务和增强服务

2.传输层基本服务——复用和分用

2.1 复用和分用概述

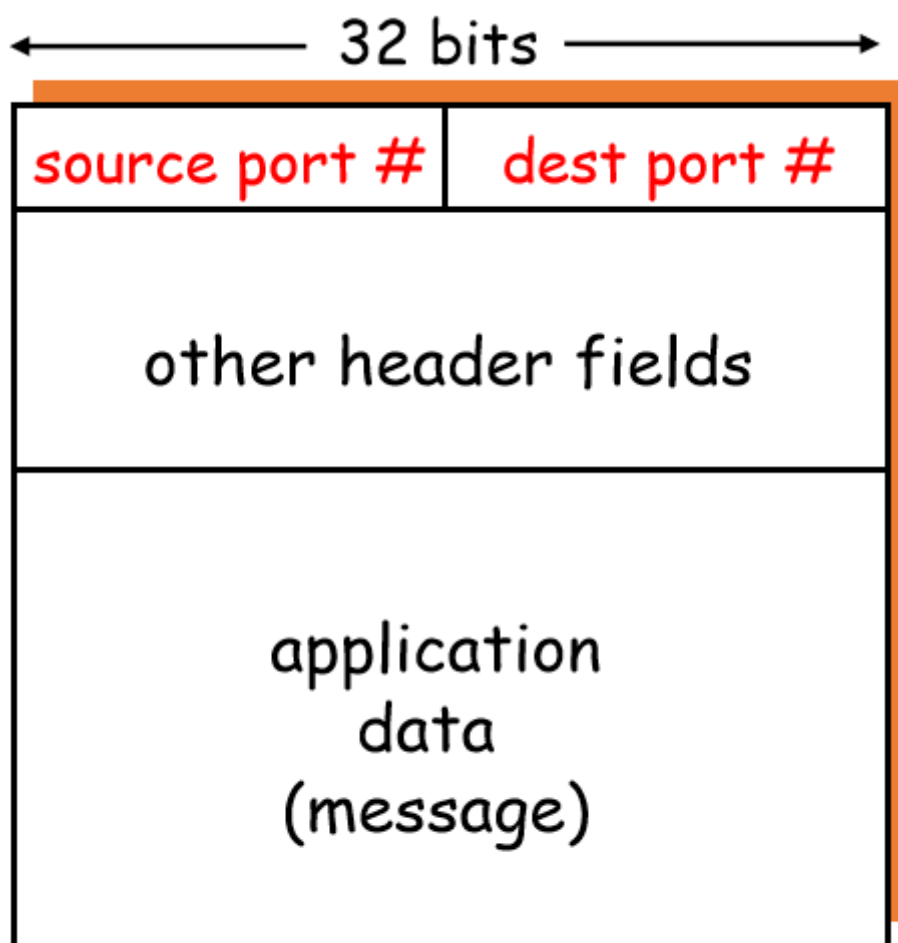
传输层基本服务：将主机间交付扩展到进程间交付，通过复用和分用实现



- **复用:**
发送方传输层将套接字标识置于报文段中，交给网络层
- **分用:**
接收方传输层根据报文段中的套接字标识，将报文段交付到正确的套接字

2.2 套接字标识与端口号

- 端口号是套接字标识的一部分：
 - 每个套接字在本地关联一个端口号
 - 端口号是一个16比特的数
- 端口号的分类：
 - 熟知端口：0 ~ 1023，由公共域协议使用
 - 注册端口：1024 ~ 49151，需要向IANA注册才能使用
 - 动态和/或私有端口：49152 ~ 65535，一般程序使用
- 报文段中有两个字段携带端口号
 - 源端口号：与发送进程关联的本地端口号
 - 目的端口号：与接收进程关联的本地端口号



TCP/UDP报文段格式

2.3 TCP/UDP套接字（复用和分用）

UDP套接字

- 使用<IP地址，端口号>二元组标识UDP套接字
- 服务器使用一个套接字服务所有客户

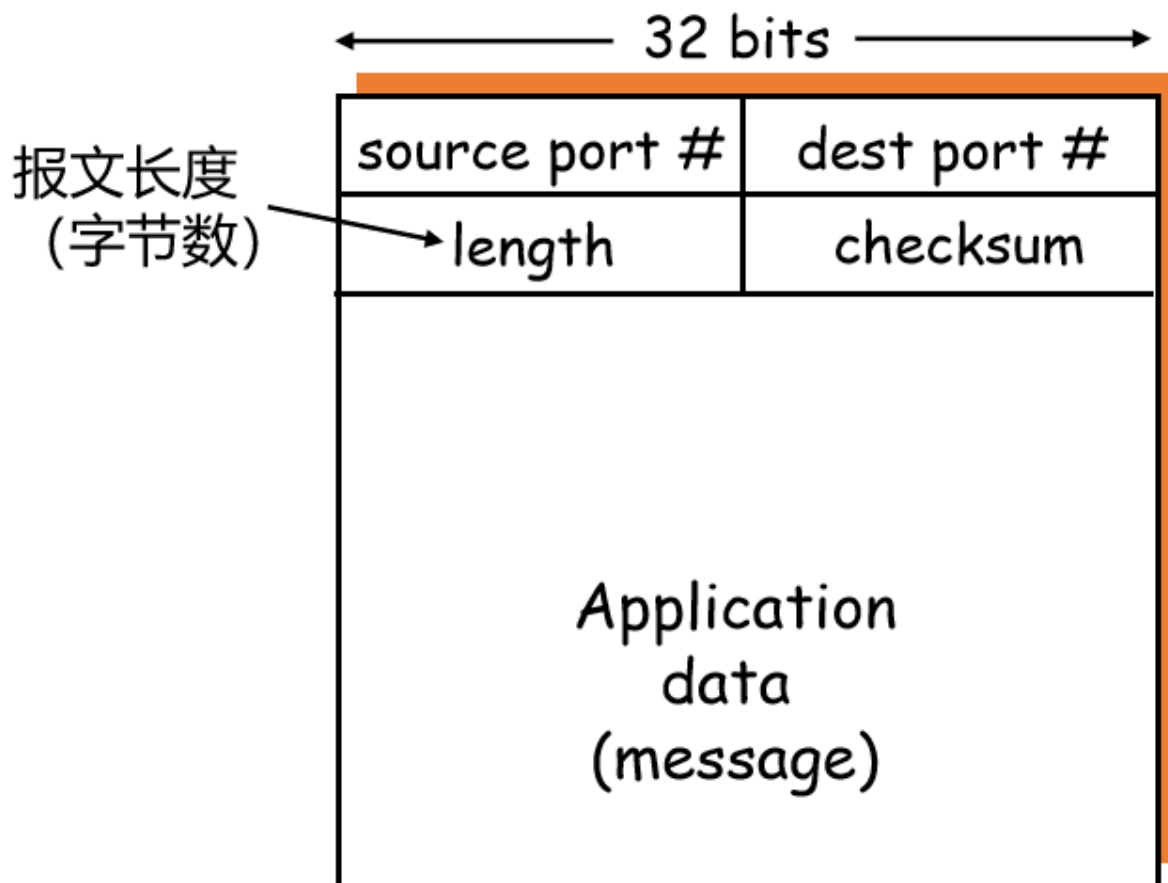
TCP套接字

- 使用<源IP地址，目的IP地址，源端口号，目的端口号> 四元组标识连接套接字
- 服务器使用一个监听套接字和多个连接套接字服务多个客户，每个连接套接字服务一个客户

3.无连接传输：UDP

3.1 UDP报文段结构

- UDP报文：
- 报头：携带协议处理需要的信息
 - 载荷（payload）：携带上层数据
- 用于复用和分用的字段：
 - 源端口号
 - 目的端口号
- 用于检测报文错误的字段：
 - 报文总长度
 - 校验和（checksum）



UDP报文格式

3.2 UDP校验和（checksum）

校验和字段的作用：对传输的报文段进行检错
以下是报文段的字段

key	human	hex
Source	192.168.1.106	c0a8 016a
Destination	11.111.111.111	0b6f 6f6f
Protocol	UDP(17)	11
Length	17	11
Source Port	63549	f83d
Destination Port	12345	3039
Length	17	11
Checksum	0xb12d	b12d
Data	hello UDP	6865 6c6c 6f20 5544 5000

将上表中所有的 16 进制数加起来，之后取反码。有一点需要注意的是，如果遇到最高位进位，那么需要对结果进行回卷，意思是

$$\begin{array}{r}
 1100\ 1101\ 1000\ 0001 \\
 0110\ 1011\ 0110\ 0101 \\
 \hline
 1\ 0011\ 1000\ 1110\ 0110 \\
 \downarrow \\
 0011\ 1000\ 1110\ 0110 \\
 1 \\
 \hline
 0011\ 1000\ 1110\ 0111
 \end{array}$$

求出来的即是校验和。

3.3 为什么需要UDP?

为什么需要UDP?

- 应用可以尽可能快地发送报文：
 - 无建立连接的延迟
 - 不限制发送速率（不进行拥塞控制和流量控制）
- 报头开销小
- 协议处理简单

UDP适合哪些应用?

- 容忍丢包但对延迟敏感的应用：
 - 如流媒体

- 以单次请求/响应为主的应用：
如DNS
- 若应用要求基于UDP进行可靠传输：
由应用层实现可靠性

4.面向连接的传输：TCP

4.1 可靠传输

数据包**有序、无差错**到达接收端

如何实现可靠传输，基本原则是什么？

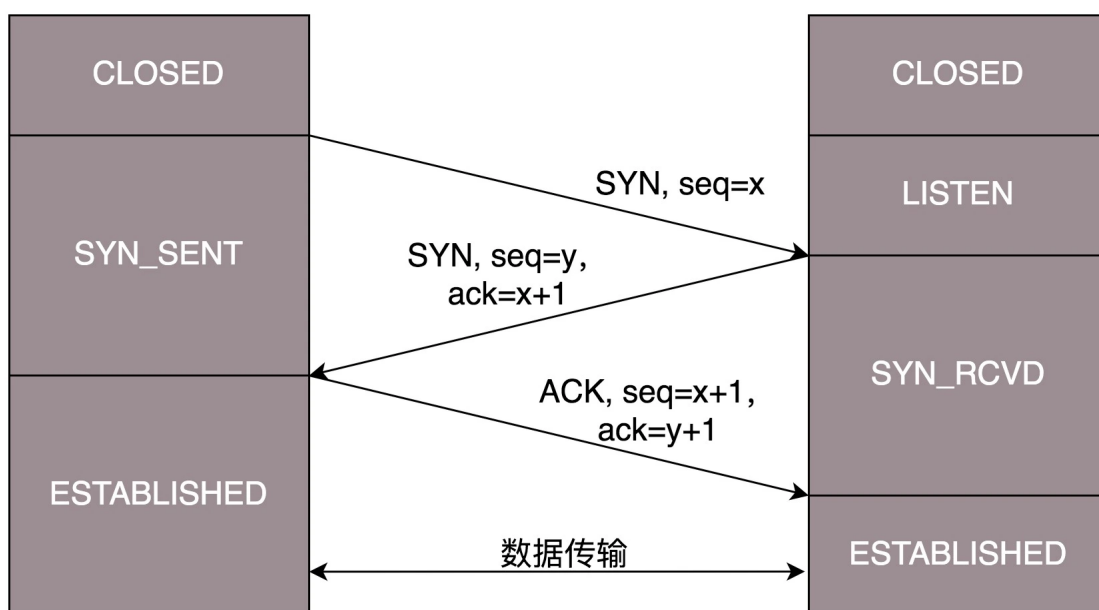
- 利用ACK确认
- 重传机制
- 差错检测

可靠传输实现举例-Stop and Wait

注：该实现俗称tcp的三次握手和四次挥手。三次挥手我们也常称为“请求 -> 应答 -> 应答之应答”的三个回合，主要就是为了建立连接

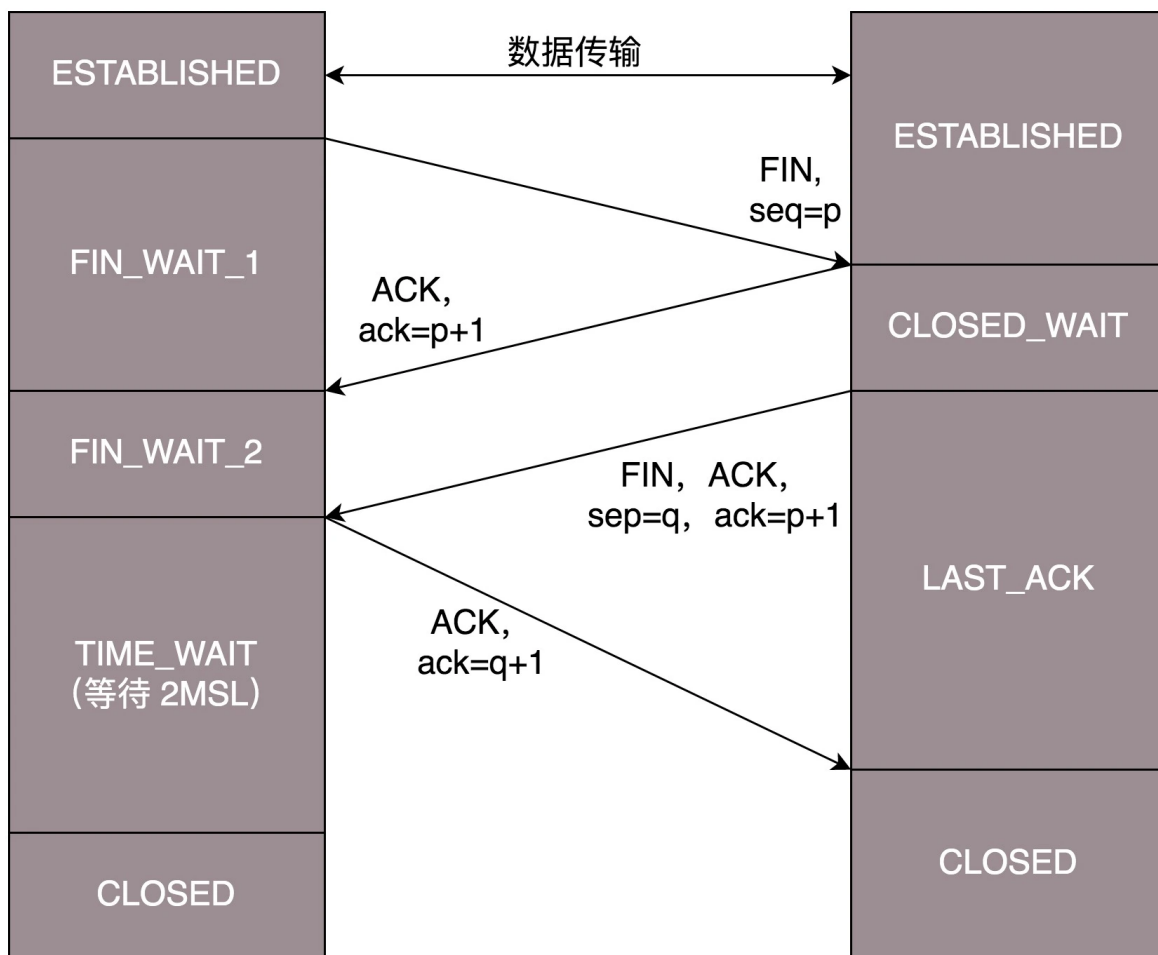
建立一条TCP连接需要确定两件事：

- 双方都同意建立连接（知晓另一方想建立连接）
- 初始化连接参数（序号，MSS等）



https://blog.csdn.net/qg_45101869

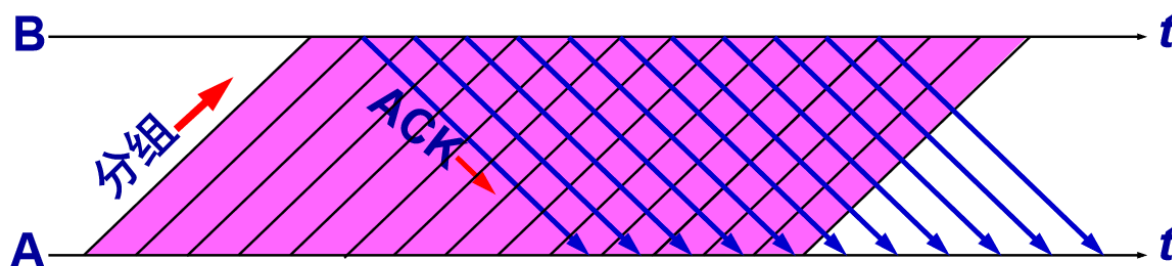
四次挥手：目的就是确保断开连接时双方都是确认结束的状态



https://blog.csdn.net/qq_48101869

4.2 流水线技术

由于信道上一直有数据不间断地传送，这种传输方式可获得很高的信道利用率。



流水线传输可提高信道利用率

https://blog.csdn.net/qq_48101869

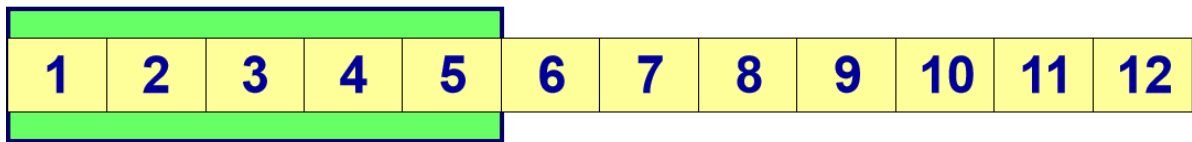
①GBN 协议 (回退 N 步协议)

发送窗口

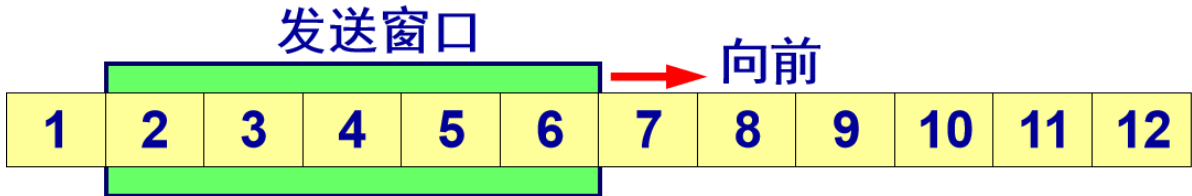
发送方维持的发送窗口，它的意义是：位于发送窗口内的分组都可连续发送出去，而不需要等待对方的确认。这样，信道利用率就提高了。

发送方每收到一个确认，就把发送窗口向前滑动一个分组的位置。

发送窗口



(a) 发送方维持发送窗口（发送窗口是 5）



(b) 收到一个确认后发送窗口向前滑动

累积确认

- 接收方一般采用累积确认的方式。即不必对收到的分组逐个发送确认，而是**对按序到达的最后一个分组发送确认**，这样就表示：**到这个分组为止的所有分组都已正确收到了**。
- 优点：容易实现，即使确认丢失也不必重传。
- 缺点：不能向发送方反映出接收方已经正确收到的所有分组的信息。

如果发送方发送了前 5 个分组，而中间的第 3 个分组丢失了。这时接收方只能对前两个分组发出确认。发送方无法知道后面三个分组的下落，而只好把后面的三个分组都再重传一次。

这就叫做 **Go-back-N（回退 N）**，表示**需要再退回来重传已发送过的 N 个分组**。

可见当通信线路质量不好时，连续 ARQ 协议会带来负面的影响。

②SR 协议（选择重传协议）

SR 协议在 GBN 协议的基础上进行了改进，它通过让发送方仅重传那些它怀疑在接收方出错（即丢失或受损）的分组而避免了不必要的重传。选择重传协议只重传真正丢失的分组。

- 选择重传的接收窗口与发送窗口一样大
- 选择重传协议允许与接收窗口一样多的分组失序到达,并保存这些失序到达的分组,直到连续的一组分组被交付给应用层。
- 标记发出分组,当 $ACK=S_f$ 时,将窗口滑过所有连续的已确认的分组
 - 如果还有未确认的分组,则重发所有检测到的未被确认的分组并重启计时器
 - 如果所有分组都被确认了则停止计时器
- 确认，确认号(ACK)只定义完好接收的那一个分组的序号,并不反馈任何其他分组的信息

注：SR协议ack确认和GBN的累计确认不同，是一个一个确认

计时器

理论上选择重传协议要为每个分组使用一个计时器。当某个计时器超时后,只有相应的分组被重传。换言之，返回N协议将所有的分组当做一个整体对待，而选择重传协议则分别对待每一个分组。但是大多数SR的运输层仅使用了一个计时器。注意只使用一个计时器而做到跟踪所有发出去的分组的情况的做法是：标记发出分组,当 $ACK=S_f$ 时,将窗口滑过所有连续的已确认的分组，如果还有未确认的分组，则重发所有检测到的未被确认的分组并重启计时器，如果所有分组都被确认了则停止计时器。

快速重传

- 仅靠超时重发丢失的报文段，恢复太慢！

- **发送方可利用重复ACK检测报文段丢失：**
 - 发送方通常连续发送许多报文段
 - 若仅有个别报文段丢失，发送方将收到多个重复序号的ACK
 - 多数情况下IP按序交付分组，重复ACK极有可能因丢包产生
- TCP协议规定：当发送方收到对同一序号的3次重复确认时，立即重发包含该序号的报文段
- 所谓快速重传，就是在**定时器到期前重发丢失的报文段**

③TCP 可靠通信的具体实现

- TCP 连接的每一端都必须设有两个窗口——一个**发送窗口**和一个**接收窗口**。
- TCP 的可靠传输机制用**字节的序号**进行控制。TCP 所有的确认都是基于序号而不是基于报文段。
- TCP 两端的四个窗口经常处于**动态变化**之中。
- TCP连接的往返时间 RTT 也不是固定不变的。需要使用特定的算法**估算较为合理的重传时间**

GBN、SR和TCP小结

Go-Back-N协议

接收方：

- 使用累积确认
- **不缓存失序的分组**
- 对失序分组发送重复ACK

发送方：

- 超时后重传从基序号开始的所有分组

SR

接收方：

- 缓存失序的分组
- 单独确认每个正确收到的分组

发送方：

- 每个分组使用一个定时器
- 仅重传未被确认的分组

TCP协议

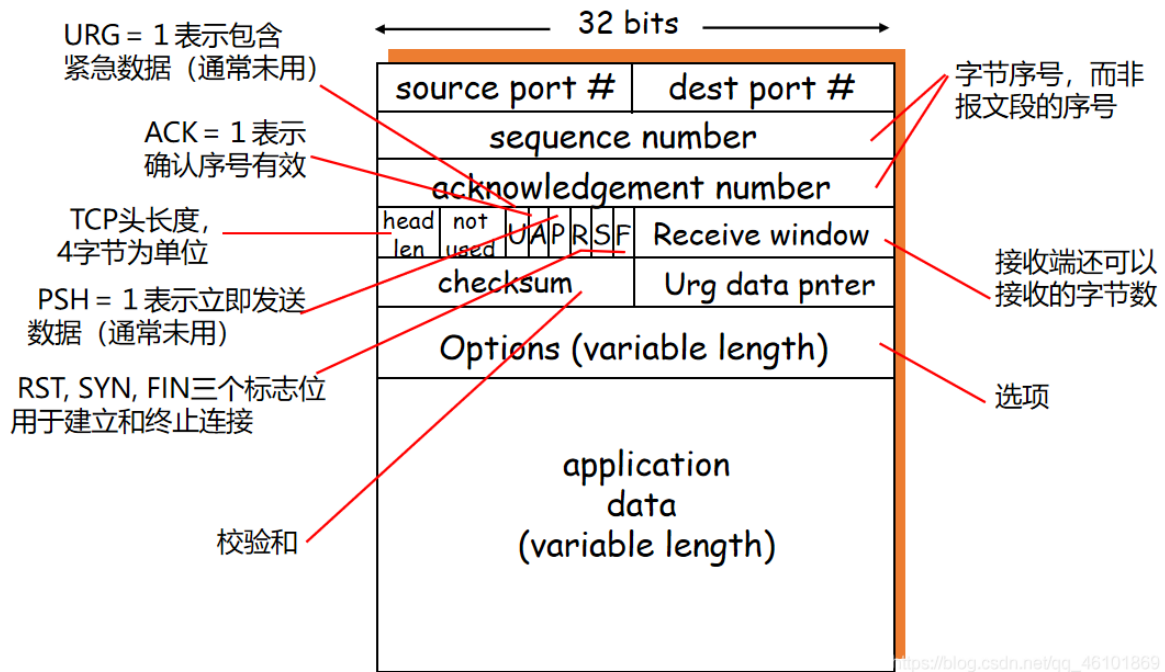
接收方：

- 使用累积确认
- **缓存失序的报文段**
- 对失序报文段发送重复ACK
- **增加了推迟确认**

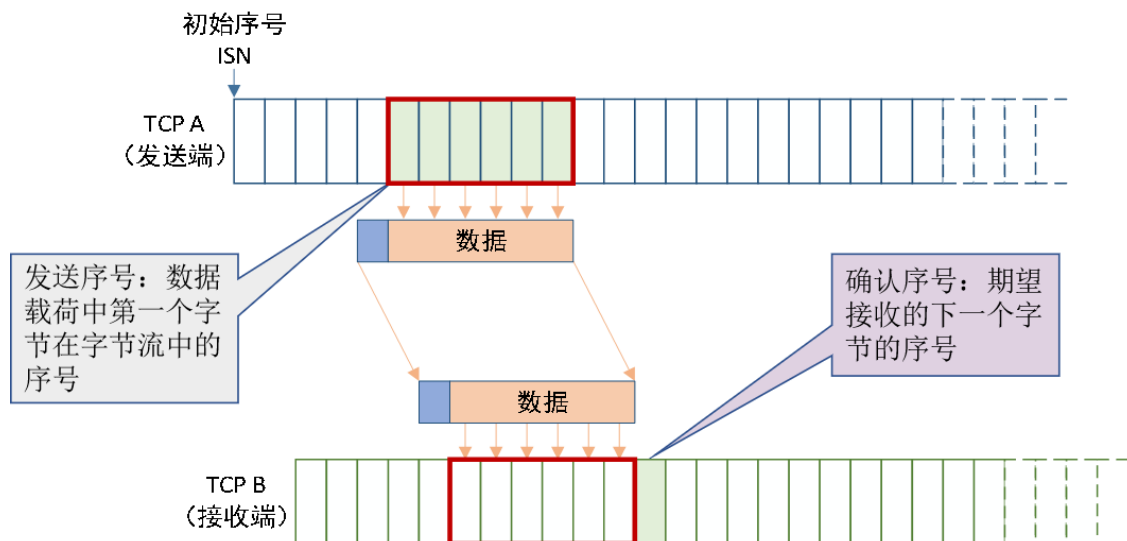
发送方：

- 超时后仅重传最早未确认的报文段
- **增加了快速重传**

4.3 TCP报文段结构



- 最大段长度 (MSS) :
 - TCP段中可以携带的最大数据字节数
 - 建立连接时, 每个主机可声明自己能够接受的MSS, 缺省为536字节
- 窗口比例因子 (window scale) :
 - 建立连接时, 双方可以协商一个窗口比例因子
 - 实际接收窗口大小 = window size * $2^{\text{window scale}}$
- 选择确认 (SACK) :
 - 最初的TCP协议只使用累积确认
 - 改进的TCP协议引入选择确认, 允许接收端指出缺失的数据字节
- 发送序号和确认序号的含义



初始序号的选取: 每个TCP实体维护一个32位计数器, 该计数器每4微秒增1, 建立连接时从中读取计数器当前值 (依赖具体实现, 见连接管理)

5.TCP流量控制

TCP接收端

- 使用显式的窗口通告, 告知发送方可用的缓存空间大小
- 在接收窗口较小时, 推迟发送确认

- 仅当接收窗口显著增加时，通告新的窗口大小

TCP发送端

- 使用Nagle算法确定发送时机
- 使用接收窗口限制发送的数据量，已发送未确认的字节数不超过接收窗口的大小

Nagle算法的解决方法：

- 在新建连接上，当应用数据到来时，组成一个TCP段发送（那怕只有一个字节）
- 在收到确认之前，后续到来的数据放在发送缓存中
- 当数据量达到一个MSS或上一次传输的确认到来（取两者的较小时间），用一个TCP段将缓存的字节全部发走

Nagle算法的优点：

- 适应网络延时、MSS长度及应用速度的各种组合 常规情况下不会降低网络的吞吐量

6.拥塞控制

发送方根据自己感知的网络拥塞程度，限制其发送速率

6.1 拥塞控制的类型

- 网络辅助的拥塞控制
 - 路由器向端系统提供显式的反馈，例如：
 - 设置拥塞指示比特
 - 给出发送速率指示
 - ATM、X.25采用此类方法
- 端到端拥塞控制
 - 网络层不向端系统提供反馈
 - 端系统通过观察丢包和延迟，自行推断拥塞的发生
 - TCP采用此类方法

6.2 TCP拥塞控制要解决的问题

发送方如何感知网络拥塞？

- 发送方利用丢包事件感知拥塞：
 - 拥塞造成丢包和分组延迟增大
 - 无论是丢包还是分组延迟过大，对于发送端来说都是丢包了
- 丢包事件包括：
 - 重传定时器超时
 - 发送端收到3个重复的ACK

发送方采用什么机制限制发送速率？

- 发送方使用拥塞窗口cwnd限制已发送未确认的数据量：

$\text{LastByteSent} - \text{LastByteAcked} \leq \text{cwnd}$

$$\text{rate} = \frac{\text{cwnd}}{\text{RTT}} \text{ Bytes/sec}$$

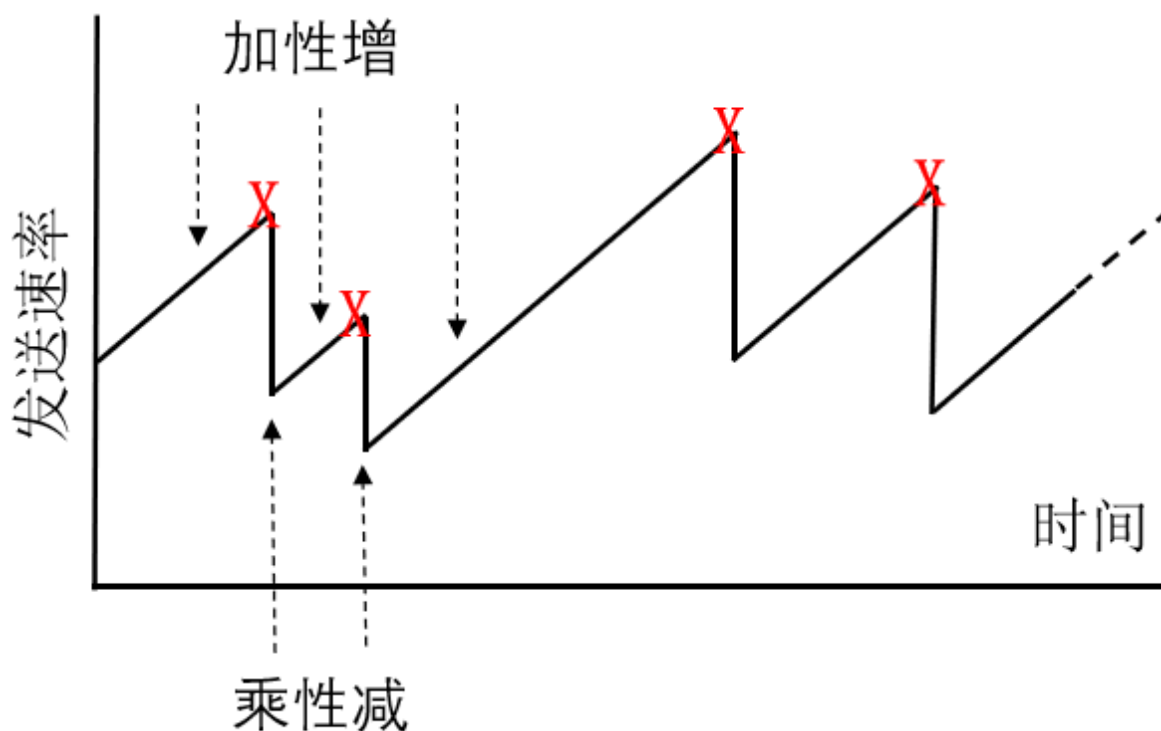
- cwnd随发送方感知的网络拥塞程度而变化

发送方感知到网络拥塞后，采取什么策略调节发送速率？

①AIMD

- 乘性减 (Multiplicative Decrease)
 - 发送方检测到丢包后，将cwnd的大小减半（但不能小于一个MSS）
 - 目的：迅速减小发送速率，缓解拥塞
- 加性增 (Additive Increase)
 - 若无丢包，每经过一个RTT，将cwnd增大一个MSS，直到检测到丢包
 - 目的：缓慢增大发送速率，避免振荡

注：MSS是发送速率TCP建立连接时双方确定的每一个报文段所能承载的最大数据长度



cwnd呈锯齿状变化

②TCP慢启动

- 慢启动的策略：
 - 每经过一个RTT，将cwnd加倍
- 慢启动的具体实施：
 - 每收到一个ACK段，cwnd增加一个MSS
 - 只要发送窗口允许，发送端可以立即发送下一个报文段
- 特点：
 - 以一个很低的速率开始，按指数增大发送速率

区分不同的丢包事件

超时：说明网络交付能力很差

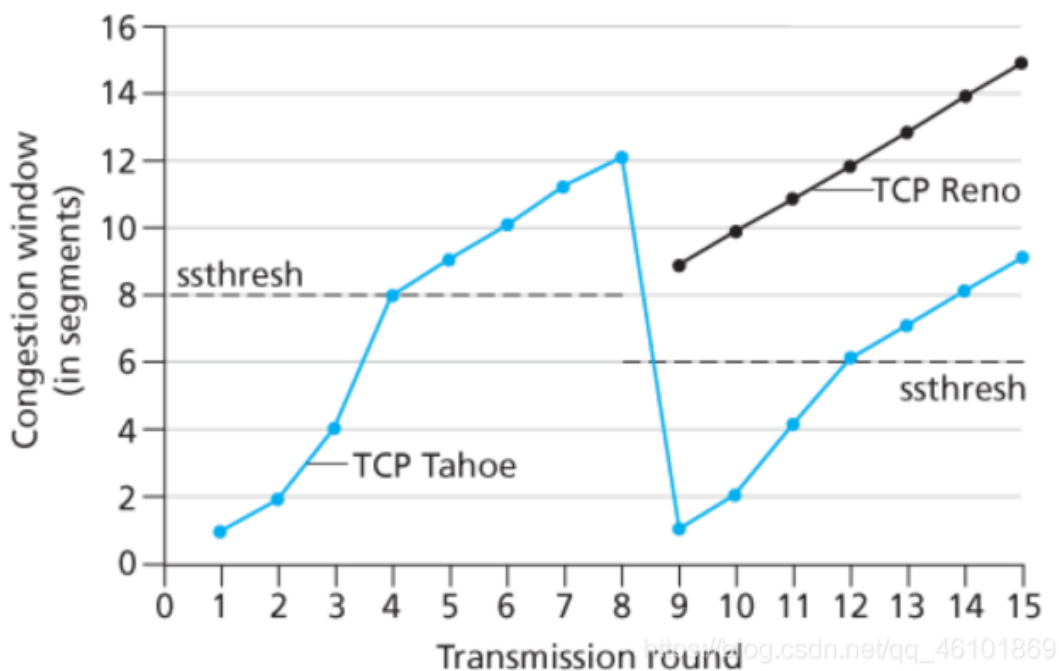
收到3个重复的ACK：说明网络仍有一定的交付能力

- 收到3个重复的ACK：
 - 将cwnd降至一半
 - 使用AIMD调节cwnd

- 超时：
 - 设置门限 $= \text{cwnd}/2$
 - $\text{cwnd} = 1\text{MSS}$
 - 使用慢启动增大 cwnd 至门限
 - 使用AIMD调节 cwnd

6.3 TCP拥塞控制的实现

- 发送方维护变量 ssthresh
- 发生丢包时, $\text{ssthresh} = \text{cwnd}/2$
- **ssthresh 是从慢启动转为拥塞避免的分水岭:**
 - cwnd 低于门限时, 执行慢启动
 - cwnd 高于门限: 执行拥塞避免
- 拥塞避免阶段, 拥塞窗口线性增长:
 - 每当收到ACK, $\text{cwnd} = \text{cwnd} + \text{MSS} * (\text{MSS}/\text{cwnd})$
- 检测到3个重复的ACK后:
 - TCP Reno实现: $\text{cwnd} = \text{ssthresh} + 3$, 线性增长
 - TCP Tahoe实现: $\text{cwnd} = 1\text{MSS}$, 慢启动



TCP发送端的事件与动作

State	Event	TCP Sender Action	Commentary
慢启动 (SS)	收到新的确认	$\text{cwnd} = \text{cwnd} + \text{MSS}$, If ($\text{cwnd} > \text{ssthresh}$) set state to "Congestion Avoidance"	每经过一个RTT, cwnd 加倍
拥塞避免 (CA)	收到新的确认	$\text{cwnd} = \text{cwnd} + \text{MSS} * (\text{MSS}/\text{cwnd})$	每经过一个RTT, cwnd 增加一个MSS
SS or CA	收到3个重复的确认	$\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = \text{ssthresh} + 3$, Set state to "Congestion Avoidance"	cwnd 减半, 然后线性增长
SS or CA	超时	$\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = 1\text{MSS}$, Set state to "Slow Start"	cwnd 降为一个MSS, 进入慢启动
SS or CA	收到一个重复的确认	统计收到的重复确认数	cwnd 和 ssthresh 都不变

六、应用层

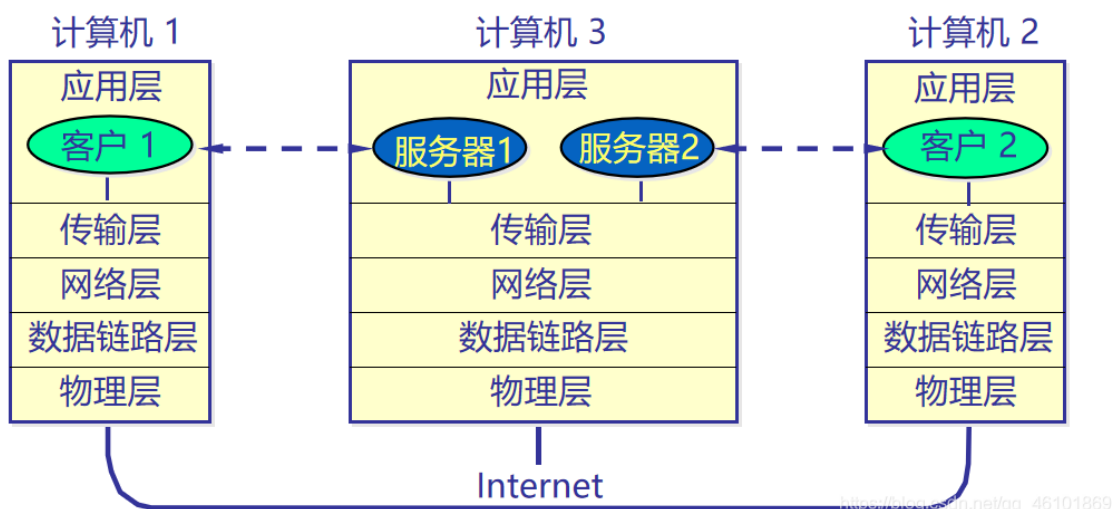
1.应用层概述

- 每个应用层协议都是为了解决某一应用问题，通过位于不同主机中的多个应用进程之间的通信和协同工作来完成
 - 两台主机通信实际是其对应的两个应用进程(process)在通信
 - **应用进程**: 为解决具体应用问题而彼此通信的进程
- 应用层的具体内容就是规定应用进程在通信时所遵循的协议
 - 客户/服务器 (C/S, Client/Server) 方式
 - 对等 (P2P, Peer to Peer) 方式

1.1 应用进程通信方式

①客户/服务器 (C/S, Client/Server) 方式

- 应用层的许多协议是基于C/S方式，例如，在移动互联网环境下，每个应用APP都是一个客户端
 - 客户(client)和服务端(server)是指通信中所涉及的2个**应用进程**
 - **客户/服务器**方式描述的是**应用进程之间服务和被服务的关系**
 - **客户**是服务请求方（主动请求服务，被服务）
 - **服务器**是服务提供方（被动接受服务请求，提供服务）
- C/S方式可以是面向连接的，也可以是无连接的
- **面向连接时，C/S通信关系一旦建立，通信就是双向的**，双方地位平等，都可发送和接收数据



客户进程的特点

- 在进行通信时临时成为客户，它也可在本地进行其它的计算
- 用户计算机上运行，在打算通信时**主动**向远地服务器发起通信
- **客户方必须知道服务器进程所在主机的IP地址才能发出服务请求**
- 需要时可以与多个服务器进行通信

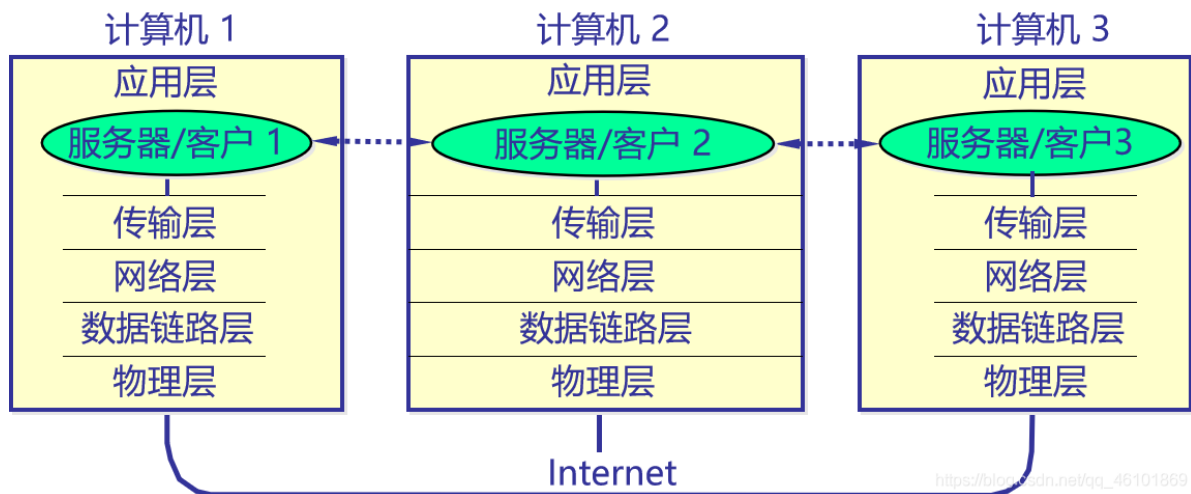
服务器进程的特点

- 专门用来提供某种服务的程序，可“同时”处理多个远地或本地客户的请求
- **必须始终处于运行状态才有可能提供服务**
- 通信开始之前服务器进程不需要知道客户进程所在主机的IP地址，无论客户请求来自哪里，服务器进程被动等待服务请求的到来即可
- 通常是当系统启动时即自动调用并一直运行着。某些服务器程序也可以由用户或其它进程在通信前启动

- **被动等待**并接受来自多个客户的通信请求

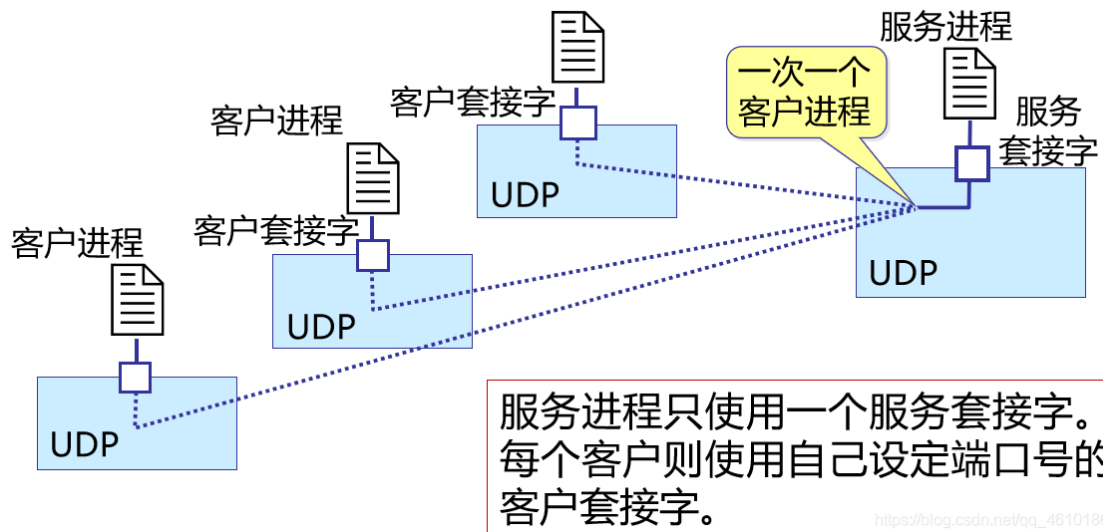
②对等（P2P，Peer to Peer）方式

- **对等方式**是指两个进程在通信时并不区分服务的请求方和服务的提供方
 - 只要两个主机都运行P2P软件，它们就可以进行平等、对等的通信
 - 双方都可以下载对方存储在硬盘中的共享文档，如果权限允许的话
- 音频/视频应用推动了P2P对等通信方式的发展（BitTorrent）
- 音频/视频流量已占主要比
- P2P方式从本质上看仍然是使用了C/S方式，但强调的是通信过程中的对等，这时**每一个P2P进程既是客户同时也是服务器**



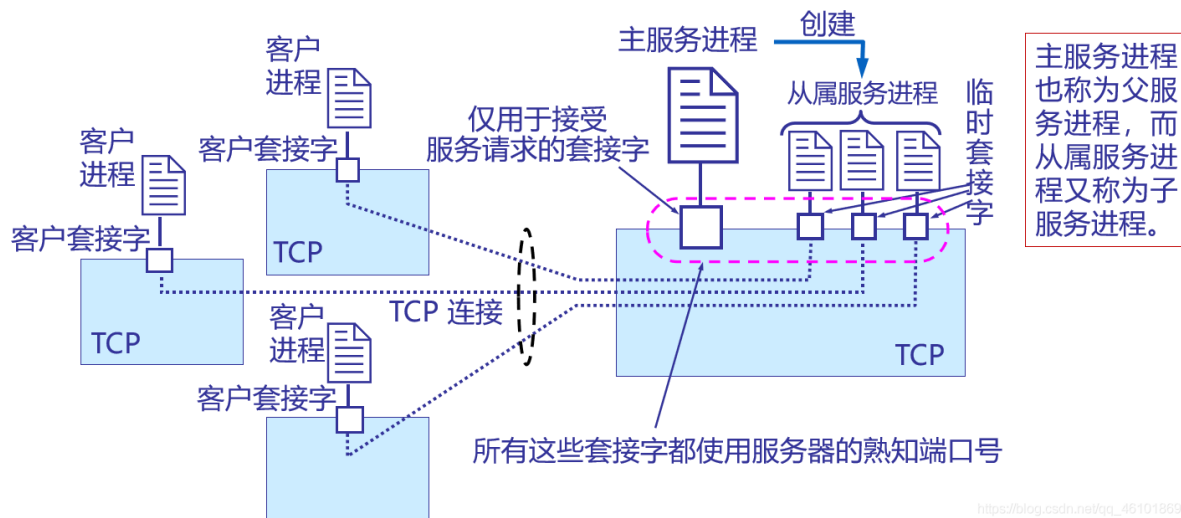
1.2 服务器进程工作方式

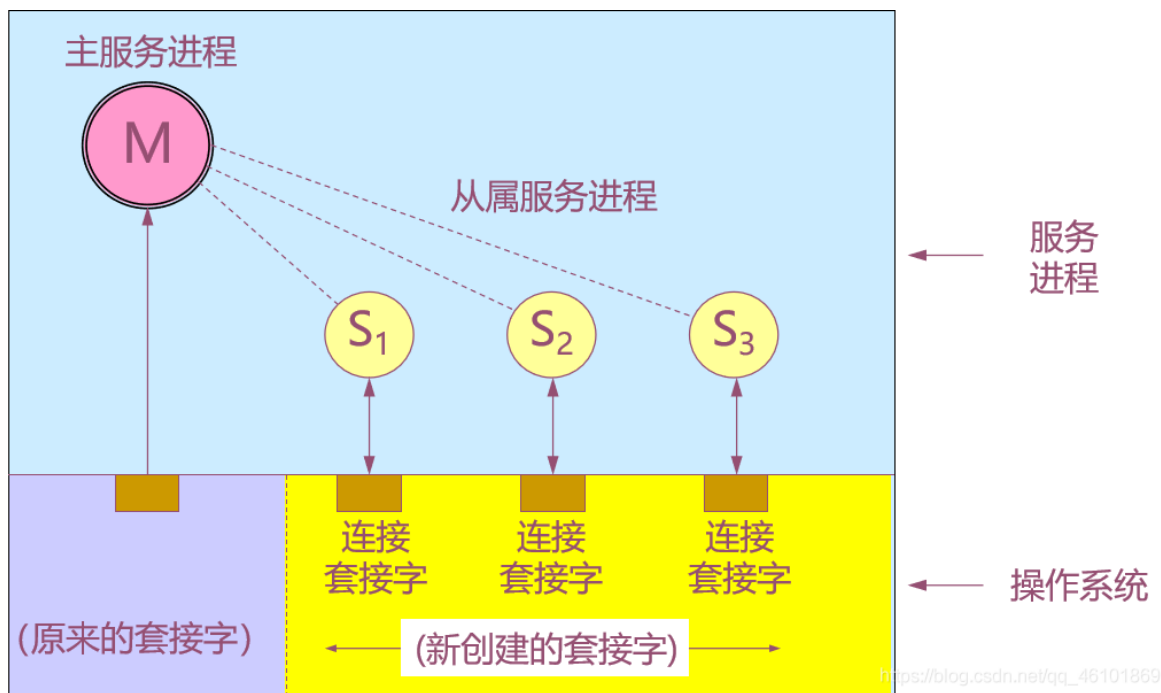
- 循环方式(iterative mode)
 - 一次只运行一个服务进程
 - 当有多个客户进程请求服务时，服务进程就按请求的先后顺序依次做出响应 (阻塞方式)
- 并发方式(concurrent mode)
 - 可以同时运行多个服务进程
 - 每一个服务进程都对某个特定的客户进程做出响应 (非阻塞方式)
- 无连接循环方式服务
 - 使用无连接的UDP服务进程通常都工作在循环方式，即一个服务进程在同一时间只能向一个客户进程提供服务。(顺序服务)
 - 服务进程收到客户进程的请求后，就发送UDP用户数据报响应该客户
 - 对其他客户进程发来的请求则暂时不予理睬，这些请求都在服务端的队列中排队等候服务进程的处理
 - 当服务进程处理完毕一个请求时，就从队列中读取来自下一个客户进程的请求，然后继续处理



- 面向连接的并发方式服务

- 面向连接的TCP服务进程通常都工作在并发服务方式，服务进程在同一时间可同时向多个客户进程提供服务。(并发服务)
- 在TCP服务进程与多个客户进程之间必须建立多条TCP连接，每条TCP连接在其数据传送完毕后释放
- 一个TCP连接对应一个（熟知）服务端口
- 主服务进程在熟知端口等待客户进程发出的请求。一旦收到客户的请求，就创建一个从属服务进程，并指明从属服务进程使用临时套接字与该客户建立TCP连接，然后主服务进程继续在原来的熟知端口等待向其他客户提供服务

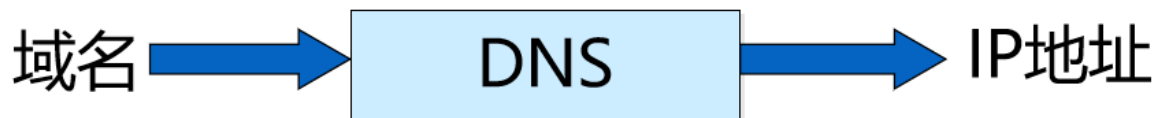




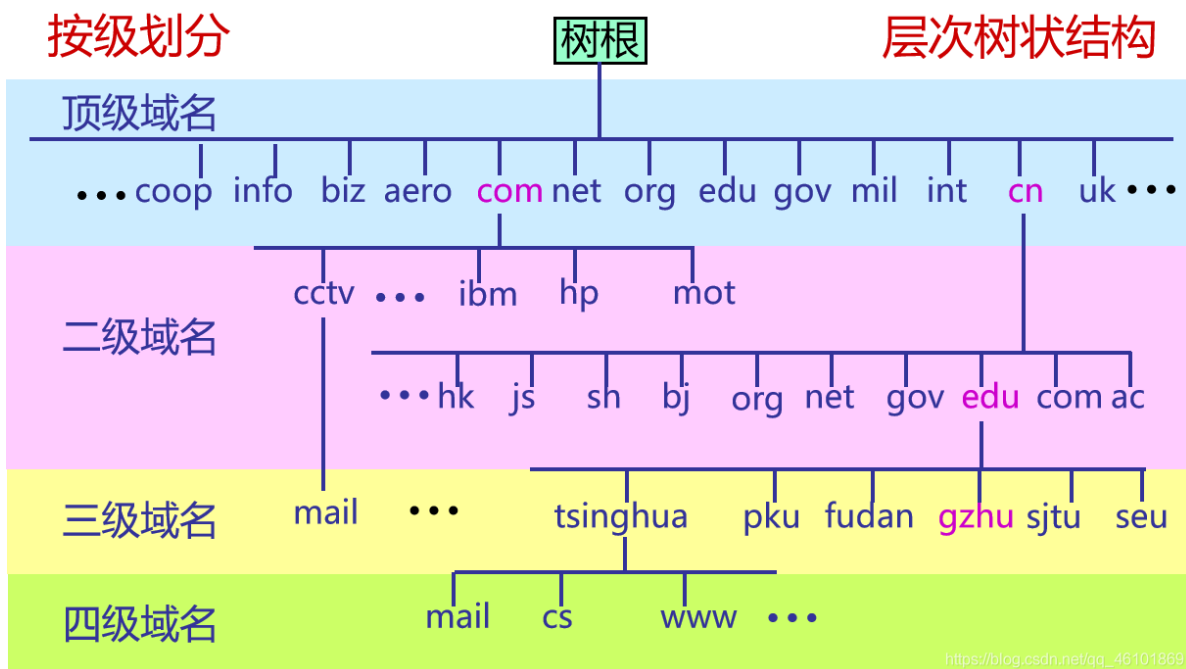
2.域名系统

2.1 概述

- **域名系统** (DNS, Domain Name System) 是互联网重要的基础设施之一，向所有需要域名解析的应用提供服务，主要负责将**可读性好的域名映射成IP地址**
- Internet采用**层次结构的命名树**作为主机的名字，并使用**分布式**的域名系统 DNS。Internet的DNS是一个**联机分布式数据库系统**
- 名字到域名的解析是由**若干个**域名服务器程序完成的。域名服务器程序在专设的结点上运行，相应的结点也称为**名字服务器**(Name Server)或**域名服务器**(Domain Name Server)



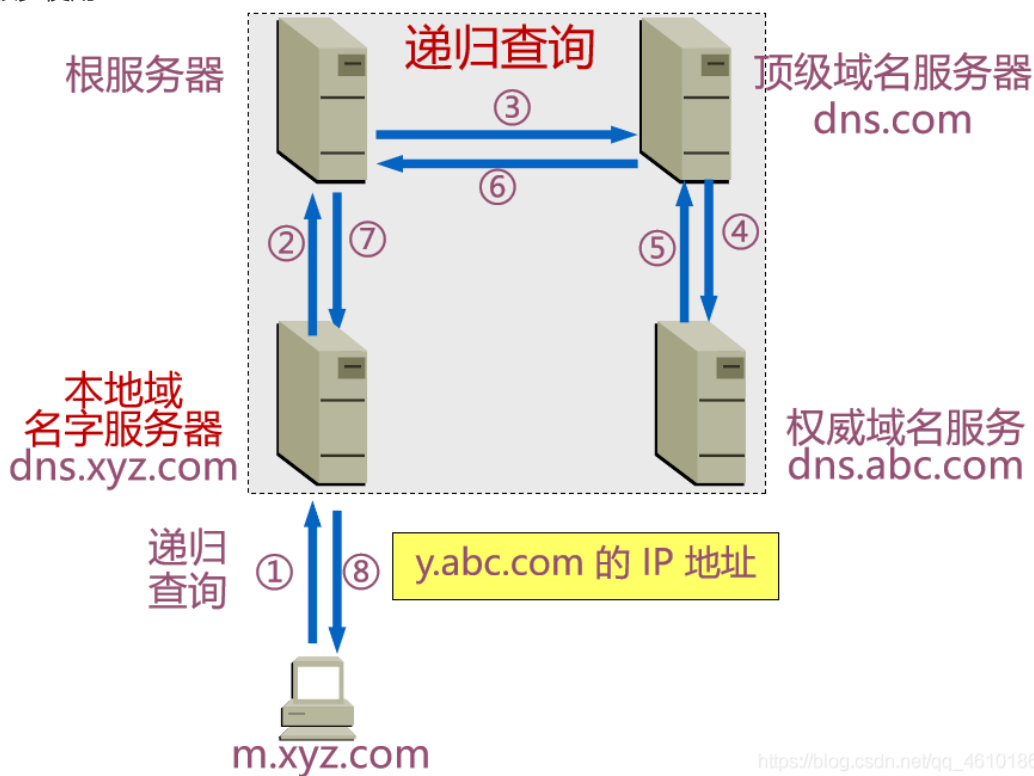
2.2 域名系统名字空间和层次结构



... .三级域名.二级域名.顶级域名

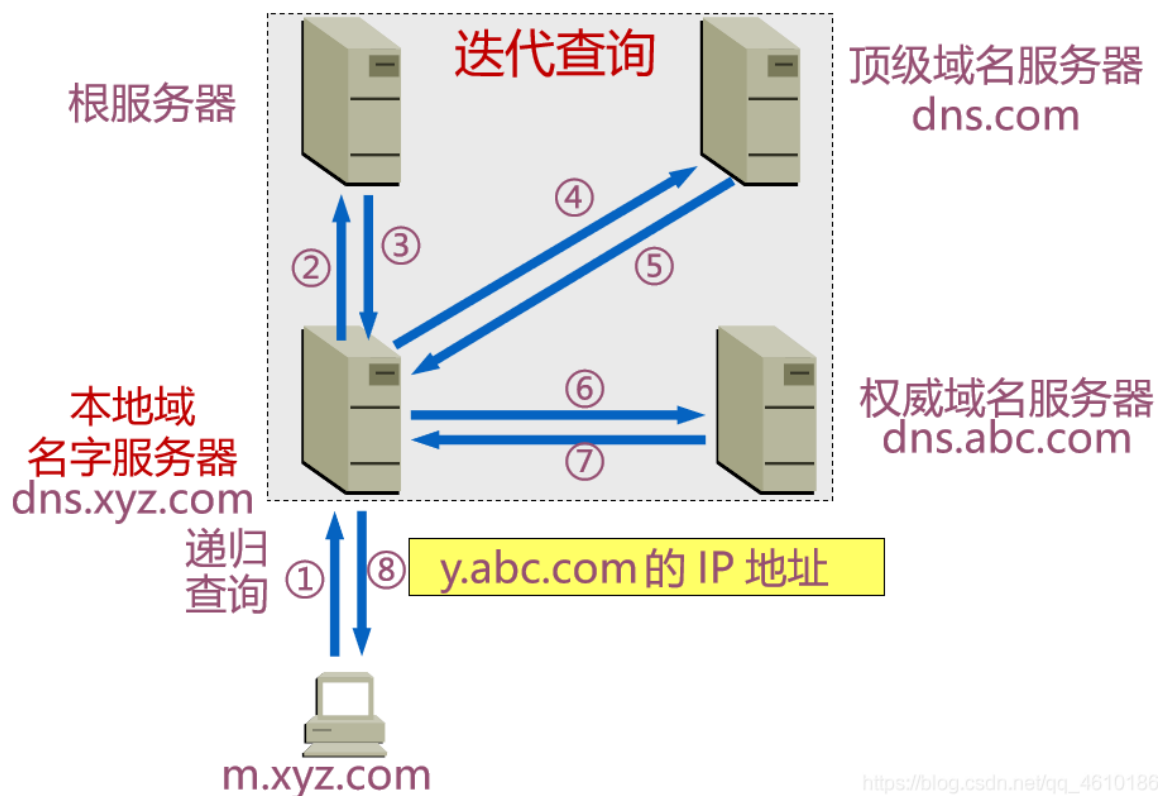
2.3 域名解析过程

- 递归查询
 - 当收到查询请求报文的域名服务器不知被查询域名的IP地址时，该域名服务器就以DNS客户的身份向下一步应查询的域名服务器发出查询请求，即替本地域名字服务器继续查询
 - 较少使用



- 迭代查询
 - 当收到查询请求报文的域名服务器不知道被查询域名的IP地址时，就把自己知道的下一步应查询的域名服务器IP地址告诉本地域名字服务器，由本地域名字服务器继续向该域名服务器查询，直到得到所要解析的域名的IP地址，或者查询不到所要解析的域名的IP地址

- 通常使用

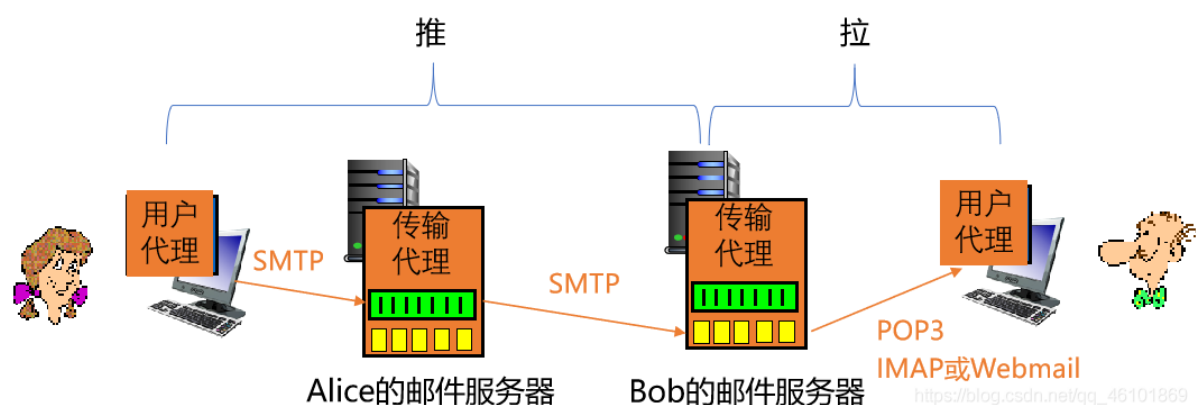


3.电子邮件

3.1 邮件发送的常用协议

简单邮件传输协议SMTP (Simple Mail Transfer Protocol) ——邮件服务器之间传递邮件使用的协议

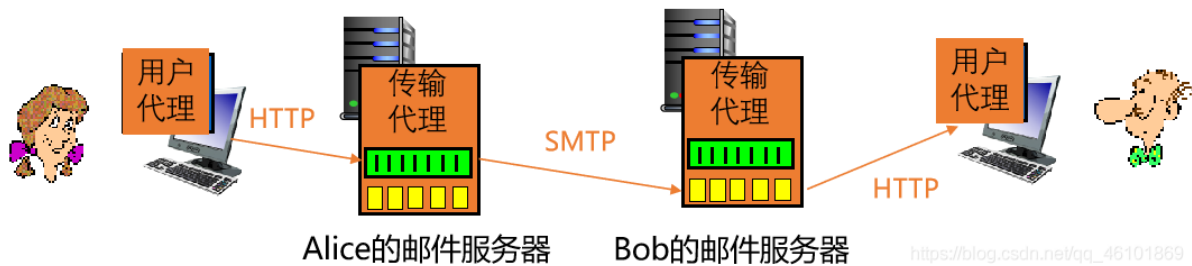
- 最终交付（邮件访问）协议::从邮件服务器的邮箱中获取邮件
 - POP3: Post Office Protocol-Version 3, 第三版邮局协议
 - IMAP: Internet Message Access Protocol, Internet邮件访问协议
 - Webmail (HTTP) : 基于Web的电子邮件



3.2 Webmail

Webmail——基于Web的电子邮件

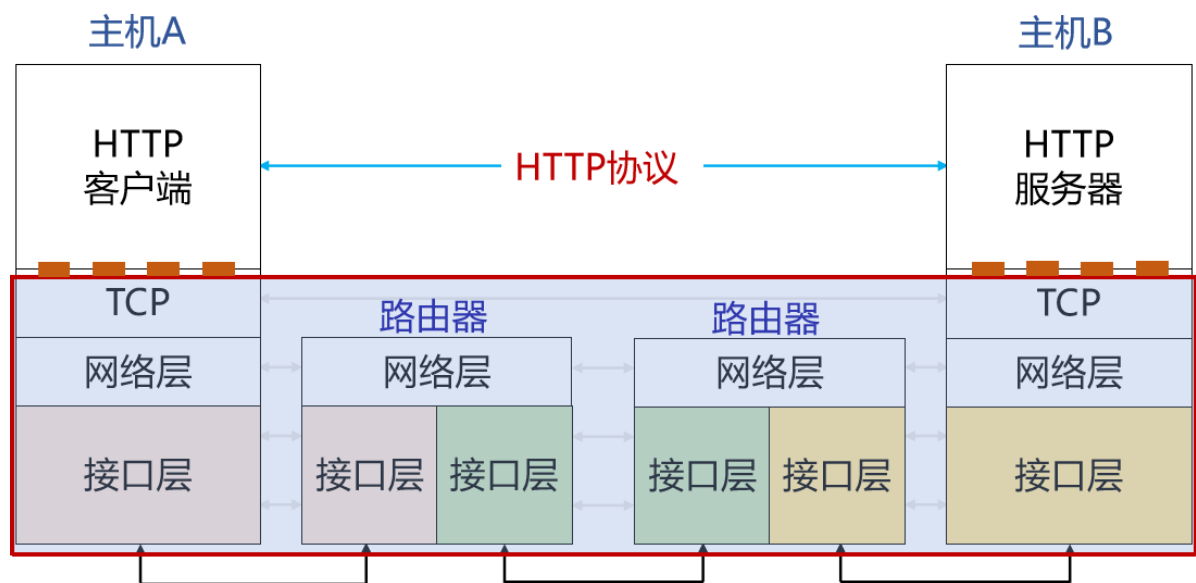
- 提供电子邮件服务的IMAP和SMTP替代方案
- 使用Web作为界面，用户代理就是普通的浏览器
- 用户及其远程邮箱之间的通信通过HTTP进行



4.WWW

4.1 WWW体系结构与协议

- WWW=World Wide Web=万维网
- HTTP服务器和客户端，以及它们之间执行的HTTP协议



服务器

- Web页面 (HTML文档)：包含多种对象或链接
- Web对象 (包括：静态对象和动态对象)：可以是 HTML文档、图像文件、视频文件、声音文件、脚本文件等
- 对象用URL (统一资源定位符) 编址：协议类型://主机名:端口//路径和文件名

客户端

- 发出请求、接收响应、解释HTML文档并显示
- 有些对象需要浏览器安装插件



统一资源定位器URLs

- 例如：<http://www.phdcomics.com:8000/comics.php>

协议类型 主机名即服务器 端口 路径和文件名

www协议

名字	用途	实例
http	超文本HTML	http://www.xaut.edu.cn/xxgk/xxjj.htm
https	安全超文本	https://www.overleaf.com/
ftp	FTP	ftp://ftp.xaut.edu.cn
file	本地文件	file:///usr/xaut/prog.c
mailto	发送邮件	mailto:xautmail@xaut.edu.cn
rtsp	流媒体	rtsp://youtube.com/montypython.mpg
sip	多媒体呼叫	sip:eve@advesary.com https://blog.csdn.net/qq_46101869

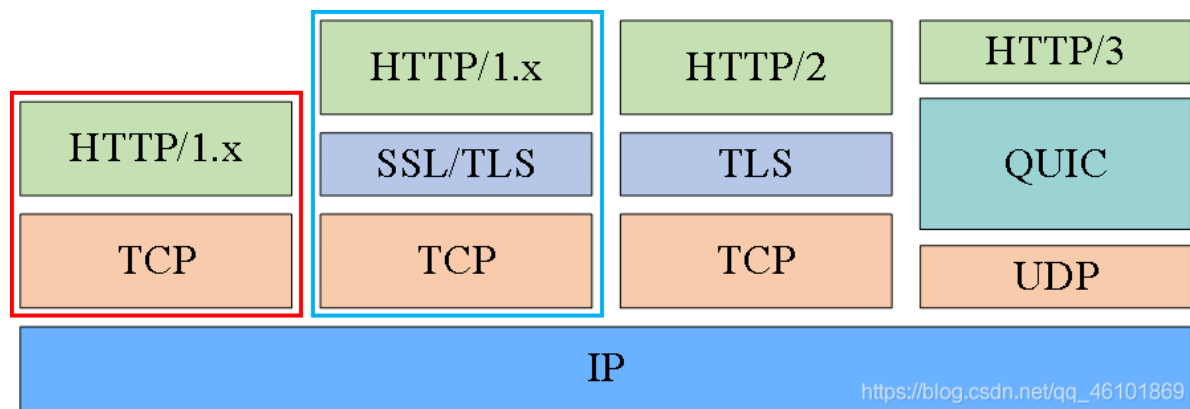
4.2 HTTP

概述

- 超文本传输协议HTTP（HyperText Transfer Protocol）在传输层通常使用TCP协议，缺省使用TCP的80端口
- HTTP为无状态协议，服务器端不保留之前请求的状态信息
 - 无状态协议：效率低、但简单
 - 有状态协议：维护状态相对复杂，需要维护历史信息，在客户端或服务器出现故障时，需要保持状态的一致性等
- HTTP标准
 - HTTP/1.0: RFC 1945（1996年）
 - HTTP/1.1: RFC 2616（1999年）
 - HTTP/2: RFC 7540（2015年）、RFC 8740（2020年）

HTTP发展现状

- HTTP/1.0（1996）
 - 无状态，非持久连接
- 与HTTP/1.1（1999）
 - 支持长连接和流水线机制
 - 缓存策略优化、部分资源请求及断点续传
- HTTPS: HTTP+TLS（2008）
 - 增加SSL/TLS（TLS 1.2）层，在TCP之上提供安全机制
- HTTP/2.0（2015、2020）
 - 目标：提高带宽利用率、降低延迟
 - 增加二进制格式、TCP多路复用、头压缩、服务端推送等功能



Web安全与隐私：Cookie

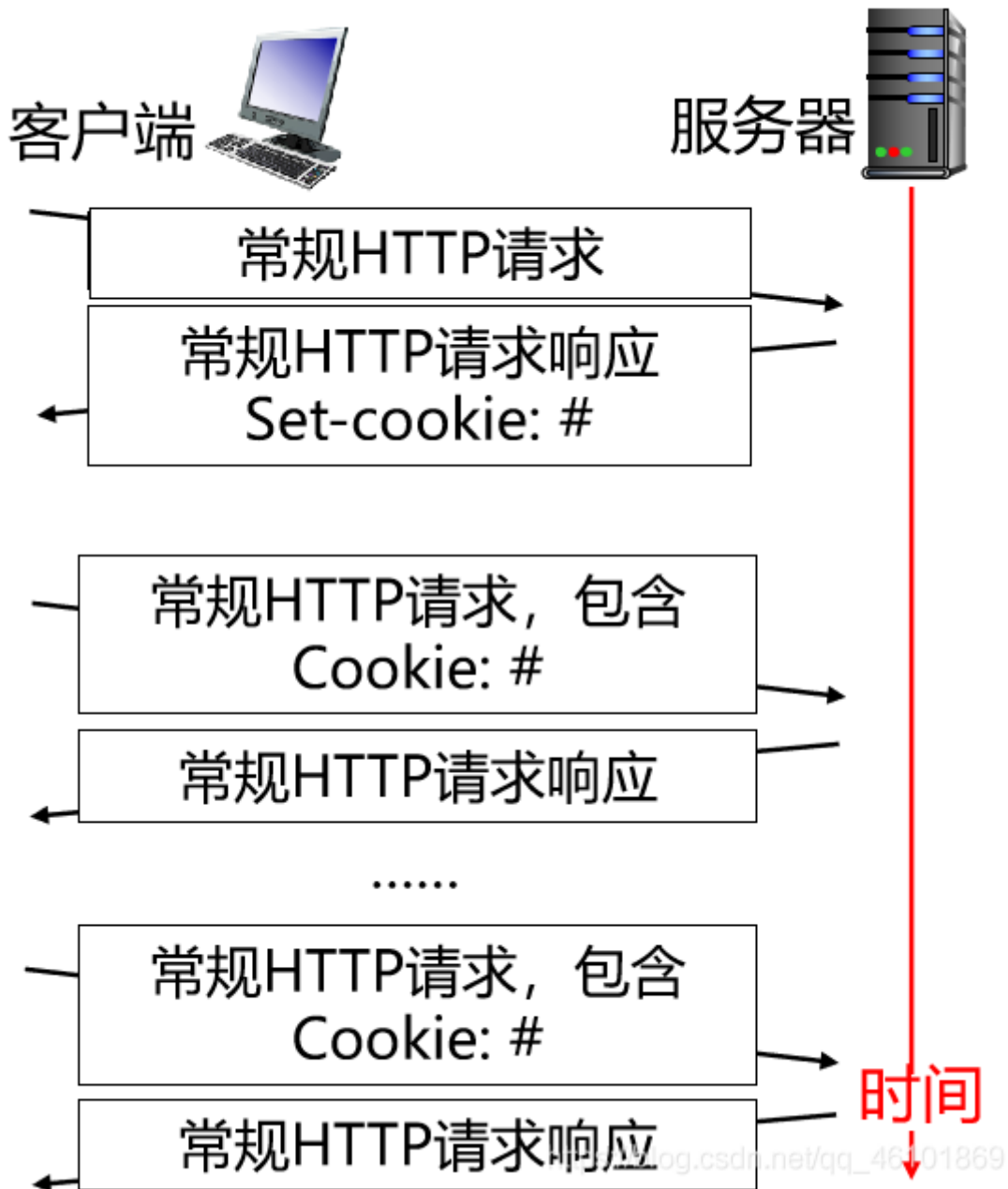
HTTP 无状态协议，服务器 用cookies保持用户状态

- HTTP在响应的首部行里使用一个关键字头set-cookie：选择的cookie号具有唯一性
- 后继的HTTP请求中使用服务器响应分配的cookie：
- Cookie文件保存在用户的主机中，内容是服务器返回的一些附加信息，由用户主机中的浏览器管理
- Web服务器建立后端数据库，记录用户信息，cookie作为关键字

例如：

Set-Cookie: SID=31d4d96e407aad42; Path=/; Domain=example.com

Cookie: SID=31d4d96e407aad42



• Cookies一般包含5个字段

- 域指明Cookie来自何方, 每个域为每个客户分配Cookie有数量限制
- 路径标明服务器的文件树中哪些部分可以使用该Cookie

- 内容采用“名字=值”的形式，是Cookie存放内容的地方，可以达到4K容量，内容只是字符串，不是可执行程序
- 安全指示浏览器只向使用安全传输连接的服务器返回Cookie

域	路径	内容	过期	安全
Toms-casino.com	/	CustomerID=297793521	15-10-1017:00	是
Jills-store.com	/	Cart=1-00501;1-07031;2-13721	11-1-1114:22	不
Aportal.com	/	Prefs=Stk:CSCO+ORCL;Spt:Jets	31-12-2023:59	不
Sneaky.com	/	UserID=4627239101	31-12-1923:59	不

- Cookie技术是把双刃剑，能分析用户喜好，向用户进行个性化推荐
 - 用Cookie在某网站标识用户信息，查找用户以前浏览网站记录
 - 用Cookie记录用户购物清单
 - 用Cookie可以保存4K内容，跟踪用户浏览网站的喜好
 - 用Cookie跨站点跟踪用户点击广告
- Cookie技术是把双刃剑，也能跟踪用户网络浏览痕迹，泄露用户隐私
 - Cookie跟踪用户以前浏览过哪些网站，跟踪用户频繁浏览哪类网站
 - Cookie收集用户信息，用户网络交互时关注的关键词
- Cookie容易嵌入间谍程序，这是个误区，Cookie文件保存的只是文本串，没有可执行程序
 - 用户可以设置浏览器限制使用Cookie

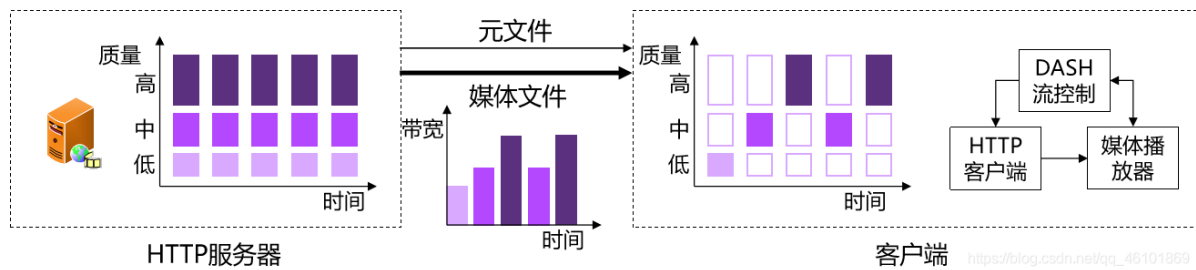
5.流媒体

5.1 概述

- 流媒体概念
 - 连续媒体（音视频）经压缩编码、数据打包后，经过网络发送给接收方
 - 接收方对数据进行重组、解码和播放
- 流媒体的特性
 - 端到端时延约束
 - 时序性约束：流媒体数据必须按照一定的顺序连续播放
 - 具有一定程度的容错性：丢失部分数据包也可完成基本功能
- 流媒体面临的挑战
 - 约束条件：网络特性（带宽有限、动态变化、延迟与抖动、丢失、异构性）
 - 目标：流媒体服务质量要素（画质、启动延迟、平滑、交互性）
 - 如何在“尽力服务”的网络传输条件下获得良好的视频质量？

5.2 流媒体动态自适应传输

- DASH (Dynamic Adaptive Streaming over HTTP)
 - 动态自适应流媒体传输协议DASH，由MPEG组织制定的标准
 - 类似协议：苹果HTTP Live Streaming (HLS)；Adobe的HTTP Dynamic Streaming (HDS)；微软的Microsoft Smooth Streaming
- DASH 基本思想
 - 完整视频被拆分为固定时长 (2s-10s) 的视频片段(segment)，每段提供不同码率
 - 视频片段与其对应的元文件 (URL) 一同存放于DASH服务器
 - 客户端基于网络条件、缓冲大小等，对每个视频片段，自适应选择合适的视频码率来下



DASH中普遍使用的自适应码率ABR (Adaptive bitrate)

6.内容分发网络CDN

6.1 概述

- 内容分发网络CDN
 - Content Delivery Network, or Content Distribution Network
- 基本思想源于MIT对Web服务瞬间拥塞问题的解决 (1998)
 - 一种Web缓存系统, **靠近网络边缘 (用户) 提供内容服务**
 - 目前提供更丰富的服务, 包括静态内容、流媒体、用户上传视频等
- 主要优点
 - 降低响应时延, 避免网络拥塞
 - 避免原始服务器过载及防止DDoS攻击
 - 分布式架构, 具有良好的可扩展性
 - 对用户透明, 无需用户感知

6.2 关键问题

怎样将内容 (如从百万的视频中选定的内容) 分发给同时发起访问的数百万用户?

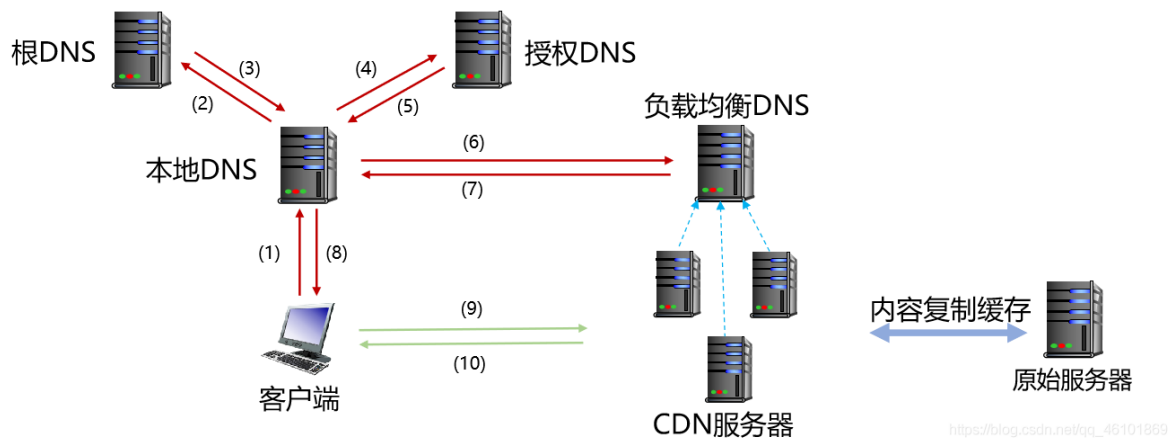
6.3 机理与解决方式

DNS重定向实现CDN

- 将请求调度到较近或负载较轻的CDN服务器
- **HTTP重定向请求内容**, 服务提供者返回清单CDN
 - 原始服务器决策CDN服务器
 - HTTP响应: 状态码30X, Location: 指明新的位置

DNS辅助实现CDN

- 负载均衡DNS负责决策CDN服务器选择
- 负载均衡DNS需要收集CDN服务器的位置和负载情况
- 如果找不到被请求的对象, 需要从原始服务器获取



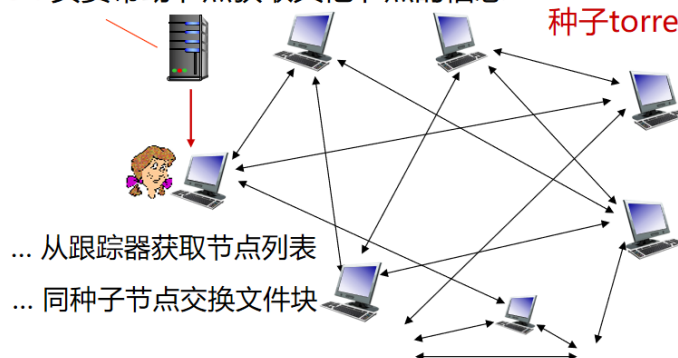
7.P2P网络

P2P文件分发协议：BitTorrent

- 文件被划分为256Kb大小的块
- 具有种子(torrents)的节点发送或接收文件

跟踪器tracker: 负责帮助节点获取其他节点的信息

种子torrent: 交换文件块的节点

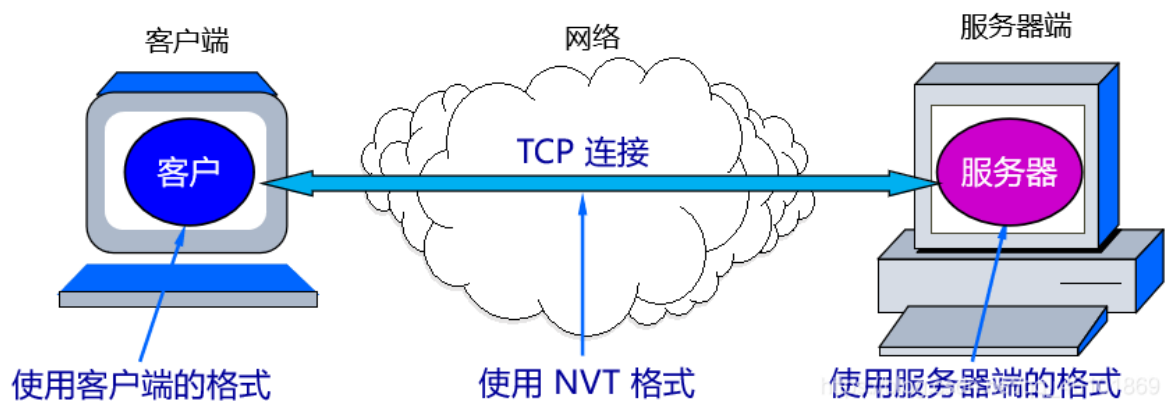


8.远程登录Telnet

- Telnet协议引入网络**虚拟终端NVT** (Network Virtual Terminal)，使用一种专门的键盘定义来屏蔽不同计算机系统对键盘输入的差异性，同时定义客户进程与远程服务器进程之间的交互过程
- NVT是Telnet协议定义的一组**通用字符集**，通过这种统一的数据表示方式，来保证不同硬件、软件与数据格式的终端与主机之间通信的兼容性
- 本地终端输入的字符首先由本地Telnet客户进程转换为NVT格式，通过网络将NVT格式的字符传输到远程主机，远程Telnet服务器进程再将NVT格式的字符转换为远程主机能够识别和处理的字符格式
- 使用Telnet协议在网络中传输的数据都是NVT格式，不同的用户终端与服务器进程均与本地终端格式无关

Telnet的工作过程

- 本地Telnet客户进程与远程主机上的Telnet服务器进程建立TCP连接
- 将本地终端上输入的用户名和口令及以后输入的任何命令或字符以网络虚拟终端NVT格式传输给远程主机
- 将远程主机输出的NVT格式的数据转化为本地所接受的格式送回本地终端，包括输入命令回显和命令执行结果
- 本地终端对远程主机撤销连接，从而结束 Telnet远程登录过程



9.FTP

9.1 概述

文件传输协议FTP(File Transfer Protocol) 是Internet上使用最广泛的应用层协议之一

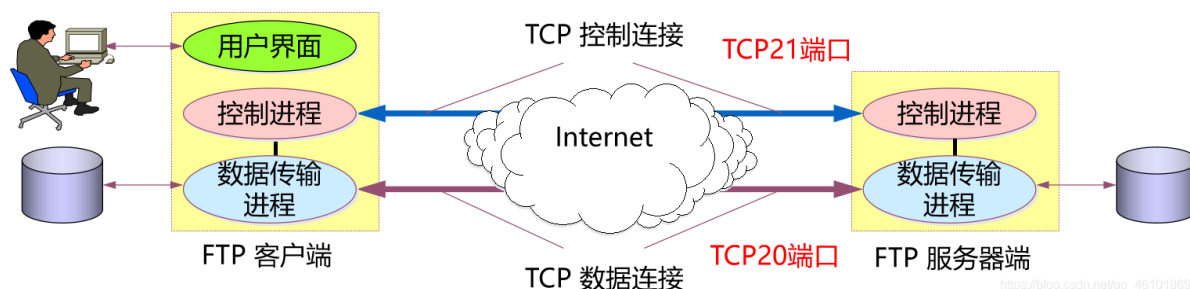
- FTP提供交互式的访问，允许用户指明文件的类型与格式，并允许文件具有存取权限
- FTP屏蔽了各计算机系统的细节，适用于在异构网络中任意计算机之间传送文件
- RFC 959早在1985年就已经成为Internet的正式标准
- FTP使用C/S方式实现

9.2 工作过程

- 服务器主进程打开TCP21端口，等待客户进程发出的连接请求
- 客户可以用分配的任意一个本地端口号与服务器进程的TCP21端口进行连接
- 客户请求到来时，服务器主进程启动从属进程来处理客户进程发来的请求
- 服务器从属进程对客户进程的请求处理完毕后即终止，但从属进程在运行期间根据需要还可能创建其他一些子进程
- 服务器主进程返回，继续等待接收其他客户进程发来的连接请求，服务器主进程与从属进程并行工作

FTP的两个端口与两个连接

- **控制连接**在整个会话期间一直保持，客户进程发出的文件传输请求通过控制连接发送给**服务器控制进程（工作在TCP21端口）**，但控制连接不用来传输文件
- 服务器控制进程在接收到客户进程发送来的文件传输请求后就创建**数据传输进程（工作在TCP20端口）**和**数据连接**
- 数据连接用来连接客户进程和服务器数据传输进程，实际完成文件的传输。服务器数据传输进程在文件传输完毕后关闭数据连接并结束运行



写在最后

这篇知识汇总我写了好久，因为ppt的缘故，知识整理需要自己消化一遍然后挑选重要的部分整理摘录下来，同时对于某些知识点的组织结构做了相应调整，对于不太清楚的地方，我也去查阅了相关资料做了补充。但需要注意的是有些知识点并不完整，该知识点整理也只是根据我们当时考试的考点来整理的，所以不足之处请见谅。

**愿我们以梦为马，不负人生韶华！
与君共勉**